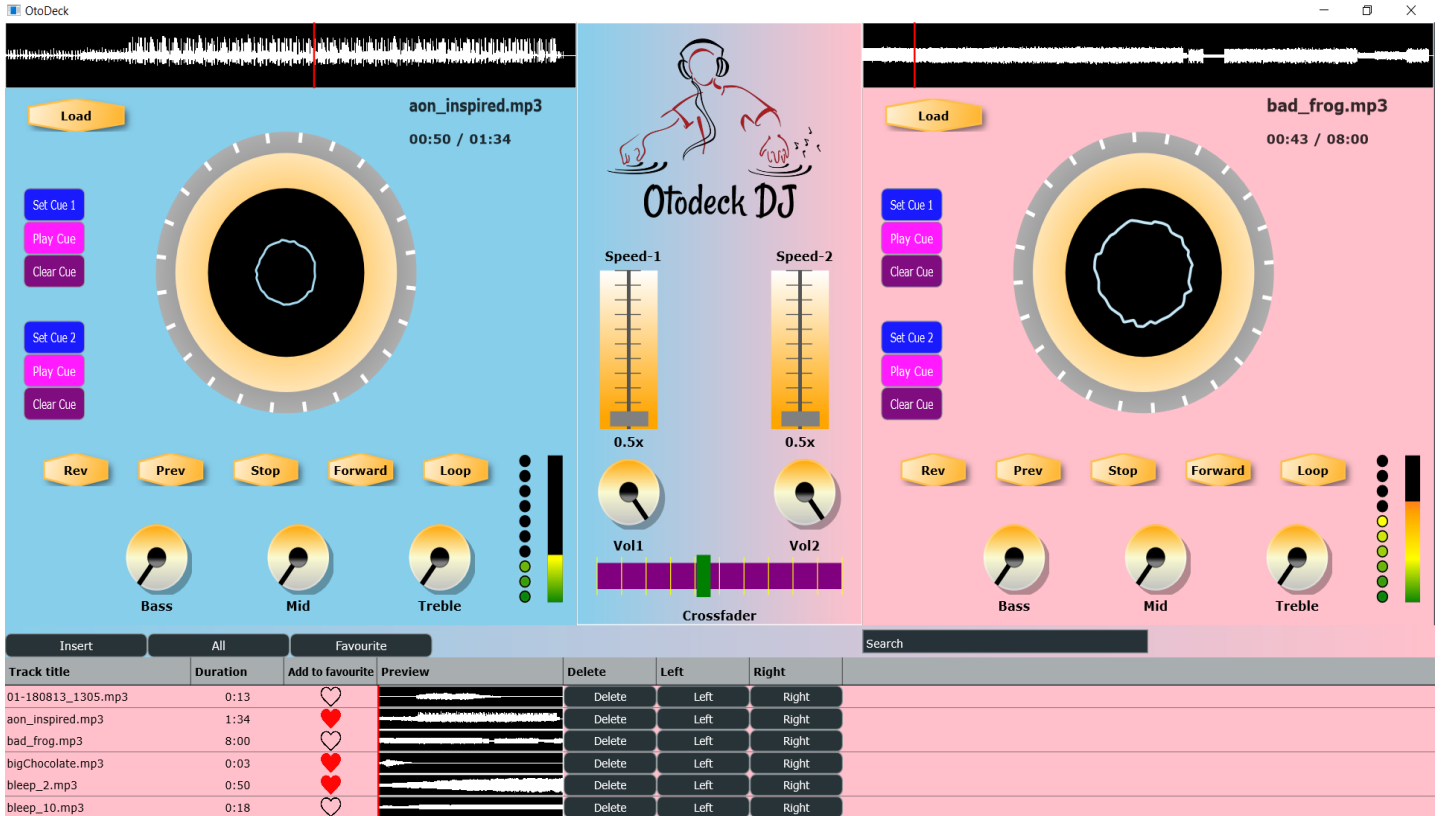


OOP-2005: Final Project Report



R1: All the basic functionality

R1A: Load audio files into audio players

i) Loading audio files through load button

1. In deckGUI constructor:

The first step to loading audio files via the load button starts in the DeckGUI constructor. Here's how it works:

- The loadButton is created and made visible by calling addAndMakeVisible(), which adds it to the graphical user interface (GUI).
- A listener is attached to the loadButton using loadButton.addListener(this);. This sets up an event handler that listens for user interaction—when the button is clicked, the listener will trigger the appropriate function.
- The button is labelled with "Load" using loadButton.setText("Load");, and a custom look-and-feel is applied for a unique appearance via loadButton.setLookAndFeel(&customLookAndFeel);.

```
// Initialize and set up the load button
addAndMakeVisible(loadButton);
loadButton.addListener(this);
loadButton.setText("Load");
loadButton.setLookAndFeel(&customLookAndFeel);
```

2. In Resized Function (deckGUI.cpp):

The resized() function is responsible for ensuring the correct positioning and size of the loadButton. As the window or interface resizes, this function recalculates the dimensions and location of the button.

- The button is positioned on the UI using the setBounds() method, which specifies its location and size. In this case, the loadButton is placed on the left side with calculated width and height to fit the layout.

```
loadButton.setBounds(0, rowH * 2 + 10, getWidth() / 4, rowH);
```

3. In ButtonClicked Function:

When the user clicks the loadButton, the buttonClicked() method is triggered. The following steps occur:

- A FileChooser is opened to allow the user to select an audio file from their system.

- After the user selects a file, the system checks whether the file is already in the playlist by comparing it with the existing entries. This is done to avoid duplicates.
- If the file is a duplicate, an alert box warns the user that the track won't be added again.
- If it's a new file, it is loaded into the DJAudioPlayer, the waveform display is updated to show the audio's visual representation, the track name is displayed, and the track is added to the playlist along with its duration.

```

        if (button == &loadButton)
        {
            auto fileChooserFlags =
                FileBrowserComponent::canSelectFiles;

            fChooser.launchAsync(fileChooserFlags, [this](const
FileChooser& chooser)
            {
                File chosenFile = chooser.getResult();
                if (chosenFile.existsAsFile())
                {
                    URL audioURL = URL{ chosenFile };
                    deckCleared = false;

                    // Check for duplicates in the playlist
                    auto trackEntries = playlistComponent-
>getTrackEntries();
                    bool isDuplicate =
std::any_of(trackEntries->begin(), trackEntries->end(),
                [&audioURL](const TrackEntry&
entry) { return entry.audioURL == audioURL; });

                    if (isDuplicate)
                    {
juce::AlertWindow::showMessageBoxAsync(juce::AlertWindow::W
arningIcon,

                                "Duplicate Track",
                                "The track '" +
audioURL.getFileName() + "' is already in the playlist and
will not be added again.",
                                "OK");
                    }
                    else
                    {
                        std::cout << "Loading file: " <<
chosenFile.getFullPathName() << std::endl;
                        player->loadURL(audioURL);
                        waveFormDisplay.loadURL(audioURL);

                        trackName.setText(audioURL.getFileName(),
dontSendNotification);

                        playlistComponent->addToTrackEntries(audioURL, player->
getTrackDuration());
                    }
                }
            });
        }
    }

```

4. In DJAudioPalyer:

The DJAudioPlayer::loadURL() method is responsible for loading the selected audio file into the player for playback. Here's how it works:

- The method creates an AudioFormatReader for the selected audio file by using the file's URL.
- If the reader is successfully created, the file is set as the source for the transportSource, which handles the audio playback.
- The loaded file is then stored in the loadedURL for future reference.

```
// loadURL: Loads an audio file from the provided URL into the
// player.
// This method creates an AudioFormatReader for the file and
// sets it as the source for playback.
void DJAudioPlayer::loadURL(URL audioURL)
{
    auto* reader =
formatManager.createReaderFor(audioURL.createInputStream(false
));
    if (reader == nullptr) {
        DBG("Failed to create reader for the provided URL.");
        return;
    }
    std::unique_ptr<AudioFormatReaderSource> newSource(new
AudioFormatReaderSource(reader, true));
    transportSource.setSource(newSource.get(), 0, nullptr,
reader->sampleRate);
    readerSource.reset(newSource.release());
    loadedURL = audioURL;
}
```

ii) Loading audio files through drag and drop

1. In DeckGUI:

This method enables users to drag and drop audio files directly onto the deck. The filesDropped() function is triggered when a file is dropped onto the application. Here's how it works:

- When a file is dropped, the method checks if only one file has been dropped.
- The dropped file is then loaded into the DJAudioPlayer using the loadURL() function.
- The waveform display is updated with the visual representation of the audio file, the track name is displayed, and the track is added to the playlist just like when loading through the button.

```
void DeckGUI::filesDropped(const juce::StringArray& files, int
x, int y)
{
    std::cout << "DeckGUI::filesDropped" << std::endl;
    if (files.size() == 1)
    {
```

```

        juce::File droppedFile{ files[0] };
        juce::URL fileURL{ droppedFile };
        player->loadURL(fileURL);
        waveformDisplay.loadURL(fileURL);
        trackName.setText(droppedFile.getFileName(),
juce::dontSendNotification);
        playlistComponent->addToTrackEntries(fileURL, player-
>getTrackDuration());
    }
}

```

2. In DJAudioPlayer

The same loadURL() function is used to load audio files for both drag-and-drop and load button functionality. This method:

- Uses the URL of the dropped file to create an AudioFormatReader.
- The reader is used to set the file as the source for playback via the transportSource, ensuring smooth playback.

```

// loadURL: Loads an audio file from the provided URL into the
// player.
// This method creates an AudioFormatReader for the file and
// sets it as the source for playback.
void DJAudioPlayer::loadURL(URL audioURL)
{
    auto* reader =
formatManager.createReaderFor(audioURL.createInputStream(false
));
    if (reader == nullptr) {
        DBG("Failed to create reader for the provided URL.");
        return;
    }
    std::unique_ptr<AudioFormatReaderSource> newSource(new
AudioFormatReaderSource(reader, true));
    transportSource.setSource(newSource.get(), 0, nullptr,
reader->sampleRate);
    readerSource.reset(newSource.release());
    loadedURL = audioURL;
}

```

R1B: Play two or more tracks

1. In MainComponent

- The MainComponent is derived from juce::AudioAppComponent, which provides core functionality for managing audio processing and GUI.
- Two instances of DJAudioPlayer (player1 and player2) are used for audio playback. These players are connected to DeckGUI instances (deckGUI1 and deckGUI2) that provide the graphical interface for controlling playback, volume, speed, etc.

- A MixerAudioSource is used to mix the audio from both decks before it's output to the speakers or audio device.
- The PlaylistComponent manages the track entries and communicates with both decks to load, play, and manipulate tracks.
- prepareToPlay() is called to prepare both players for audio playback by setting up sample rate and block size.
- getNextAudioBlock() continuously fills the audio buffer with audio samples from both decks, mixed by mixerSource, allowing simultaneous playback.

```
class MainComponent : public juce::AudioAppComponent
{
public:
    // Constructor - sets up the main component.
    MainComponent();

    // Destructor - cleans up any resources when the component is
    // destroyed.
    ~MainComponent() override;

    // Prepares the audio callback, typically called before playback
    // starts.
    void prepareToPlay(int samplesPerBlockExpected, double
    sampleRate) override;

    // Called repeatedly to process audio blocks.
    void getNextAudioBlock(const juce::AudioSourceChannelInfo&
    bufferToFill) override;

    // Releases resources when the audio processing stops.
    void releaseResources() override;

    // Paints the component, handling any custom GUI drawing.
    void paint(juce::Graphics& g) override;

    // Called when the component is resized, updates the layout of
    // child components.
    void resized() override;

private:
    // AudioFormatManager is responsible for managing the available
    // audio formats.
    AudioFormatManager formatManager;

    // AudioThumbnailCache is used to cache waveform thumbnails for
    // faster drawing.
    AudioThumbnailCache thumbCache{ 100 };

    // PlaylistComponent manages the list of tracks and interaction
    // with the DeckGUIs.
    // It references both deckGUI1 and deckGUI2 for controlling audio
    // playback.
    PlaylistComponent playlistComponent{ &deckGUI1, &deckGUI2 };

    // Two DJAudioPlayer instances for managing audio playback logic.
    DJAudioPlayer player1{ formatManager };
    DJAudioPlayer player2{ formatManager };

    // Two DeckGUI instances that provide the GUI for interacting
    // with the DJAudioPlayers.
```

```

    DeckGUI deckGUI1{ &player1, formatManager, thumbCache,
    &playlistComponent, juce::Colours::skyblue };
    DeckGUI deckGUI2{ &player2, formatManager, thumbCache,
    &playlistComponent, juce::Colours::pink };

    // MiddleDeck is a GUI component that manages crossfading between
    player1 and player2.
    MiddleDeck middleDeck{ &player1, &player2 };

    // MixerAudioSource is used to mix the audio output from both
    DJAudioPlayers.
    MixerAudioSource mixerSource;

    // This macro helps to prevent copying and memory leaks in JUCE
    components.
    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(MainComponent)
};

```

2. In DJAudioPlayer.cpp

- prepareToPlay() prepares the player for playback by setting up the buffer and sample rate.
- getNextAudioBlock() is called continuously during playback to fill the audio buffer with samples.
- The transportSource is used to manage audio streaming, and resampleSource handles sample rate conversion if needed.
- The start(), stop(), and isPlaying() methods control playback.

```

DJAudioPlayer::DJAudioPlayer(AudioFormatManager& _formatManager)
    : formatManager(_formatManager), resampleSource(&transportSource,
false)
{
    //Other code
}

// Destructor: Cleans up any resources used by the DJAudioPlayer.
DJAudioPlayer::~DJAudioPlayer()
{
}

// prepareToPlay: Prepares the audio player for playback by
initializing buffers and sources.
// This method is called before playback starts to ensure everything
is set up correctly.
void DJAudioPlayer::prepareToPlay(int samplesPerBlockExpected, double
sampleRate)
{
    transportSource.prepareToPlay(samplesPerBlockExpected,
sampleRate);
    resampleSource.prepareToPlay(samplesPerBlockExpected,
sampleRate);
}

// getNextAudioBlock: Retrieves the next block of audio samples and
processes it through the EQProcessor.
// This method is called continuously during playback to provide
audio data to the output device.
void DJAudioPlayer::getNextAudioBlock(const
juce::AudioSourceChannelInfo& bufferToFill)
{
}

```

```

        resampleSource.getNextAudioBlock(bufferToFill); // Get the audio
        block from the resampler

        // Calculate the RMS level for the left channel
        auto* channelData = bufferToFill.buffer->getReadPointer(0,
        bufferToFill.startSample);

    }

    // releaseResources: Releases resources allocated during
    prepareToPlay.
    // This method is called when playback stops or when the audio source
    is no longer needed.
    void DJAudioPlayer::releaseResources()
    {
        transportSource.releaseResources();
        resampleSource.releaseResources();
    }

```

3. PlaylistComponent.cpp

- The constructor initializes the playlist component, linking both deckGUI1 and deckGUI2.
- The formatManager is responsible for registering and handling different audio formats (e.g., MP3, WAV), allowing flexibility in file types.

```

PlaylistComponent::PlaylistComponent(DeckGUI* _deckGUI1,
    DeckGUI* _deckGUI2)

    // Initialising member variables in constructor
    : trackFound(false), // Boolean to indicate if track is found
    during search
    deckGUI1(_deckGUI1), // Deck 1 object (pass pointer from GUI)
    deckGUI2(_deckGUI2), // Deck 2 object (pass pointer from GUI)
    filterManager(trackEntries) // Instantiate PlaylistFilterManager
    to manage track filtering
    {
        // Register audio formats (e.g. .mp3, .wav) with format manager
        to handle various file types
        formatManager.registerBasicFormats();
    }

```

4. In DeckGUI.cpp

- The constructor sets up the deck's interface, connecting it to the audio player (DJAudioPlayer), waveform display, and playlist component.
- The playButton is set up with an event listener. When clicked, it calls the togglePlay() method to control playback.

```

DeckGUI::DeckGUI(DJAudioPlayer* _player,
    AudioFormatManager& formatManagerToUse,
    AudioThumbnailCache& cacheToUse,
    PlaylistComponent* _playlistComponent,
    juce::Colour backgroundColor)

    : player(_player),
    waveformDisplay(formatManagerToUse, cacheToUse),
    playlistComponent(_playlistComponent),
    trackEntries(playlistComponent->getTrackEntries()),
    backgroundColour(backgroundColor)

```



```

{
    // Initialize and set up the play/stop button
    addAndMakeVisible(playButton);
    playButton.addListener(this);
    playButton.setButtonText("Play");
    playButton.setLookAndFeel(&customLookAndFeel);
    playButton.onClick = [this] { togglePlay(); }; // Set the action
    to toggle play/stop
}

```

5. In DeckGUI.cpp

- If the player is currently playing, the togglePlay() method stops the playback and updates the button text to "Play."
- If the player is not playing, it starts playback and changes the button text to "Stop."

```

// togglePlay: Toggles between play and stop states.
void DeckGUI::togglePlay()
{
    if (player->isPlaying()) {
        player->stop(); // Stop playback
        playButton.setButtonText("Play"); // Update button text to
        "Play"
    }
    else {
        player->start(); // Start playback
        playButton.setButtonText("Stop"); // Update button text to
        "Stop"
    }
}
6.

```

7. In DJAudioPlayer

- start() begins playback by calling transportSource.start().
- stop() stops playback by calling transportSource.stop().
- isPlaying() checks whether the audio is currently playing by querying the state of the transportSource.

```

// start: Starts audio playback.
void DJAudioPlayer::start()
{
    transportSource.start();
}

// stop: Stops audio playback.
void DJAudioPlayer::stop()
{
    transportSource.stop();
}

// isPlaying: Checks if the audio is currently playing.
// @return True if playing, false otherwise.
bool DJAudioPlayer::isPlaying()
{
    return transportSource.isPlaying();
}

```

R1C: Mix the tracks by varying each of their volumes

i) Setting volume

1. In the constructor of the MiddleDeck

- Two sliders (vol1 and vol2) are created for adjusting the volume of player1 and player2.
- The sliders are customized using a CustomLookAndFeel to apply a specific visual style.
- The sliders' range is set between 0.1 and 1.0 (representing 10% to 100% volume).
- The sliders are set to use a rotary style, and their initial values are set to full volume (1.0).
- Labels are created and customized to display "Vol1" and "Vol2" below the corresponding sliders.
- Listeners are added to the sliders so that when their values change, the corresponding player's volume is adjusted.

```
MiddleDeck::MiddleDeck(DJAudioPlayer* _player1, DJAudioPlayer*
_player2)
: player1(_player1),
  player2(_player2)
{

    // Initialize the custom LookAndFeel and apply it to the sliders
    customLookAndFeel = std::make_unique<CustomLookAndFeel>();

    vol1.setLookAndFeel(customLookAndFeel.get());
    vol2.setLookAndFeel(customLookAndFeel.get());

    // Set up slider ranges and initial values
    vol1.setRange(0.1, 1.0);
    vol1.setValue(1.0);
    vol2.setRange(0.1, 1.0);
    vol2.setValue(1.0);

    // Set slider styles (no text box, rotary for volume, linear
vertical for speed)
    vol1.setTextBoxStyle(Slider::NoTextBox, true, 0, 0);
    vol2.setTextBoxStyle(Slider::NoTextBox, true, 0, 0);
    vol1.setSliderStyle(Slider::Rotary);
    vol2.setSliderStyle(Slider::Rotary);

    // Make sliders visible
    addAndMakeVisible(vol1);
    addAndMakeVisible(vol2);

    // Initialize and style labels
    vol1Label.setText("Vol1", dontSendNotification);
    vol2Label.setText("Vol2", dontSendNotification);

    //positioning center
    vol1Label.setJustificationType(juce::Justification::centred);
    vol2Label.setJustificationType(juce::Justification::centred);

    //Set font style
```

```

        vol1Label.setFont(juce::Font(16.0f, juce::Font::bold));
        vol1Label.setColour(juce::Label::textColourId,
juce::Colours::black);

        vol2Label.setFont(juce::Font(16.0f, juce::Font::bold));
        vol2Label.setColour(juce::Label::textColourId,
juce::Colours::black);

        // Make labels visible
        addAndMakeVisible(vol1Label);
        addAndMakeVisible(vol2Label);

        // Add listeners to sliders to handle value changes
        vol1.addListener(this);
        vol2.addListener(this);
    }

```

2. In the destructor of MiddleDeck

- When MiddleDeck is destroyed, the custom look and feel applied to the sliders is removed to prevent memory leaks or visual glitches.

```

MiddleDeck::~MiddleDeck()
{
    vol1.setLookAndFeel(nullptr);
    vol2.setLookAndFeel(nullptr);
}

```

3. In resized function

- Every time the component is resized, this function recalculates the positions and sizes of the sliders and labels, ensuring they are correctly aligned within the MiddleDeck.

```

void MiddleDeck::resized()
{
    int halfWidth = getWidth() / 3;
    int smallHeight = getHeight() / 7;

    // Declare label height and Y offset
    int labelHeight = 20;
    int labelYOffset = 5;

    // Set the bounds for the sliders
    vol1.setBounds(5, smallHeight * 5 + labelYOffset, halfWidth,
smallHeight - 10);
    vol2.setBounds((halfWidth * 2) - 10, smallHeight * 5 +
labelYOffset, halfWidth, smallHeight - 10);

    // Set the bounds for the labels
    vol1Label.setBounds(5, smallHeight * 6, halfWidth, labelHeight);
    vol2Label.setBounds((halfWidth * 2) - 10, smallHeight * 6,
halfWidth, labelHeight);
}

```

4. In sliderValueChanged(Slider* slider) function

- When the user adjusts either vol1 or vol2, this function is triggered, and the gain of the respective player is updated in real time.

```

void MiddleDeck::sliderValueChanged(Slider* slider)
{
    if (//)
    {
        // Adjust the mix ratio for both players based on the
        crossfader's value
    }
    else if (slider == &vol1)
    {
        // Set the gain for player 1 based on the vol1 slider's value
        player1->setGain(slider->getValue());
    }
    else if (slider == &vol2)
    {
        // Set the gain for player 2 based on the vol2 slider's value
        player2->setGain(slider->getValue());
    }
}

```

5. In DJAudioPlayer

- This method is called when the volume slider changes. It ensures that the gain is properly bounded between 0.0 and 1.0, preventing excessive volume changes.

```

// setGain: Sets the gain (volume) of the audio playback.
// @param gain The gain value, should be between 0 and 1.
void DJAudioPlayer::setGain(double gain)
{
    if (gain < 0.0f) {
        transportSource.setGain(0.0f);
    }
    else if (gain > 1.0f) {
        transportSource.setGain(1.0f);
    }
    else {
        transportSource.setGain(gain);
    }
}

```

ii) Mixing sounds

1. In MainComponent.h

- The mixerSource takes the output of player1 and player2 and mixes them together into a single audio output stream.

```

// MixerAudioSource is used to mix the audio output
from both DJAudioPlayers.
MixerAudioSource mixerSource;

```

2. In MainComponent.cpp

- The prepareToPlay(), getNextAudioBlock(), and releaseResources() methods manage the audio mixing and playback process.

- `prepareToPlay()`: This method is called before audio playback begins to set up the mixer and players.
- `getNextAudioBlock()`: Continuously retrieves audio data from both players, mixes them, and sends the combined audio to the output.
- `releaseResources()`: Releases any resources used during playback, ensuring no memory leaks.

```
// Called before playback starts to prepare the
// audio callback.
void MainComponent::prepareToPlay(int
samplesPerBlockExpected, double sampleRate)
{
    // Prepare the mixer source and add both
    // players as input sources.

    mixerSource.prepareToPlay(samplesPerBlockExpected
, sampleRate);
    mixerSource.addInputSource(&player1, false);
    mixerSource.addInputSource(&player2, false);
}

// Called repeatedly to process audio blocks.
void MainComponent::getNextAudioBlock(const
juce::AudioSourceChannelInfo& bufferToFill)
{
    mixerSource.getNextAudioBlock(bufferToFill);
    // Mix the audio from both players into the
    // output buffer.
}

// Called when playback stops to release audio
// resources.
void MainComponent::releaseResources()
{
    mixerSource.releaseResources();
}
```

R1D: Speed up and slow down the tracks

1. In the constructor of the `middleDeck`
 - Custom Look and Feel: The sliders for speed control are styled using a `CustomLookAndFeel` object for a consistent visual design across the interface.
 - Slider Setup: Two sliders (`speed1` for `player1` and `speed2` for `player2`) are created to control the playback speed. The sliders range from 0.1 to 10.0, representing speeds from 10% to 1000% of the normal speed.
 - Labels: Each slider has corresponding labels (`speed1Label` and `speed2Label`) to display which slider controls which player. Value labels (`speed1ValueLabel` and `speed2ValueLabel`) are also created to show the current speed multiplier (e.g., 1x for normal speed).
 - Listeners: Event listeners are added to the sliders, so when the slider values are adjusted, the corresponding player's speed is updated in real-time.

- When MiddleDeck is initialized, the speed control sliders and labels are set up and displayed. The sliders allow real-time adjustments to the speed of the audio tracks.
- Event listeners on the sliders will trigger actions when the user adjusts the speed, updating both the playback speed and the displayed speed value.

```

MiddleDeck::MiddleDeck(DJAudioPlayer* _player1, DJAudioPlayer*
_player2)
: player1(_player1),
  player2(_player2)
{
    // Initialize the custom LookAndFeel and apply it to the sliders
    customLookAndFeel = std::make_unique<CustomLookAndFeel>();

    speed1.setLookAndFeel(customLookAndFeel.get());
    speed2.setLookAndFeel(customLookAndFeel.get());

    // Set up slider ranges and initial values
    speed1.setRange(0.1, 10.0);
    speed1.setValue(0.5);
    speed2.setRange(0.1, 10.0);
    speed2.setValue(0.5);

    // Set slider styles (no text box, rotary for volume, linear
    vertical for speed)
    speed1.setTextBoxStyle(Slider::NoTextBox, true, 0, 0);
    speed2.setTextBoxStyle(Slider::NoTextBox, true, 0, 0);

    speed1.setSliderStyle(Slider::LinearVertical);
    speed2.setSliderStyle(Slider::LinearVertical);

    // Make sliders visible
    addAndMakeVisible(speed1);
    addAndMakeVisible(speed2);

    // Initialize and style labels
    speed1Label.setText("Speed-1", dontSendNotification);
    speed2Label.setText("Speed-2", dontSendNotification);

    //positioning center
    speed1Label.setJustificationType(juce::Justification::centred);
    speed2Label.setJustificationType(juce::Justification::centred);

    //Set font style

    vol2Label.setFont(juce::Font(16.0f, juce::Font::bold));
    vol2Label.setColour(juce::Label::textColourId,
juce::Colours::black);

    speed1Label.setFont(juce::Font(16.0f, juce::Font::bold));
    speed1Label.setColour(juce::Label::textColourId,
juce::Colours::black);

    speed2Label.setFont(juce::Font(16.0f, juce::Font::bold));
    speed2Label.setColour(juce::Label::textColourId,
juce::Colours::black);

    speed1ValueLabel.setJustificationType(juce::Justification::centred);
    speed1ValueLabel.setFont(juce::Font(16.0f, juce::Font::bold));

```

```

        speed1ValueLabel.setColour(juce::Label::textColourId,
juce::Colours::black);

speed2ValueLabel.setJustificationType(juce::Justification::centred);
        speed2ValueLabel.setFont(juce::Font(16.0f, juce::Font::bold));
        speed2ValueLabel.setColour(juce::Label::textColourId,
juce::Colours::black);

        // Make labels visible
        addAndMakeVisible(speed1Label);
        addAndMakeVisible(speed2Label);
        addAndMakeVisible(speed1ValueLabel);
        addAndMakeVisible(speed2ValueLabel);

        // Add listeners to sliders to handle value changes
        speed1.addListener(this);
        speed2.addListener(this);

```

2. In the destructor of middleDeck

- When MiddleDeck is destroyed, the custom styling for the sliders is removed to avoid memory leaks or other issues.

```

MiddleDeck::~MiddleDeck()
{
    speed1.setLookAndFeel(nullptr);
    speed2.setLookAndFeel(nullptr);
}

```

3. In the resized function

- The function calculates the width and height for the sliders and labels, placing them appropriately within the MiddleDeck.
- Sliders are placed vertically, with labels positioned above and value labels below them to indicate the current playback speed.

```

void MiddleDeck::resized()
{
    int halfWidth = getWidth() / 3;
    int smallHeight = getHeight() / 7;

    // Declare label height and Y offset
    int labelHeight = 20;
    int labelYOffset = 5;

    // Set the bounds for the sliders
    speed1.setBounds(15, smallHeight * 3 - labelHeight, halfWidth -
20, smallHeight * 2);
    speed2.setBounds((halfWidth * 2) - 5, smallHeight * 3 -
labelHeight, halfWidth - 20, smallHeight * 2);

    // Set the bounds for the labels
    speed1Label.setBounds(10, smallHeight * 3 - (labelHeight * 3 +
labelYOffset), halfWidth, smallHeight);
    speed2Label.setBounds(halfWidth * 2 - 10, smallHeight * 3 -
(labelHeight * 3 + labelYOffset), halfWidth, smallHeight);
    speed1ValueLabel.setBounds(5, smallHeight * 4 + labelHeight * 2 -
labelYOffset, halfWidth, smallHeight);

```

```

        speed2ValueLabel.setBounds(halfWidth * 2 - 15, smallHeight * 4 +
        labelHeight * 2 - labelYOffset, halfWidth, smallHeight);
    }

```

4. In `sliderValueChanged(Slider* slider)` function

- When `speed1` is adjusted, the speed of `player1` is updated, and the label below the slider is updated to reflect the new speed (e.g., 1.5x).
- Similarly, when `speed2` is adjusted, the speed of `player2` is updated and reflected in the `speed2ValueLabel`.

```

void MiddleDeck::sliderValueChanged(Slider* slider)
{
    if (//)
    {
    }
    else if (slider == &speed1)
    {
        // Set the speed for player 1 and update the speed1ValueLabel
        player1->setSpeed(slider->getValue());
        speed1ValueLabel.setText(juce::String(slider->getValue(), 1)
+ "x", juce::dontSendNotification);
    }
    else if (slider == &speed2)
    {
        // Set the speed for player 2 and update the speed2ValueLabel
        player2->setSpeed(slider->getValue());
        speed2ValueLabel.setText(juce::String(slider->getValue(), 1)
+ "x", juce::dontSendNotification);
    }
}

```

5. In `DJAudioPlayer`

- The method accepts a ratio that can range from 0.1 to 10.0, representing 10% to 1000% of the original speed.
- The `resampleSource` is responsible for handling the actual change in speed by adjusting the resampling ratio, which directly affects the playback speed.

```

// setSpeed: Sets the playback speed by adjusting the resampling
ratio.
// @param ratio The speed ratio, typically between 0.1 and 10.
void DJAudioPlayer::setSpeed(double ratio)
{
    if (ratio < 0.1 || ratio > 10.0) {
        std::cout << "DJAudioPlayer::setSpeed ratio should be between
0.1 to 10" << std::endl;
    }
    else {
        resampleSource.setResamplingRatio(ratio);
    }
}

```

R2: Customise the user interface (UI)

R2A: New GUI layout.

i) MiddleDeck

1. MiddleDeck.h

- DJAudioPlayer Pointers: player1 and player2 point to the two audio players controlled by this component.
- Sliders: There are five sliders—vol1, vol2 for volume, speed1, speed2 for speed, and crossfader for blending the audio between the two players.
- Labels: Labels are provided to display text alongside sliders for easy identification.
- Custom LookAndFeel: A custom visual style is applied to the sliders and labels via a CustomLookAndFeel object.
- The header file sets up the interface, declaring the sliders and labels that will be used in the MiddleDeck.
- It defines the constructor, destructor, and key methods (paint, resized, and sliderValueChanged) that will handle the component's functionality.

```
/*
=====
=====

MiddleDeck.h
Created: 1 Aug 2024 8:43:59pm
Author: Admin

=====
=====
*/

#pragma once

#include <JuceHeader.h>           // JUCE framework header
for GUI components and utilities
#include "DJAudioPlayer.h"       // Custom class for
handling audio playback
#include "CustomLookAndFeel.h"   // Custom LookAndFeel
for styling the sliders and labels

//=====
=====
// MiddleDeck is a custom component that acts as the
central control deck for a DJ application.
// It allows for controlling two audio players via
sliders for volume, speed, and crossfading.
class MiddleDeck : public juce::Component,      //
Inherits from JUCE Component to create a custom GUI
component
    public juce::Slider::Listener // Implements
Slider::Listener to handle slider value changes

{
public:
```

```

        // Constructor: Initializes the MiddleDeck with two
        DJAudioPlayer instances.
        // @param player1 Pointer to the first DJAudioPlayer
        instance.
        // @param player2 Pointer to the second
        DJAudioPlayer instance.
        MiddleDeck(DJAudioPlayer* player1, DJAudioPlayer*
        player2);

        // Destructor: Cleans up resources used by the
        MiddleDeck.
        ~MiddleDeck() override;

        // paint: Handles the drawing of the MiddleDeck
        component.
        // This method draws the background, images, and
        text labels.
        void paint(juce::Graphics&) override;

        // resized: Adjusts the layout of the MiddleDeck's
        sliders and labels when the component size changes.
        void resized() override;

        // sliderValueChanged: Called when a slider's value
        changes.
        // Updates the corresponding DJAudioPlayer or the
        displayed value based on which slider was changed.
        void sliderValueChanged(Slider* slider) override;

private:
        DJAudioPlayer* player1; // Pointer to the first
        DJAudioPlayer instance
        DJAudioPlayer* player2; // Pointer to the second
        DJAudioPlayer instance

        // Sliders for controlling volume, speed, and
        crossfading between the two players
        Slider crossfader;
        Slider vol1;
        Slider vol2;
        Slider speed1;
        Slider speed2;

        // Labels for displaying text next to the sliders
        Label vol1Label;
        Label vol2Label;
        Label crossfaderLabel;
        Label speed1Label;
        Label speed2Label;
        Label speed1ValueLabel; // Label to display the
        current speed for player 1
        Label speed2ValueLabel; // Label to display the
        current speed for player 2

        juce::Image image; // Image displayed in the
        MiddleDeck component

        // Unique pointer to a custom LookAndFeel instance
        for styling the sliders and labels
        std::unique_ptr<CustomLookAndFeel>
        customLookAndFeel;

```

```

// Macro to prevent copying and memory leaks
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(MiddleDeck)
};

```

2. MiddleDeck.cpp

Constructor:

- Sliders are initialized with custom look-and-feel and appropriate ranges for volume (0.1 to 1.0) and speed (0.1 to 10.0).
- A crossfader slider is added to control the mix between player1 and player2.
- Labels are set up for each slider to make the interface user-friendly.
- An image is loaded to provide a visual element in the MiddleDeck.

Destructor:

- The destructor resets the custom look-and-feel for each slider to ensure proper memory management and cleanup.

Painting the UI:

- The paint() method draws a gradient background, the image, and the "Otodeck DJ" text with a custom font.

Resizing Components:

- The resized() method sets the bounds for sliders and labels, ensuring proper placement when the window is resized.

Slider Value Changes:

- The sliderValueChanged() method reacts to changes in the sliders for volume, speed, and crossfading. It updates the corresponding player and value labels based on the changes.

```

/*
=====
=====

MiddleDeck.cpp
Created: 1 Aug 2024 8:43:59pm
Author: Admin

=====
=====
*/

#include <JuceHeader.h>           // JUCE framework header
#include "MiddleDeck.h"          // Include the header
file for the MiddleDeck class

//=====
=====

```

```

// Constructor: Initializes the MiddleDeck component
with two DJAudioPlayer instances.
// Sets up the sliders, labels, and the custom
LookAndFeel for the component.
MiddleDeck::MiddleDeck(DJAudioPlayer* _player1,
DJAudioPlayer* _player2)
: player1(_player1),
player2(_player2)
{
// Initialize the custom LookAndFeel and apply it to the
sliders
customLookAndFeel =
std::make_unique<CustomLookAndFeel>();

vol1.setLookAndFeel(customLookAndFeel.get());
vol2.setLookAndFeel(customLookAndFeel.get());
speed1.setLookAndFeel(customLookAndFeel.get());
speed2.setLookAndFeel(customLookAndFeel.get());
crossfader.setLookAndFeel(customLookAndFeel.get());

// Set up slider ranges and initial values
vol1.setRange(0.1, 1.0);
vol1.setValue(1.0);
vol2.setRange(0.1, 1.0);
vol2.setValue(1.0);
crossfader.setRange(0.1, 1.0);
crossfader.setValue(0.5);
speed1.setRange(0.1, 10.0);
speed1.setValue(0.5);
speed2.setRange(0.1, 10.0);
speed2.setValue(0.5);

// Set slider styles (no text box, rotary for volume,
linear vertical for speed)
crossfader.setTextBoxStyle(Slider::NoTextBox, true, 0,
0);
vol1.setTextBoxStyle(Slider::NoTextBox, true, 0, 0);
vol2.setTextBoxStyle(Slider::NoTextBox, true, 0, 0);
speed1.setTextBoxStyle(Slider::NoTextBox, true, 0, 0);
speed2.setTextBoxStyle(Slider::NoTextBox, true, 0, 0);

vol1.setSliderStyle(Slider::Rotary);
vol2.setSliderStyle(Slider::Rotary);
speed1.setSliderStyle(Slider::LinearVertical);
speed2.setSliderStyle(Slider::LinearVertical);

// Make sliders visible
addAndMakeVisible(vol1);
addAndMakeVisible(vol2);
addAndMakeVisible(speed1);
addAndMakeVisible(speed2);
addAndMakeVisible(crossfader);

// Initialize and style labels
vol1Label.setText("Vol1", dontSendNotification);
vol2Label.setText("Vol2", dontSendNotification);
speed1Label.setText("Speed-1", dontSendNotification);
speed2Label.setText("Speed-2", dontSendNotification);
crossfaderLabel.setText("Crossfader",
dontSendNotification);

//positioning center

```

```

vol1Label.setJustificationType(juce::Justification::centred);
vol2Label.setJustificationType(juce::Justification::centred);
speed1Label.setJustificationType(juce::Justification::centred);
speed2Label.setJustificationType(juce::Justification::centred);
crossfaderLabel.setJustificationType(juce::Justification::centred);

//Set font style
vol1Label.setFont(juce::Font(16.0f, juce::Font::bold));
vol1Label.setColour(juce::Label::textColourId, juce::Colours::black);

vol2Label.setFont(juce::Font(16.0f, juce::Font::bold));
vol2Label.setColour(juce::Label::textColourId, juce::Colours::black);

speed1Label.setFont(juce::Font(16.0f, juce::Font::bold));
speed1Label.setColour(juce::Label::textColourId, juce::Colours::black);

speed2Label.setFont(juce::Font(16.0f, juce::Font::bold));
speed2Label.setColour(juce::Label::textColourId, juce::Colours::black);

speed1ValueLabel.setJustificationType(juce::Justification::centred);
speed1ValueLabel.setFont(juce::Font(16.0f, juce::Font::bold));
speed1ValueLabel.setColour(juce::Label::textColourId, juce::Colours::black);

speed2ValueLabel.setJustificationType(juce::Justification::centred);
speed2ValueLabel.setFont(juce::Font(16.0f, juce::Font::bold));
speed2ValueLabel.setColour(juce::Label::textColourId, juce::Colours::black);

crossfaderLabel.setFont(juce::Font(16.0f, juce::Font::bold));
crossfaderLabel.setColour(juce::Label::textColourId, juce::Colours::black);

// Make labels visible
addAndMakeVisible(vol1Label);
addAndMakeVisible(vol2Label);
addAndMakeVisible(speed1Label);
addAndMakeVisible(speed2Label);
addAndMakeVisible(speed1ValueLabel);
addAndMakeVisible(speed2ValueLabel);
addAndMakeVisible(crossfaderLabel);

// Add listeners to sliders to handle value changes
crossfader.addListener(this);
vol1.addListener(this);
vol2.addListener(this);
speed1.addListener(this);

```

```

speed2.addListener(this);

// Load an image from memory to display in the component
image =
juce::ImageCache::getFromMemory(BinaryData::dj_png,
BinaryData::dj_pngSize);
}

// Destructor: Cleans up resources used by the
MiddleDeck component, including resetting the
LookAndFeel for the sliders.
MiddleDeck::~MiddleDeck()
{
vol1.setLookAndFeel(nullptr);
vol2.setLookAndFeel(nullptr);
speed1.setLookAndFeel(nullptr);
speed2.setLookAndFeel(nullptr);
crossfader.setLookAndFeel(nullptr);
}

// paint: Handles the drawing of the MiddleDeck
component.
// This method draws the background gradient, the image,
and the text labels.
void MiddleDeck::paint(juce::Graphics& g)
{
// Create a horizontal gradient from left (sky blue) to
right (pink)
juce::ColourGradient gradient(juce::Colours::skyblue, 0,
0, juce::Colours::pink, (float)getWidth(), 0, false);

// Fill the entire background with the gradient
g.setGradientFill(gradient);
g.fillAll();

// Set the color for the outline and draw it
g.setColour(juce::Colours::white);
g.drawRect(getLocalBounds(), 1); // Draw an outline
around the component

// Draw the image above the text, scaling it to fit the
desired area
if (image.isValid())
{
g.drawImage(image, 25, 10, getWidth() * 0.8, getHeight()
/ 4, 0, 0, image.getWidth(), image.getHeight());
}

// Set the color and font for the text
g.setColour(juce::Colours::black);
juce::Font customFont =
juce::Font(juce::Typeface::createSystemTypefaceFor(Binary
Data::Rancho_ttf, BinaryData::Rancho_ttfSize));
customFont.setHeight(55.0f); // Set font size
g.setFont(customFont);

// Draw the "Otodeck DJ" text below the image
g.drawText("Otodeck DJ",
getLocalBounds().withTrimmedTop(getHeight() / 4),
juce::Justification::centredTop, true);
}

```

```

// resized: Adjusts the layout of the sliders and labels
when the component size changes.
// Positions each slider and label within the component
based on the current dimensions.
void MiddleDeck::resized()
{
    int halfWidth = getWidth() / 3;
    int smallHeight = getHeight() / 7;

    // Declare label height and Y offset
    int labelHeight = 20;
    int labelYOffset = 5;

    // Set the bounds for the sliders
    speed1.setBounds(15, smallHeight * 3 - labelHeight,
        halfWidth - 20, smallHeight * 2);
    speed2.setBounds((halfWidth * 2) - 5, smallHeight * 3 -
        labelHeight, halfWidth - 20, smallHeight * 2);
    vol1.setBounds(5, smallHeight * 5 + labelYOffset,
        halfWidth, smallHeight - 10);
    vol2.setBounds((halfWidth * 2) - 10, smallHeight * 5 +
        labelYOffset, halfWidth, smallHeight - 10);
    crossfader.setBounds(10, smallHeight * 6 + labelHeight,
        getWidth() - 20, smallHeight / 2);

    // Set the bounds for the labels
    speed1Label.setBounds(10, smallHeight * 3 - (labelHeight
        * 3 + labelYOffset), halfWidth, smallHeight);
    speed2Label.setBounds(halfWidth * 2 - 10, smallHeight *
        3 - (labelHeight * 3 + labelYOffset), halfWidth,
        smallHeight);
    speed1ValueLabel.setBounds(5, smallHeight * 4 +
        labelHeight * 2 - labelYOffset, halfWidth, smallHeight);
    speed2ValueLabel.setBounds(halfWidth * 2 - 15,
        smallHeight * 4 + labelHeight * 2 - labelYOffset,
        halfWidth, smallHeight);
    vol1Label.setBounds(5, smallHeight * 6, halfWidth,
        labelHeight);
    vol2Label.setBounds((halfWidth * 2) - 10, smallHeight *
        6, halfWidth, labelHeight);
    crossfaderLabel.setBounds(0, getHeight() - labelHeight,
        getWidth(), labelHeight);
}

// sliderValueChanged: Called when a slider's value
changes.
// Updates the corresponding DJAudioPlayer or the
displayed speed value based on which slider was changed.
void MiddleDeck::sliderValueChanged(Slider* slider)
{
    if (slider == &crossfader)
    {
        // Adjust the mix ratio for both players based on the
        crossfader's value
        player1->setMixRatio(1.0f - slider->getValue());
        player2->setMixRatio(slider->getValue());
    }
    else if (slider == &vol1)
    {
        // Set the gain for player 1 based on the vol1 slider's
        value
        player1->setGain(slider->getValue());
    }
}

```

```

else if (slider == &vol2)
{
    // Set the gain for player 2 based on the vol2 slider's
    value
    player2->setGain(slider->getValue());
}
else if (slider == &speed1)
{
    // Set the speed for player 1 and update the
    speed1ValueLabel
    player1->setSpeed(slider->getValue());
    speed1ValueLabel.setText(juce::String(slider-
    >getValue(), 1) + "x", juce::dontSendNotification);
}
else if (slider == &speed2)
{
    // Set the speed for player 2 and update the
    speed2ValueLabel
    player2->setSpeed(slider->getValue());
    speed2ValueLabel.setText(juce::String(slider-
    >getValue(), 1) + "x", juce::dontSendNotification);
}
}

```

3. In MainComponent.h

- The MainComponent contains instances of MiddleDeck, DJAudioPlayer, DeckGUI, and PlaylistComponent, allowing for full control over audio playback and mixing.

```

    // AudioFormatManager is responsible for managing
    the available audio formats.
    AudioFormatManager formatManager;

    // AudioThumbnailCache is used to cache waveform
    thumbnails for faster drawing.
    AudioThumbnailCache thumbCache{ 100 };

    // PlaylistComponent manages the list of tracks and
    interaction with the DeckGUIs.
    // It references both deckGUI1 and deckGUI2 for
    controlling audio playback.
    PlaylistComponent playlistComponent{ &deckGUI1,
    &deckGUI2 };

    // Two DJAudioPlayer instances for managing audio
    playback logic.
    DJAudioPlayer player1{ formatManager };
    DJAudioPlayer player2{ formatManager };

    // Two DeckGUI instances that provide the GUI for
    interacting with the DJAudioPlayers.
    DeckGUI deckGUI1{ &player1, formatManager,
    thumbCache, &playlistComponent, juce::Colours::skyblue
    };
    DeckGUI deckGUI2{ &player2, formatManager,
    thumbCache, &playlistComponent, juce::Colours::pink };

    // MiddleDeck is a GUI component that manages
    crossfading between player1 and player2.

```



```

        MiddleDeck middleDeck{ &player1, &player2 };

        // MixerAudioSource is used to mix the audio output
        from both DJAudioPlayers.
        MixerAudioSource mixerSource;

        // This macro helps to prevent copying and memory
        leaks in JUCE components.

        JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(MainComponent)

```

4. In MainComponent.cpp

- The MainComponent initializes all the components, sets up the audio channels, and manages the layout of the entire DJ interface. It handles the interaction between two DeckGUI components, the MiddleDeck, and the PlaylistComponent.
- The MainComponent sets up and displays the MiddleDeck between the two DeckGUI instances. It integrates the playlist component and manages audio mixing through the MixerAudioSource.

```

MainComponent::MainComponent()
{
    // Set the size of the component.
    setSize(800, 600);

    // Request permission to access the microphone if
    required.
    if
    (juce::RuntimePermissions::isRequired(juce::RuntimePermissions::recordAudio)
    &&
    !juce::RuntimePermissions::isGranted(juce::RuntimePermissions::recordAudio))
    {

        juce::RuntimePermissions::request(juce::RuntimePermissions::recordAudio,
        [&](bool granted) { setAudioChannels(granted
        ? 2 : 0, 2); });
    }
    else
    {
        // Open audio channels (0 input channels, 2
        output channels).
        setAudioChannels(0, 2);
    }

    // Add the DeckGUI components to the main component.
    addAndMakeVisible(deckGUI1);
    addAndMakeVisible(deckGUI2);
    addAndMakeVisible(middleDeck);

    // Add the PlaylistComponent to the main component.
    addAndMakeVisible(playlistComponent);

    // Register basic audio formats (e.g., WAV, MP3)
    with the AudioFormatManager.
    formatManager.registerBasicFormats();
}

```

```

}

// Destructor - shuts down audio and cleans up
resources.
MainComponent::~MainComponent()
{
    shutdownAudio(); // Stops the audio processing and
releases resources.
}

//=====
// Called before playback starts to prepare the audio
callback.
void MainComponent::prepareToPlay(int
samplesPerBlockExpected, double sampleRate)
{
    // Prepare the mixer source and add both players as
input sources.
    mixerSource.prepareToPlay(samplesPerBlockExpected,
sampleRate);
    mixerSource.addInputSource(&player1, false);
    mixerSource.addInputSource(&player2, false);
}

// Called repeatedly to process audio blocks.
void MainComponent::getNextAudioBlock(const
juce::AudioSourceChannelInfo& bufferToFill)
{
    mixerSource.getNextAudioBlock(bufferToFill); // Mix
the audio from both players into the output buffer.
}

// Called when playback stops to release audio
resources.
void MainComponent::releaseResources()
{
    player1.releaseResources();
    player2.releaseResources();
    mixerSource.releaseResources();
}

//=====
// Paints the component (e.g., background, text).
void MainComponent::paint(juce::Graphics& g)
{
    g.fillAll(getLookAndFeel().findColour(juce::ResizableWin
dow::backgroundColourId)); // Fill background.
    g.setFont(20.0f); // Set font size for any custom
text drawing.
}

// Updates the layout of child components when the main
component is resized.
void MainComponent::resized()
{
    int unit = getWidth() / 5;

    // Set the bounds of DeckGUI1, DeckGUI2, and
MiddleDeck.

```

```

        deckGUI1.setBounds(0, 0, unit * 2, getHeight() * 3 / 4);
        deckGUI2.setBounds(unit * 3, 0, unit * 2,
getHeight() * 3 / 4);
        middleDeck.setBounds(unit * 2, 0, unit, getHeight()
* 3 / 4);

        // Set the bounds of the PlaylistComponent.
        playlistComponent.setBounds(0, getHeight() * 3 / 4,
getWidth(), getHeight() * 1 / 4);
    }

```

ii) CustomLookAndFeel

1. In CustomLookAndFeel.h

- **Constructor/Destructor:** Initializes default colours for sliders and buttons, such as the red thumb for sliders and black background for buttons.
- **drawRotarySlider:** Customizes the rotary slider's appearance, adding depth with shadows, a gradient-filled knob, and a pointer to indicate the position.
- **drawLinearSlider:** Handles the drawing of linear sliders, both horizontal (used for crossfading) and vertical (used for volume and speed). This method adds custom scaling lines and gradient backgrounds.
- **drawButtonBackground:** Creates a hexagonal button with a shadow and glowing effect when hovered or clicked. The button shape is drawn programmatically, providing a more dynamic look.
- **drawButtonText:** Styles the text within buttons, adjusting based on hover or click states and ensuring the text is centered within the button.

```

/*
=====
=====

    CustomLookAndFeel.h
    Created: 3 Aug 2024 12:14:58pm
    Author: Admin

=====
=====
*/

#pragma once

#include <JuceHeader.h> // JUCE framework header for GUI
components and utilities

//=====
=====

```

```

// CustomLookAndFeel is a class that extends the JUCE
LookAndFeel_V4 to customize the appearance of sliders,
buttons, and other GUI components.
// It overrides various drawing methods to provide a
unique look for the application's interface.
class CustomLookAndFeel : public juce::LookAndFeel_V4 //
Inherits from LookAndFeel_V4 to customize the appearance
of components
{
public:
    // Constructor: Sets up the custom LookAndFeel with
    default colours for various components.
    CustomLookAndFeel();

    // Destructor: Cleans up any resources used by the
    CustomLookAndFeel.
    ~CustomLookAndFeel();

    // drawRotarySlider: Customizes the appearance of
    rotary sliders.
    // This method is responsible for drawing the rotary
    knob, background, and pointer.
    void drawRotarySlider(juce::Graphics& g, int x, int
y, int width, int height, float sliderPos,
    const float rotaryStartAngle, const float
rotaryEndAngle, juce::Slider&) override;

    // drawLinearSlider: Customizes the appearance of
    linear sliders.
    // This method is responsible for drawing both
    horizontal and vertical sliders.
    void drawLinearSlider(Graphics& g, int x, int y, int
width, int height, float sliderPos,
    float minSliderPos, float maxSliderPos, const
Slider::SliderStyle,
    Slider& slider) override;

    // drawButtonBackground: Customizes the appearance
    of button backgrounds.
    // This method is responsible for drawing the
    button's shape, background, and effects like shadows and
    glows.
    void drawButtonBackground(juce::Graphics& g,
juce::Button& button,
    const juce::Colour& backgroundColour,
    bool isMouseOverButton, bool isButtonDown)
override;

    // drawButtonText: Customizes the appearance of
    button text.
    // This method is responsible for drawing the text
    on buttons, considering hover and click states.
    void drawButtonText(juce::Graphics& g,
juce::TextButton& button,
    bool isMouseOverButton, bool isButtonDown)
override;
};

```

2. In CustomLookAndFeel.cpp

- **Constructor:**

- Defines colours for components like the slider thumb (red) and button background (black), with text changing from black (default) to red (when pressed).
- **drawRotarySlider:**
 - Draws the rotary slider using a circular gradient for a metallic look, a shadow for depth, and a pointer to show the current slider value.
 - Utilises the `juce::AffineTransform` to rotate the pointer as the slider value changes.
- **drawLinearSlider:**
 - For horizontal sliders (e.g., crossfader), it draws a purple background, yellow scaling lines, and a green thumb.
 - For vertical sliders, it uses a white-to-orange gradient with grey scaling lines and a central dark-grey line.
 - Draws a custom thumb based on the slider position.
- **drawButtonBackground:**
 - Creates a hexagonal button shape with a shadow effect for depth and glow effect when hovered.
 - Uses the `juce::Path` to define the hexagon, and adds a gradient fill inside the button.
 - When hovered, a white overlay is added for emphasis.
- **drawButtonText:**
 - Ensures button text is bold and centred, adjusting based on the button's state (pressed or hovered).
 - Utilises `drawFittedText` to ensure the text fits within the button's bounds.

```

/*
=====
=====

CustomLookAndFeel.cpp
Created: 3 Aug 2024 12:14:58pm
Author: Admin

=====
=====
*/

#include "CustomLookAndFeel.h" // Include the header
file for the CustomLookAndFeel class

//=====
=====
// Constructor: Sets up the CustomLookAndFeel by
initializing default colors for various components.
// It defines the thumb color for sliders and the text
and background colors for buttons.
CustomLookAndFeel::CustomLookAndFeel()
{
    // Set default colours for components

```

```

        setColour(juce::Slider::thumbColourId,
juce::Colours::red);
        setColour(juce::TextButton::buttonColourId,
juce::Colours::black);
        setColour(juce::TextButton::textColourOffId,
juce::Colours::black);
        setColour(juce::TextButton::textColourOnId,
juce::Colours::red);
    }

    // Destructor: Resets the LookAndFeel to the system
    default if this instance is currently set as the default
    LookAndFeel.
    CustomLookAndFeel::~~CustomLookAndFeel()
    {
        // Reset to the default system LookAndFeel if this
        is currently set as default
        if (&juce::LookAndFeel::getDefaultLookAndFeel() ==
this)
        {

juce::LookAndFeel::setDefaultLookAndFeel(nullptr);
        }
    }

    // drawRotarySlider: Customizes the appearance of rotary
    sliders.
    // Draws a rotary knob with a metal effect, including a
    shadow, gradient fill, and a pointer.
    void CustomLookAndFeel::drawRotarySlider(juce::Graphics&
g, int x, int y, int width, int height, float sliderPos,
        const float rotaryStartAngle, const float
rotaryEndAngle, juce::Slider&)
    {
        // Calculate dimensions and center point for the
        rotary slider
        double radius = (float)juce::jmin(width / 2, height
/ 2) - 4.0f;
        double centreX = (float)x + (float)width * 0.5f;
        double centreY = (float)y + (float)height * 0.5f;
        double rx = centreX - radius;
        double ry = centreY - radius;
        double rw = radius * 2.0f;
        double angle = rotaryStartAngle + sliderPos *
(rotaryEndAngle - rotaryStartAngle);

        // Draw background shadow for depth effect
        g.setColour(juce::Colours::black.withAlpha(0.3f));
        g.fillEllipse(rx + 6, ry + 6, rw, rw); // Offset for
        shadow effect

        // Create a gradient for the main knob with a metal
        effect
        juce::ColourGradient gradient(juce::Colours::orange,
centreX, centreY - radius,
            juce::Colours::silver, centreX, centreY +
radius,
            false); // Linear gradient
        gradient.addColour(0.5,
juce::Colours::lightgoldenrodyellow);
        g.setGradientFill(gradient);
        g.fillEllipse(rx, ry, rw, rw);
    }

```

```

// Add inner highlight for the knob
g.setColour(juce::Colours::white.withAlpha(0.3f));
g.drawEllipse(rx + 1, ry + 1, rw - 2, rw - 2, 2.0f);

// Draw the pointer for the knob
juce::Path p;
double pointerLength = radius;
double pointerThickness = 5.0f;
p.addRectangle(-pointerThickness * 0.5f, -radius,
pointerThickness, pointerLength);

p.applyTransform(juce::AffineTransform::rotation(angle).
translated(centreX, centreY));

g.setColour(juce::Colours::black);
g.fillPath(p);

// Draw the center circle with a gradient effect
juce::ColourGradient
centerGradient(juce::Colours::black, centreX, centreY,
juce::Colours::grey, centreX, centreY + 5,
false); // Linear gradient
g.setGradientFill(centerGradient);
double smallw = rw * 0.3;
g.fillEllipse(centreX - (smallw / 2), centreY -
(smallw / 2), smallw, smallw);

// Add a shadow around the center circle for extra
depth
g.setColour(juce::Colours::black.withAlpha(0.5f));
g.drawEllipse(centreX - (smallw / 2), centreY -
(smallw / 2), smallw, smallw, 1.0f);
}

// drawLinearSlider: Customizes the appearance of linear
sliders.
// Handles both horizontal and vertical sliders, adding
gradient fills, scaling lines, and custom thumbs.
void CustomLookAndFeel::drawLinearSlider(Graphics& g,
int x, int y, int width, int height, float sliderPos,
float minSliderPos, float maxSliderPos, const
Slider::SliderStyle style,
Slider& slider)
{
    if (style == Slider::LinearHorizontal)
    {
        // Draw the background for horizontal slider
        (crossfader)
        g.setColour(juce::Colours::purple);
        g.fillRect(static_cast<float>(x),
static_cast<float>(y) + 8, static_cast<float>(width),
static_cast<float>(height) - 16);

        // Draw scaling lines across the slider
        (horizontal lines)
        g.setColour(juce::Colours::yellow);
        for (int i = 0; i <= 10; ++i)
        {
            float lineX = static_cast<float>(x + width *
i / 10.0f);
            g.drawLine(lineX, static_cast<float>(y + 2),
lineX, static_cast<float>(y + height - 2), 1.0f);
        }
    }
}

```

```

        // Draw the thumb (handle) for the horizontal
slider
        g.setColour(juce::Colours::green);
        g.fillRect(sliderPos - 10.0f,
static_cast<float>(y), 15.0f,
static_cast<float>(height));
    }
    else
    {
        // For vertical sliders, draw a gradient-filled
background
        juce::ColourGradient
gradient(juce::Colours::white, static_cast<float>(x),
static_cast<float>(y),
        juce::Colours::orange,
static_cast<float>(x), static_cast<float>(y + height),
        false); // Linear gradient
        g.setGradientFill(gradient);
        g.fillRect(static_cast<float>(x) + 10,
static_cast<float>(y), static_cast<float>(width) - 20,
static_cast<float>(height) + 20);

        // Draw scaling lines along the vertical slider
        g.setColour(juce::Colours::darkgrey);
        for (int i = 0; i <= 12; ++i)
        {
            float lineY = static_cast<float>(y + height
* i / 10.0f);
            g.drawLine(static_cast<float>(x + (width /
2) - 15), lineY,
                static_cast<float>(x + width * 2 / 3),
lineY, 1.0f);
        }

        // Draw the slider line in the center (dark
grey, always visible)
        float centerX = static_cast<float>(x + width /
2);
        g.setColour(juce::Colours::darkgrey);
        g.fillRect(centerX - 2.0f,
static_cast<float>(y), 4.0f, static_cast<float>(height)
+ 20);

        // Draw the thumb (white rectangle) for the
vertical slider
        g.setColour(juce::Colours::grey);
        g.fillRect(centerX - static_cast<float>(width /
4), sliderPos, static_cast<float>(width / 2), 15.0f);
    }
}

// drawButtonBackground: Customizes the appearance of
button backgrounds.
// Draws a hexagon shape for the button, with shadow and
glowing effects based on mouse interaction.
void
CustomLookAndFeel::drawButtonBackground(juce::Graphics&
g, juce::Button& button,
    const juce::Colour& backgroundColour, bool
isMouseOverButton, bool isButtonDown)
{

```



```

        juice::Rectangle<int>
        buttonArea(button.getLocalBounds()); // Get the bounds
        of the button area

        // Create a hexagon path for the button
        juice::Path hexagon;
        float buttonWidth =
static_cast<float>(buttonArea.getWidth());
        float buttonHeight =
static_cast<float>(buttonArea.getHeight());
        float sideLength = buttonWidth / 3.0f;

        // Define hexagon points (6 points for the hexagon)
        hexagon.startNewSubPath(buttonArea.getCentreX(),
buttonArea.getY()); // Top point
        hexagon.lineTo(buttonArea.getRight() - sideLength /
2, buttonArea.getY() + buttonHeight / 4); // Top right
        hexagon.lineTo(buttonArea.getRight() - sideLength /
2, buttonArea.getBottom() - buttonHeight / 4); // Bottom
right
        hexagon.lineTo(buttonArea.getCentreX(),
buttonArea.getBottom()); // Bottom point
        hexagon.lineTo(buttonArea.getX() + sideLength / 2,
buttonArea.getBottom() - buttonHeight / 4); // Bottom
left
        hexagon.lineTo(buttonArea.getX() + sideLength / 2,
buttonArea.getY() + buttonHeight / 4); // Top left
        hexagon.closeSubPath(); // Complete the hexagon

        // Draw shadow for the hexagon
        juice::Colour shadowColour =
juice::Colours::black.withAlpha(0.5f);
        juice::DropShadow shadowEffect(shadowColour, 8,
juice::Point<int>(4, 4)); // Shadow with an offset
        shadowEffect.drawForPath(g, hexagon);

        // Draw glowing effect if the mouse is over the
button
        if (isMouseOverButton) {
            juice::Colour glowColour =
juice::Colours::magenta.withAlpha(0.3f);
            juice::DropShadow glowShadow(glowColour, 15,
juice::Point<int>(0, 0)); // Centered glow effect
            glowShadow.drawForPath(g, hexagon);
        }

        // Draw the hexagon background with a gradient fill
        juice::ColourGradient gradient(juice::Colours::white,
buttonArea.getTopLeft().toFloat(),
            juice::Colours::orange,
buttonArea.getBottomRight().toFloat(), false); // Linear
gradient
        g.setGradientFill(gradient);
        g.fillPath(hexagon);

        // Draw the hexagon border
        g.setColour(juice::Colours::orange.brighter());
        g.strokePath(hexagon, juice::PathStrokeType(2.0f));

        // Add a light transparent overlay when the mouse is
over the button
        if (isMouseOverButton) {

```

```

        g.setColour(juce::Colours::white.withAlpha(0.1f));
        g.fillPath(hexagon);
    }
}

// drawButtonText: Customizes the appearance of button
// text.
// Draws the text within the button, taking into account
// the hover and click states.
void CustomLookAndFeel::drawButtonText(juce::Graphics&
g, juce::TextButton& button,
    bool isMouseOverButton, bool isButtonDown)
{
    juce::Rectangle<int> textArea =
button.getLocalBounds().reduced(10); // Add padding to
keep text inside the hexagon
    juce::Colour textColour =
button.findColour(button.getToggleState() ?
    juce::TextButton::textColourOnId :
    juce::TextButton::textColourOffId);

    g.setColour(textColour);

    // Set font for the button text
    juce::Font font(15.0f, juce::Font::bold);
    g.setFont(font);

    // Draw the text centered within the reduced bounds
    g.drawFittedText(button.getButtonText(), textArea,
juce::Justification::centred, 1);
}

```

3. In DeckGUI and MiddleDeck

- The custom look-and-feel is assigned to buttons and sliders using the setLookAndFeel() method.

```

CustomLookAndFeel customLookAndFeel; // Custom look
and feel for the UI components
//And applied like this in Constructor

loadButton.setLookAndFeel(&customLookAndFeel);
//Other buttons and sliders

```

iii) WaveFormDisplay

1. WaveFormDisplay.h

```

/*
=====
=====

```

WaveFormDisplay.h
Created: 29 Jul 2024 11:44:28am
Author: Admin

```
=====
=====
*/

#pragma once

#include <JuceHeader.h> // JUCE framework header for
audio and GUI functionalities
#include "DJAudioPlayer.h" // Custom class to control
audio playback

//=====
=====
// WaveFormDisplay is a GUI component that visualises
the waveform of an audio file.
// It also responds to changes in the audio file or
playback position.
class WaveFormDisplay : public juce::Component,      //
Inherits from juce::Component to create a custom GUI
component
    public juce::ChangeListener // Implements
ChangeListener to respond to changes from audioThumbnail
{
public:
    // Constructor: Takes references to
    AudioFormatManager and AudioThumbnailCache to initialize
    the audio thumbnail
    WaveFormDisplay(AudioFormatManager&
formatManagerToUse,
        AudioThumbnailCache& cacheToUse);

    // Destructor: Cleans up resources
    ~WaveFormDisplay() override;

    // paint: Draws the waveform and other visual
    elements on the screen
    void paint(juce::Graphics&) override;

    // resized: Adjusts the layout of any subcomponents
    if the component size changes
    void resized() override;

    // changeListenerCallback: Called when there is a
    change in the audioThumbnail, such as loading a new
    audio file
    void changeListenerCallback(ChangeBroadcaster*
source) override;

    // loadURL: Loads an audio file from a URL into the
    audio thumbnail for visualisation
    void loadURL(URL audioURL);

    // setPositionRelative: Updates the display to
    reflect the current playback position
    void setPositionRelative(double pos);
```

```

        // mouseDrag: Handles user dragging on the waveform
        to change the playback position
        void mouseDrag(const juce::MouseEvent& event);

        // setPlayer: Sets the DJAudioPlayer instance to
        control playback
        void setPlayer(DJAudioPlayer* playerToUse);

        // Flag to indicate if a file has been successfully
        loaded into the audio thumbnail
        bool fileLoaded;

private:
        // AudioThumbnail: Generates and stores a visual
        representation of the audio file
        AudioThumbnail audioThumb;

        // Stores the current playback position relative to
        the length of the audio file
        double position;

        // Pointer to a DJAudioPlayer instance for
        controlling playback
        DJAudioPlayer* player;

        // Macro to prevent copying and memory leaks
        JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(WaveFormDis
        play)
    };

```

2. waveformDisplay.cpp

Code Flow Explanation:

- **Constructor:**
 - audioThumb is initialised with a resolution of 1000 samples and connects to formatManagerToUse and cacheToUse, which handle audio format management and caching.
 - fileLoaded is initially set to false, indicating that no file is loaded.
 - The component registers itself as a ChangeListener for the audioThumb, allowing it to respond to changes in the audio thumbnail, such as when a new file is loaded.
- **Destructor:**
 - No manual cleanup is required because the audioThumb and player pointers are smartly managed.

setPlayer Overview: This method links the waveform display with a DJAudioPlayer instance, allowing the waveform display to control audio playback.

Code Flow Explanation:

- This method assigns the player pointer to the DJAudioPlayer instance passed as a parameter.

- Once the player is set, the waveform display can interact with the player to adjust the playback position and visualisation.

paint Overview: This method is responsible for drawing the waveform, the playback position, or a message if no file is loaded.

Code Flow Explanation:

- The background is filled with black, and a white border is drawn around the component.
- If a file is loaded, the waveform is drawn using `audioThumb.drawChannel()`. The method also draws a red line to indicate the current playback position.
- If no file is loaded, a message ("File not loaded...") is displayed in the center of the component.

resized Overview: This method handles any necessary adjustments when the waveform display is resized.

Code Flow Explanation:

- In this case, the `resized()` method is currently empty because there are no child components to adjust, but it would typically handle resizing.

loadURL Overview: This method asynchronously loads an audio file from a URL into the waveform display, allowing it to visualise the file.

Code Flow Explanation:

- When a new URL is passed, the current waveform is cleared using `audioThumb.clear()`.
- The file is then asynchronously loaded using `audioThumb.setSource()`, which handles loading the file data.
- If the file is successfully loaded, `fileLoaded` is set to true, and the waveform is repainted to show the new audio file.
- If the loading fails, an error message is printed to the console.

changeListenerCallback Overview: This method is triggered when the audio thumbnail is updated, typically when a new audio file is loaded.

Code Flow Explanation:

- Whenever a change occurs in `audioThumb` (like loading a new audio file), the `repaint()` method is called to update the waveform display.

setPositionRelative Overview: This method updates the visual position of the playback head in the waveform display.

Code Flow Explanation:

- The playback position is updated relative to the total length of the audio file, which ranges from 0 to 1.
- If the new position differs from the current one, the position variable is updated, and the waveform display is repainted to reflect the change.

mouseDrag Overview: This method allows users to drag the playback head along the waveform to seek within the audio file.

Code Flow Explanation:

- When the user drags the mouse across the waveform, the newPosition is calculated based on the horizontal position of the drag event relative to the waveform's width.
- The new playback position is clamped to ensure it remains within the valid range [0, 1].
- The player's playback position is updated using player->setPositionRelative(newPosition), and the visual playback position is also updated to reflect the drag.

```

/*
=====
=====

WaveFormDisplay.cpp
Created: 29 Jul 2024 11:44:28am
Author: Admin

=====
=====
*/

#include <JuceHeader.h>           // JUCE framework
header for implementing audio and GUI functionalities
#include "WaveFormDisplay.h"     // Header file for the
WaveFormDisplay class

//=====
=====
// Constructor: Initializes the audio thumbnail and
registers the ChangeListener for the thumbnail.
// The parameters are references to the
AudioFormatManager and AudioThumbnailCache to handle
audio formats and caching.
WaveFormDisplay::WaveFormDisplay(AudioFormatManager&
formatManagerToUse,
    AudioThumbnailCache& cacheToUse)
    : audioThumb(1000, formatManagerToUse, cacheToUse),
// Initialize audio thumbnail with a resolution of 1000
samples
    fileLoaded(false),
// Set the fileLoaded flag to false initially
    position(0),
// Initialize playback position to 0
    player(nullptr)
// Initialize player pointer to null
{
    audioThumb.addListener(this); // Register this
class as a listener to changes in the audio thumbnail
}

// Destructor: No specific cleanup is required, as smart
pointers handle the deletion of the player and
audioThumbnail.
WaveFormDisplay::~WaveFormDisplay()
{

```

```

}

// setPlayer: Sets the DJAudioPlayer instance that will
// be used to control audio playback.
void WaveFormDisplay::setPlayer(DJAudioPlayer*
playerToUse)
{
    player = playerToUse; // Assign the provided player
instance to the member variable
}

// paint: Draws the waveform, a red playhead line, or a
// placeholder message if no file is loaded.
void WaveFormDisplay::paint(juce::Graphics& g)
{
    g.fillAll(juce::Colours::black); // Fill the
background with black colour

    g.setColour(juce::Colours::white); // Set drawing
colour to white
    g.drawRect(getLocalBounds(), 1); // Draw a white
border around the component

    if (fileLoaded)
    {
        // Draw the audio waveform if a file is loaded
        audioThumb.drawChannel(g,
            getLocalBounds(), // The area where the
waveform should be drawn
            0, // Start time of the
waveform to draw
            audioThumb.getTotalLength(), // End time of
the waveform
            0, // Channel index (0 =
left channel for stereo)
            1.0f); // Vertical zoom level

        // Draw a red line representing the current
playback position
        g.setColour(juce::Colours::red);
        g.drawRect(position * getWidth(), 0, 2,
getHeight());
    }
    else
    {
        // Display a message if no file is loaded
        g.setFont(juce::Font(20.0f));
        g.drawText("File not loaded...",
getLocalBounds(),
juce::Justification::centred, true); //
Centre the text within the component
    }
}

// resized: Called when the component is resized.
// Currently, this method is empty because there are no
// child components.
void WaveFormDisplay::resized()
{
    // Normally, this is where you would set the bounds
of any child components.
}

```

```

// loadURL: Asynchronously loads an audio file from a
// URL into the audio thumbnail.
void WaveFormDisplay::loadURL(URL audioURL)
{
    // Ensures that the following operations run on the
    // GUI thread
    juce::MessageManager::callAsync([this, audioURL]()
    {
        audioThumb.clear(); // Clear the current
        // audio thumbnail
        fileLoaded = audioThumb.setSource(new
        URLInputSource(audioURL)); // Load the new audio file
        // from the URL

        if (fileLoaded) // If the file is
        // successfully loaded
        {
            std::cout << "WFD:: Loaded!" <<
            std::endl; // Print a success message to the console
            repaint();
            // Trigger a repaint to display the waveform
        }
        else
        {
            std::cout << "WFD:: Not loaded!" <<
            std::endl; // Print an error message to the console
        }
    });
}

// changeListenerCallback: Called when the audio
// thumbnail changes, typically when a new file is loaded.
// Triggers a repaint.
void
WaveFormDisplay::changeListenerCallback(ChangeBroadcaster* source)
{
    std::cout << "Change listener callback triggered."
    << std::endl; // Log that the change listener was
    // triggered
    repaint(); // Repaint the component to reflect
    // changes in the waveform
}

// setPositionRelative: Updates the position of the
// playback head in the waveform display.
void WaveFormDisplay::setPositionRelative(double pos)
{
    if (pos != position) // If the new position is
    // different from the current one
    {
        position = pos; // Update the position
        repaint(); // Repaint the component to
        // move the red playback line
    }
}

// mouseDrag: Handles dragging on the waveform display
// to seek within the audio file.
void WaveFormDisplay::mouseDrag(const juce::MouseEvent&
event)
{

```



```

        if (fileLoaded && player != nullptr) // If a file
        is loaded and a player is set
        {
            // Calculate the new position based on the drag
            event's X position relative to the component's width
            double newPosition = event.position.x /
            getWidth();

            // Clamp the new position to ensure it's within
            the valid range [0, 1]
            newPosition = juce::jlimit(0.0, 1.0,
            newPosition);

            // Update the player's playback position
            player->setPositionRelative(newPosition);

            // Update the visual playback position
            setPositionRelative(newPosition);
        }
    }
}

```

3. In DJAudioPlayer

DJAudioPlayer (setPosition/setPositionRelative) Overview: These methods adjust the playback position of the audio in the player, either in seconds or relative to the total track length.

Code Flow Explanation:

- **setPosition:** This method sets the playback position in seconds by directly calling `transportSource.setPosition(posInSecs)`. The transport source handles playback and position.
- **setPositionRelative:** This method converts a relative position (between 0 and 1) to an absolute position in seconds based on the track's total length. It then calls `setPosition()` to move the playback head to the appropriate position.

```

// setPosition: Sets the playback position in seconds.
// @param posInSecs The position in seconds.
void DJAudioPlayer::setPosition(double posInSecs)
{
    transportSource.setPosition(posInSecs);
}

// setPositionRelative: Sets the playback position
relative to the track length.
// @param pos The relative position, between 0 and 1.
void DJAudioPlayer::setPositionRelative(double pos)
{
    if (pos < 0 || pos > 1.0) {
        std::cout << "DJAudioPlayer::setPositionRelative
pos should be between 0 to 1" << std::endl;
    }
    else {
        double posInSecs = pos *
        transportSource.getLengthInSeconds();
        setPosition(posInSecs);
    }
}

```

```
}
```

iv) JogWheel

1. JogWheel.h

```
/*
=====
=====

JogWheel.h
Created: 1 Aug 2024 9:28:14pm
Author: Admin

=====
=====
*/

#pragma once

#include <JuceHeader.h>           // JUCE framework
header for GUI components and utilities
#include "DJAudioPlayer.h"       // Custom class for
audio playback control
#include "CircularMeter.h"       // Custom component for
displaying audio levels

//=====
=====
// JogWheel is a custom component representing a jog
wheel interface for controlling audio playback.
// It allows for scrubbing through audio tracks and
provides a visual representation of the current audio
state.
class JogWheel : public juce::AnimatedAppComponent //
Inherits from AnimatedAppComponent to allow animation
updates
{
public:
    // Constructor: Initializes the JogWheel with the
given player and background color.
    // @param player Pointer to the DJAudioPlayer
instance for controlling playback.
    // @param backgroundColor The background color of
the JogWheel component.
    JogWheel(DJAudioPlayer* player, juce::Colour
backgroundColor);

    // Destructor: Cleans up any resources used by the
JogWheel.
    ~JogWheel() override;
```

```

        // paint: Handles the drawing of the JogWheel
        component.
        // This method is responsible for rendering the jog
        wheel, including its background, circles, and animated
        segments.
        void paint(juce::Graphics&) override;

        // resized: Adjusts the layout of the JogWheel and
        its child components when the component size changes.
        void resized() override;

        // update: Called by the animation system to update
        the component's state.
        // Currently, this method does not perform any
        actions but is required by the AnimatedAppComponent.
        void update() override;

        // mouseDown: Called when the mouse button is
        pressed within the component.
        // Stops the audio playback when the jog wheel is
        clicked.
        void mouseDown(const juce::MouseEvent& event)
        override;

        // mouseUp: Called when the mouse button is released
        within the component.
        // Starts the audio playback when the jog wheel is
        released.
        void mouseUp(const juce::MouseEvent& event)
        override;

        // mouseDrag: Called when the mouse is dragged
        within the component.
        // Allows the user to scrub through the audio track
        by rotating the jog wheel.
        void mouseDrag(const juce::MouseEvent& event)
        override;

private:
    DJAudioPlayer* player;           // Pointer to the
    DJAudioPlayer instance used to control audio playback
    juce::Colour backgroundColor;    // Background
    color of the JogWheel component
    juce::Point<float> point;         // Stores the
    current position of the scrubber point
    CircularMeter circularMeter;     // Circular meter
    component used to display the audio level

    double margin = 10;              // Margin around
    the jog wheel for aesthetic spacing

    // calculateSize: Determines the size of the jog
    wheel based on the component's dimensions.
    // The size is adjusted to fit within the component
    while maintaining the defined margin.
    double calculateSize();

    // drawAnimatedSegments: Draws the animated segments
    on the outer circle of the jog wheel.
    // @param g Reference to the Graphics object used
    for drawing.
    // @param center The center position of the jog
    wheel.

```

```

        // @param radius The radius of the jog wheel.
        void drawAnimatedSegments(juce::Graphics& g, double
center, int radius);

        // secondsToString: Converts a time value in seconds
to a string in "MM:SS" format.
        // @param seconds The time value in seconds.
        // @return A JUCE String representing the time in
"MM:SS" format.
        static juce::String secondsToString(double seconds)
        {
            int minutes = static_cast<int>(seconds) / 60;
            int secs = static_cast<int>(seconds) % 60;
            return juce::String::formatted("%02d:%02d",
minutes, secs);
        }

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(JogWheel)
// Macro to prevent copying and memory leaks
};

```

2. In JogWheel.cpp

- **Constructor:**
 - The JogWheel is initialised with a DJAudioPlayer instance to control audio playback and a backgroundColor for the jog wheel's look.
 - circularMeter is added as a visible child, displaying the RMS level of the currently playing audio.
 - The frame rate is set to 60 frames per second using setFramesPerSecond(60), allowing smooth animation updates for the jog wheel.
 - The constructor links the jog wheel with a player and prepares it for interactive audio control.
- **Destructor:** No manual cleanup is necessary; resources are automatically managed.

paint

Overview: This method draws the jog wheel, including the outer rings, animated segments, and the time display.

Code Flow Explanation:

- The method first fills the background with the backgroundColor.
- The size and position of the jog wheel are calculated with calculateSize().
- Three circles (outer, middle, and inner) are drawn with different gradients to create a layered effect.
- The outer circle contains animated segments, which are drawn using the drawAnimatedSegments() function. These segments rotate to represent playback progress.
- The inner circle serves as the visual display for the time or track progress.

drawAnimatedSegments

Overview: Draws animated segments around the jog wheel, visually indicating playback progress.

Code Flow Explanation:

- The segments are drawn in a circular arrangement, each representing a portion of the track.
- The number of segments is set to 24, with each segment representing an equal portion of the jog wheel.
- As the track plays, the segments animate, creating a spinning effect to visualise the playback position. The animation is synchronised with the playback position from `player->getCurrentPosition()`.

calculateSize

Overview: Calculates the jog wheel's size, adjusting it to fit the available space within the component.

Code Flow Explanation:

- The jog wheel size is calculated based on the smaller dimension (either width or height) of the component, minus a defined margin (`margin`), ensuring it remains visually balanced.

resized

Overview: Adjusts the layout of the jog wheel's child components (like the circular meter) when the component size changes.

Code Flow Explanation:

- This method calls `calculateSize()` to ensure that the jog wheel's elements (circular meter and scrubber point) are positioned correctly when resized.
- The circular meter is centred within the jog wheel, and if playback is at the start, the scrubber is positioned at the beginning.

update

Overview: Updates the component at a rate of 60 frames per second.

Code Flow Explanation:

- This method currently doesn't perform any actions but could be extended to handle animations or updates that are tied to the playback progress.

mouseDown

Overview: Pauses the audio playback when the user clicks on the jog wheel.

Code Flow Explanation:

- The method calls `player->stop()` when the mouse button is pressed, temporarily halting the playback until further interaction (e.g., mouse release).

mouseUp

Overview: Resumes audio playback when the mouse button is released.

Code Flow Explanation:

- Upon releasing the mouse, `player->start()` is called to resume playback from the current position.

mouseDrag

Overview: Allows users to scrub through the track by dragging the jog wheel.

Code Flow Explanation:

- The method calculates the angle of the mouse position relative to the jog wheel's centre.
- This angle is converted into a position in seconds, which updates the track position by calling `player->setPosition(newPosition)`.
- This provides precise control over playback, allowing users to seek to specific moments by rotating the jog wheel.

```

/*
=====
=====

JogWheel.cpp
Created: 1 Aug 2024 9:28:14pm
Author: Admin

=====
=====
*/

#include <JuceHeader.h>           // JUCE framework
header
#include "JogWheel.h"           // Header file for the
JogWheel class
#include <cmath>                 // Standard library
header for mathematical functions

#ifndef M_PI                     // Check if M_PI is
defined; if not, define it
#define M_PI 3.14159265358979323846
#endif

//=====
=====
// Constructor: Initializes the JogWheel with the
provided DJAudioPlayer and background color.
// The circular meter is also initialized with a
function to retrieve the current RMS level from the
player.
JogWheel::JogWheel(DJAudioPlayer* _player, juce::Colour
backgroundColor)
    : player(_player),
      backgroundColor(backgroundColor),
      circularMeter([_player] { return _player-
>getRMSLevel(); }, juce::Colours::skyblue)
{
    setFramesPerSecond(60);      // Set the
animation frame rate to 60 FPS
    addAndMakeVisible(circularMeter); // Add the
circular meter as a visible child component
}

// Destructor: Cleans up any resources used by the
JogWheel.
JogWheel::~JogWheel() {}

```

```

// paint: Handles the drawing of the JogWheel component.
// This method draws the background, the jog wheel's
// various circles, and the animated segments.
void JogWheel::paint(juce::Graphics& g)
{
    // Fill the component's background with the
    // specified background color
    g.fillAll(backgroundColor);

    // Calculate the size and center of the jog wheel
    double size = calculateSize();
    double center = size / 2 + margin;
    int radius = size / 2;

    // Draw the outer silver circle with a gradient from
    // dark grey to light grey
    juce::ColourGradient outerCircleGradient(
        juce::Colours::darkgrey, center, center,
        juce::Colours::lightgrey, center + radius,
        center + radius,
        true
    );
    g.setGradientFill(outerCircleGradient);
    g.fillEllipse(margin, margin, size, size);

    // Draw the middle white circle with an orange
    // gradient border
    double innerSize = size * 0.85;
    juce::ColourGradient middleCircleGradient(
        juce::Colours::orange, center, center,
        juce::Colours::white, center + innerSize / 2,
        center + innerSize / 2,
        true
    );
    g.setGradientFill(middleCircleGradient);
    g.fillEllipse(margin + (size - innerSize) / 2,
        margin + (size - innerSize) / 2, innerSize, innerSize);

    // Draw the inner black circle for the time display
    double timeDisplaySize = size * 0.6;
    g.setColour(juce::Colours::black);
    g.fillEllipse(margin + (size - timeDisplaySize) / 2,
        margin + (size - timeDisplaySize) / 2, timeDisplaySize,
        timeDisplaySize);

    // Draw the animated segments on the outer circle
    drawAnimatedSegments(g, center, radius);
}

// drawAnimatedSegments: Draws the animated segments on
// the outer circle of the jog wheel.
// These segments rotate to indicate the playback
// position.
void JogWheel::drawAnimatedSegments(juce::Graphics& g,
    double center, int radius)
{
    int numSegments = 24;
    // Number of segments to draw around the jog wheel
    double segmentAngle = 2.0 * M_PI / numSegments;
    // Angle between each segment

```

```

        double animationProgress = fmod(player-
>getCurrentPosition() * 0.5, 1.0); // Example animation
progress

        for (int i = 0; i < numSegments; ++i)
        {
            double angle = i * segmentAngle +
animationProgress * segmentAngle;
            double startX = center + (radius - 10) *
std::cos(angle);
            double startY = center + (radius - 10) *
std::sin(angle);
            double endX = center + radius * std::cos(angle);
            double endY = center + radius * std::sin(angle);

            // Set the color and thickness for each segment
(can animate these as well)
            g.setColour(juce::Colours::white);
            g.drawLine(startX, startY, endX, endY, 4.0f); //
Adjust thickness as needed
        }
    }

    // calculateSize: Determines the size of the jog wheel
based on the component's dimensions.
// The size is adjusted to fit within the component
while maintaining the defined margin.
double JogWheel::calculateSize()
{
    double width = getWidth();
    double height = getHeight();
    return ((width <= height) ? width : height) -
(margin * 2);
}

    // resized: Called when the component is resized.
// This method adjusts the bounds of the circular meter
and positions it within the jog wheel.
void JogWheel::resized()
{
    double size = calculateSize();
    double center = size / 2 + margin;
    int radius = size / 2;

    // Set the bounds of the circular meter in the
center of the jog wheel
    circularMeter.setBounds(center - size * 0.25, center
- size * 0.25, size * 0.5, size * 0.5);

    // If the playback position is at the beginning, set
the initial point position on the jog wheel
    if (player->getCurrentPosition() == 0)
    {
        point.x = center + radius * std::cos(0);
        point.y = center + radius * std::sin(0);
    }
}

    // update: This method is called by the animation system
to update the component's state.
// Currently, it does not perform any actions but is
required by the AnimatedAppComponent.
void JogWheel::update() {}

```



```

// mouseDown: Called when the mouse button is pressed
within the component.
// Stops the audio playback when the jog wheel is
clicked.
void JogWheel::mouseDown(const juce::MouseEvent& event)
{
    player->stop();
}

// mouseUp: Called when the mouse button is released
within the component.
// Starts the audio playback when the jog wheel is
released.
void JogWheel::mouseUp(const juce::MouseEvent& event)
{
    player->start();
}

// mouseDrag: Called when the mouse is dragged within
the component.
// Allows the user to scrub through the audio track by
rotating the jog wheel.
void JogWheel::mouseDrag(const juce::MouseEvent& event)
{
    double size = calculateSize();
    double center = size / 2 + margin;
    int radius = size / 2;

    // Calculate the angle of the mouse position
    relative to the center of the jog wheel
    double angle = std::atan2(event.y - center, event.x
- center);

    // Convert the angle to a playback position in
seconds
    double newPosition = angle / (2.0 * M_PI) * player-
>getTotalLength();

    // If the new position is negative, adjust it to a
positive value within the total length
    if (newPosition < 0)
        newPosition += player->getTotalLength();

    // Set the new playback position in the
DJAudioPlayer
    player->setPosition(newPosition);
}

```

3. In DJAudioPlayer

- **setPosition:** Sets the absolute playback position in seconds by calling `transportSource.setPosition(posInSecs)`.
- **setPositionRelative:** Converts a relative position (from 0 to 1) to an absolute time in seconds, using the track's total length, and updates the position.
- **start/stop:** Start and stop audio playback using `transportSource.start()` and `transportSource.stop()`, respectively.

- **isPlaying:** Returns whether the audio is currently playing by checking `transportSource.isPlaying()`.
- **getPositionRelative:** Returns the current playback position as a fraction (0 to 1) of the total track length.
- **getCurrentPosition:** Provides the current playback position in seconds.
- **getTotalLength:** Retrieves the total duration of the track in seconds, used to calculate the relative and absolute positions in other methods.

```
// setPosition: Sets the playback position in seconds.
// @param posInSecs The position in seconds.
void DJAudioPlayer::setPosition(double posInSecs)
{
    transportSource.setPosition(posInSecs);
}

// setPositionRelative: Sets the playback position
// relative to the track length.
// @param pos The relative position, between 0 and 1.
void DJAudioPlayer::setPositionRelative(double pos)
{
    if (pos < 0 || pos > 1.0) {
        std::cout << "DJAudioPlayer::setPositionRelative
pos should be between 0 to 1" << std::endl;
    }
    else {
        double posInSecs = pos *
transportSource.getLengthInSeconds();
        setPosition(posInSecs);
    }
}

// start: Starts audio playback.
void DJAudioPlayer::start()
{
    transportSource.start();
}

// stop: Stops audio playback.
void DJAudioPlayer::stop()
{
    transportSource.stop();
}

// isPlaying: Checks if the audio is currently playing.
// @return True if playing, false otherwise.
bool DJAudioPlayer::isPlaying()
{
    return transportSource.isPlaying();
}

// getPositionRelative: Returns the current playback
// position relative to the track length.
// @return The relative position, between 0 and 1.
double DJAudioPlayer::getPositionRelative()
{
    double pos = transportSource.getLengthInSeconds() >
0 ? transportSource.getCurrentPosition() /
transportSource.getLengthInSeconds() : 0;
}
```

```

        if (isnan(pos)) {
            return 0;
        }
        return pos;
    }

    // getCurrentPosition: Returns the current playback
    position in seconds.
    // @return The current playback position in seconds.
    double DJAudioPlayer::getCurrentPosition()
    {
        return transportSource.getCurrentPosition();
    }

    // getTotalLength: Returns the total length of the
    loaded track in seconds.
    // @return The total length in seconds.
    double DJAudioPlayer::getTotalLength() const
    {
        return transportSource.getLengthInSeconds();
    }

```

v) Circular Meter

1. CircularMeter.h

```

/*
=====
=====

CircularMeter.h
Created: 29 Aug 2024 1:51:39am
Author: Admin

=====
=====
*/

#pragma once

#include <JuceHeader.h> // JUCE framework header for GUI
and utility classes

//=====
=====
// CircularMeter is a custom JUCE Component that
visually represents a value in a circular format.
// It periodically updates and redraws itself using a
Timer.
class CircularMeter : public juce::Component,    //
Inherits from juce::Component to create a GUI component
    public juce::Timer        // Inherits from
juce::Timer to update the component at a regular
interval

```

```

{
public:
    // Constructor: Initializes the CircularMeter with a
    // function to supply its value and a colour for rendering
    CircularMeter(std::function<float()>&&
valueFunction, const juce::Colour& colour);

    // Destructor: Ensures proper cleanup when the
    // CircularMeter is destroyed
    ~CircularMeter() override;

    // paint: Handles the drawing of the CircularMeter.
    // This is where the meter's visual representation is
    // created.
    void paint(juce::Graphics& g) override;

    // resized: Adjusts the component's layout when its
    // size changes. Currently, this method does not perform
    // any operations.
    void resized() override;

    // timerCallback: Called periodically by the Timer
    // to update and repaint the component
    void timerCallback() override;

private:
    // generatePointWithRandomness: Generates a point
    // with some random displacement, used to create the
    // irregular shape of the meter
    Point<float>
generatePointWithRandomness(Point<float> p, const float
level);

    // valueSupplier: A function that provides the
    // current value to be displayed by the meter. This value
    // controls the meter's display level.
    std::function<float()> valueSupplier;

    // meterColour: The base colour used for rendering
    // the CircularMeter
    juce::Colour meterColour;

    // random: A Random object, seeded with the current
    // time, used to introduce variability in the meter's shape
    Random random{ Time::currentTimeMillis() };

    // Macro to prevent copying and memory leaks
    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(CircularMet
er)
};

```

2. CircularMeter.cpp

Code Flow Explanation:

- **Constructor:**

- **valueSupplier:** A function passed into the constructor provides the current audio level to be displayed by the meter. This function is often linked to a player's RMS level (Root Mean Square) to visualise audio levels in real-time.
- **meterColour:** The base colour for the circular meter display.
- **startTimerHz(24):** The timer is set to refresh the display 24 times per second, ensuring smooth visual updates of the meter.
- **Destructor:**
 - Stops the timer when the CircularMeter is destroyed, ensuring efficient resource cleanup.

paint

Overview: This method handles drawing the visual representation of the meter, creating a dynamic and organic look by using randomness and colour gradients.

Code Flow Explanation:

- **Bounds and Level Calculation:**
 - The meter size is calculated from the component's bounds, and the audio level is mapped between 0 and 1.
- **Reference Path (Ellipse):**
 - An ellipse is drawn to form the reference shape of the circular meter, which is dynamically deformed using randomness to create an irregular, organic effect.
- **Random Displacement:**
 - `generatePointWithRandomness` adds some random variability to each point on the ellipse to give the meter a dynamic and flowing appearance.
- **Gradient and Stroke:**
 - The `meterColour` is used in a radial gradient to fill the ellipse, and the path is stroked with varying thickness based on the audio level. This stroke gives a sense of the meter's fullness.

resized

Overview: Adjusts the layout of the circular meter when the component is resized. This method is currently a placeholder for future custom layout logic.

timerCallback

Overview: Called 24 times per second to force the component to repaint itself, ensuring the circular meter reflects real-time audio levels.

Code Flow Explanation:

- The `repaint()` method is called, which forces the `paint()` method to execute and update the meter's visual representation.

generatePointWithRandomness

Overview: Adds random displacement to each point on the circular path to make the meter's shape appear more organic and dynamic.

Code Flow Explanation:

- The `random` object generates random values that slightly shift each point's x and y coordinates based on the audio level, resulting in the visually dynamic display of the meter.

```

/*
=====
=====

CircularMeter.cpp
Created: 29 Aug 2024 1:51:39am
Author: Admin

=====
=====
*/

#include <JuceHeader.h>           // JUCE framework
header
#include "CircularMeter.h"       // Header file for the
CircularMeter class

//=====
//=====
// Constructor: Initializes the CircularMeter with the
// provided value-supplying function and colour.
// It also starts a timer to regularly update the meter
// at 24Hz.
CircularMeter::CircularMeter(std::function<float()>&&
valueFunction, const juce::Colour& colour)
    : valueSupplier(std::move(valueFunction)),
meterColour(colour)
{
    startTimerHz(24); // Start a timer that calls
timerCallback 24 times per second
}

// Destructor: Automatically stops the timer when the
CircularMeter is destroyed.
CircularMeter::~~CircularMeter()
{
}

// paint: This is where the CircularMeter's visual
representation is drawn.
// The size of the ellipse is controlled by the current
value supplied by valueSupplier.
// The ellipse is distorted with randomness to create a
dynamic, organic look.
void CircularMeter::paint(juce::Graphics& g)
{
    // Get the bounds of the component as a floating-
point rectangle
    const auto bounds = getLocalBounds().toFloat();

    // Map the supplied value to a level between 0 and 1
    const auto level = jlimit(0.f, 1.f,
jmap(valueSupplier(), 0.0f, 0.1f, 0.f, 1.f));

    // Create a reference path based on an ellipse that
will be deformed with randomness
    Path referencePath{};

```

```

referencePath.addEllipse(bounds.reduced(bounds.proportionOfHeight(0.4f * (1.f - level))));

    Path ellipse{};
    const auto numberOfPoints = 50; // Number of points
    to use when constructing the ellipse

    // Get the length of the reference ellipse path
    const auto ellipseLength =
    referencePath.getLength();

    // Start drawing the deformed ellipse by moving to a
    randomised starting point
    const auto startingPoint =
    generatePointWithRandomness(referencePath.getPath(0.f), level);
    ellipse.startNewSubPath(startingPoint);

    // Generate the points of the ellipse, adding random
    displacement at each step
    for (auto i = 1; i <= numberOfPoints; i++)
    {
        if (i == numberOfPoints)
            ellipse.lineTo(startingPoint); // Close the
            ellipse by connecting back to the starting point
        else

    ellipse.lineTo(generatePointWithRandomness(referencePath
    .getPath(ellipseLength * i / numberOfPoints),
    level));
    }
    ellipse.closeSubPath(); // Close the path to form a
    complete shape
    ellipse = ellipse.createPathWithRoundedCorners(1.f);
    // Smooth the corners of the ellipse

    // Apply a transformation to the ellipse to slightly
    shrink it based on the level
    const auto affine = AffineTransform::translation(-
    4.f * level, -4.f * level);
    ellipse.applyTransform(affine);

    // Create a radial gradient for filling the ellipse,
    transitioning from a semi-transparent to a fully
    transparent colour
    const auto gradient =
    ColourGradient(meterColour.withAlpha(0.6f),
    { getWidth() / 2.f, getHeight() / 2.f },
    meterColour.withAlpha(0.f),
    {}, true);
    g.setGradientFill(gradient);

    // Stroke the ellipse path with a thick line, the
    thickness depends on the level
    g.strokePath(ellipse, PathStrokeType{ 5.f * level,
    PathStrokeType::JointStyle::curved,
    PathStrokeType::rounded });

    // Draw another stroke around the ellipse with a
    slightly thinner line and a lighter colour
    g.setColour(meterColour.withMultipliedLightness(1.f
    + level * 0.3f));

```

```

        g.strokePath(ellipse, PathStrokeType{ 2.f + 1.f *
level, PathStrokeType::JointStyle::curved,
PathStrokeType::rounded });
    }

    // resized: This function is called whenever the
    CircularMeter is resized.
    // Currently, it doesn't perform any operations but is a
    placeholder for future layout logic.
    void CircularMeter::resized()
    {
        // This method is where you would adjust the bounds
        of any child components.
    }

    // timerCallback: This method is called periodically by
    the Timer.
    // It forces a repaint, ensuring the CircularMeter's
    display is updated in real time.
    void CircularMeter::timerCallback()
    {
        repaint(); // Repaint the component to reflect any
        changes in value or appearance
    }

    // generatePointWithRandomness: Adds random displacement
    to a given point,
    // creating a dynamic, wavy effect on the ellipse to
    make it appear more organic.
    Point<float>
    CircularMeter::generatePointWithRandomness(Point<float>
    p, const float level)
    {
        // Displace the point's x and y coordinates by a
        random amount proportional to the level
        p.addXY(random.nextFloat() * level * 8.f,
        random.nextFloat() * level * 8.f);
        return p;
    }

```

3. In JogWheel.h

```

class JogWheel : public juce::AnimatedAppComponent //
Inherits from AnimatedAppComponent to allow animation
updates
{
public:
    // Constructor: Initializes the JogWheel with the
    given player and background color.
    // @param player Pointer to the DJAudioPlayer
    instance for controlling playback.
    // @param backgroundColor The background color of
    the JogWheel component.
    JogWheel(DJAudioPlayer* player, juce::Colour
    backgroundColor);

    // Destructor: Cleans up any resources used by the
    JogWheel.
    ~JogWheel() override;

```



```

        // paint: Handles the drawing of the JogWheel
        component.
        // This method is responsible for rendering the jog
        wheel, including its background, circles, and animated
        segments.
        void paint(juce::Graphics&) override;

        // resized: Adjusts the layout of the JogWheel and
        its child components when the component size changes.
        void resized() override;

        // update: Called by the animation system to update
        the component's state.
        // Currently, this method does not perform any
        actions but is required by the AnimatedAppComponent.
        void update() override;

private:
    DJAudioPlayer* player;           // Pointer to the
    DJAudioPlayer instance used to control audio playback
    juce::Colour backgroundColor;

    CircularMeter circularMeter;     // Circular meter
    component used to display the audio level

    double margin = 10;              // Margin around
    the jog wheel for aesthetic spacing

    // calculateSize: Determines the size of the jog
    wheel based on the component's dimensions.
    // The size is adjusted to fit within the component
    while maintaining the defined margin.
    double calculateSize();

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(JogWheel)
    // Macro to prevent copying and memory leaks
};

```

4. In JogWheel.cpp

Code Flow Explanation:

- In the constructor, the CircularMeter is initialised with a function that supplies the RMS audio level from the player (player->getRMSLevel()).
- The circular meter is added as a child component and updated at 60 frames per second for smooth animation.
- **resized:** The circular meter is positioned in the centre of the jog wheel, adjusting dynamically as the jog wheel's size changes.

```

JogWheel::JogWheel(DJAudioPlayer* _player, juce::Colour
backgroundColor)
    : player(_player),
    backgroundColor(backgroundColor),
    circularMeter([_player] { return _player-
>getRMSLevel(); }, juce::Colours::skyblue)
{

```

```

        setFramesPerSecond(60);           // Set the
        animation frame rate to 60 FPS
        addAndMakeVisible(circularMeter); // Add the
        circular meter as a visible child component
    }
    //Resized function
    void JogWheel::resized()
    {
        double size = calculateSize();
        double center = size / 2 + margin;
        int radius = size / 2;

        // Set the bounds of the circular meter in the center of
        the jog wheel
        circularMeter.setBounds(center - size * 0.25, center -
        size * 0.25, size * 0.5, size * 0.5);

    }

```

5. In DJAudioPlayer

Code Flow Explanation:

- **calculateRMSLevel:**

- This method computes the RMS level of a block of audio data, which represents the average power of the signal. The sum of the squares of the sample values is averaged, and the square root of that average gives the RMS value.

- **getRMSLevel:**

- Returns the last calculated RMS value, which is used by the CircularMeter to visually represent the audio level.

```

// calculateRMSLevel: Calculates the RMS (Root Mean
Square) level for a block of audio data.
// @param data Pointer to the audio data.
// @param numSamples Number of samples to process.
// @return The calculated RMS level.
float DJAudioPlayer::calculateRMSLevel(const float*
data, int numSamples)
{
    float sumSquares = 0.0f;

    for (int i = 0; i < numSamples; ++i)
        sumSquares += data[i] * data[i];

    return std::sqrt(sumSquares / numSamples);
}

// getRMSLevel: Returns the last calculated RMS level.
// @return The RMS level.
float DJAudioPlayer::getRMSLevel() const
{
    return rmsLevel;
}

```

vi) Vertical Meter

1. LevelMeter.h

```
/*
=====
=====

LevelMeter.h
Created: 28 Aug 2024 10:35:31pm
Author: Admin

=====
=====
*/

#pragma once

#include <JuceHeader.h> // JUCE framework header for GUI
components and utilities

//=====
//=====
// LevelMeter is a custom JUCE Component that visually
represents an audio level as a vertical bar.
// It periodically updates and redraws itself using a
Timer.
class LevelMeter : public juce::Component, //
Inherits from juce::Component to create a GUI component
public juce::Timer // Inherits from
juce::Timer to update the component at regular intervals
{
public:
    // Constructor: Initializes the LevelMeter with a
function to supply its value.
    LevelMeter(std::function<float()>&& valueFunction);

    // Destructor: Ensures proper cleanup when the
LevelMeter is destroyed.
    ~LevelMeter() override;

    // paint: Handles the drawing of the LevelMeter.
This is where the meter's visual representation is
created.
    void paint(juce::Graphics& g) override;

    // resized: Adjusts the component's layout when its
size changes.
    // Used here to set up the gradient for the meter.
    void resized() override;

    // timerCallback: Called periodically by the Timer
to update and repaint the component.
    void timerCallback() override;

private:
    // valueSupplier: A function that provides the
current value to be displayed by the meter.
```

```

// This value controls the meter's level.
std::function<float()> valueSupplier;

// gradient: A ColourGradient object used to fill
the meter with a smooth transition of colours from green
to red.
juce::ColourGradient gradient{};

// Macro to prevent copying and memory leaks
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(LevelMeter)
};

```

2. LevelMeter.cpp

Code Flow Explanation:

- **Constructor:**
 - valueSupplier: A function passed into the constructor supplies the current audio level for visualisation. This function typically provides the RMS level of the audio.
 - startTimerHz(24): A timer is set to call the timerCallback method 24 times per second, ensuring that the level meter updates in near real-time.
- **Destructor:**
 - The timer is automatically stopped when the LevelMeter is destroyed, preventing resource leaks.

2. paint Method (LevelMeter.cpp)

Overview:

The paint method is responsible for drawing the vertical level meter. The current audio level determines how much of the meter is filled.

Code Flow Explanation:

- **Background Fill:**
 - The component's bounds are slightly reduced to provide padding, and the background is filled with black for contrast.
- **Gradient Fill:**
 - The gradient, which transitions from green (low) to red (high), is applied to the filled portion of the meter.
- **Scaled Height:**
 - The current audio level, provided by valueSupplier(), is mapped to a height that reflects the meter's level, filling the rectangle from the bottom up to the corresponding height.

3. resized Method (LevelMeter.cpp)

Overview:

This method is responsible for adjusting the component's layout when its size changes and for setting up the colour gradient.

Code Flow Explanation:

- **Gradient Setup:**

- A vertical ColourGradient is created, starting with green at the bottom and transitioning to red at the top. Intermediate colours like yellow and orange are added to represent varying audio levels, making the meter more visually descriptive.

4. timerCallback Method (LevelMeter.cpp)

Overview:

This method ensures that the level meter updates regularly by calling repaint() at a set frequency.

Code Flow Explanation:

- **Repainting:**
 - The timerCallback is triggered 24 times per second by the timer, and it calls repaint() to refresh the meter display. This allows the meter to reflect the most recent audio level in real time.

```

/*
=====
=====

    LevelMeter.cpp
    Created: 28 Aug 2024 10:35:31pm
    Author: Admin

=====
=====
*/

#include <JuceHeader.h>           // JUCE framework
header
#include "LevelMeter.h"         // Header file for the
LevelMeter class

//=====
=====
// Constructor: Initializes the LevelMeter with the
// provided value-supplying function.
// It also starts a timer to regularly update the meter
// at 24Hz.
LevelMeter::LevelMeter(std::function<float()>&&
valueFunction)
    : valueSupplier(std::move(valueFunction)) // Move
the value supplier function to avoid unnecessary copies
{
    startTimerHz(24); // Start a timer that calls
timerCallback 24 times per second
}

// Destructor: Automatically stops the timer when the
LevelMeter is destroyed.
LevelMeter::~LevelMeter()
{
}

```

```

// paint: This is where the LevelMeter's visual
representation is drawn.
// The height of the filled rectangle represents the
current value supplied by valueSupplier.
void LevelMeter::paint(juce::Graphics& g)
{
    // Define the bounds of the component, slightly
    reduced to give a padding effect
    auto bounds =
getLocalBounds().toFloat().reduced(2.0f);

    // Fill the background with black to create a
    contrast with the meter's colours
    g.setColour(juce::Colours::black);
    g.fillRect(bounds);

    // Set the gradient fill, which transitions from
    green to red, to indicate the level
    g.setGradientFill(gradient);

    // Map the supplied value to a height within the
    bounds of the component
    const auto scaledY = juce::jmap(valueSupplier(),
0.0f, 0.1f, 0.0f, static_cast<float>(getHeight()));

    // Fill a rectangle from the bottom of the component
    up to the scaled height
    g.fillRect(bounds.removeFromBottom(scaledY));
}

// resized: This function is called whenever the
LevelMeter is resized.
// It sets up the gradient for the meter, transitioning
from green at the bottom to red at the top.
void LevelMeter::resized()
{
    // Define the bounds of the component
    const auto bounds = getLocalBounds().toFloat();

    // Create a vertical gradient from green (bottom) to
    red (top), with intermediate colours for yellow and
    orange
    gradient = juce::ColourGradient{
juce::Colours::green, bounds.getBottomLeft(), // Start
colour and position
juce::Colours::red,
bounds.getTopLeft(), // End colour and position
false }; // Linear
gradient

    // Add intermediate colours to the gradient
    gradient.addColour(0.3, juce::Colours::yellow);
    // Yellow at 30% height
    gradient.addColour(0.6, juce::Colours::orange);
    // Orange at 60% height
    gradient.addColour(0.8,
juce::Colours::red.brighter()); // Brighter red at 80%
height
}

// timerCallback: This method is called periodically by
the Timer.

```

```
// It forces a repaint, ensuring the LevelMeter's
display is updated in real time.
void LevelMeter::timerCallback()
{
    repaint(); // Repaint the component to reflect any
changes in value or appearance
}
```

3. In DeckGUI

Code Flow Explanation:

- **LevelMeter Declaration:**
 - In the DeckGUI header file, an instance of LevelMeter is declared as levelMeter.
- **Layout in resized Function:**
 - In the resized function of DeckGUI.cpp, the bounds of levelMeter are set. This places the meter within the user interface, ensuring that it visually aligns with other deck components.

```
//In header file
LevelMeter levelMeter; // Displays the audio level
using a meter
//In resized function of cpp file
levelMeter.setBounds(rowW + sliderWidth * 4, rowH *
13, 20, sliderHeight * 2);
```

4. In DJAudioPlayer

Code Flow Explanation:

- **LevelMeter Declaration:**
 - In the DeckGUI header file, an instance of LevelMeter is declared as levelMeter.
- **Layout in resized Function:**
 - In the resized function of DeckGUI.cpp, the bounds of levelMeter are set. This places the meter within the user interface, ensuring that it visually aligns with other deck components.

```
// calculateRMSLevel: Calculates the RMS (Root Mean
Square) level for a block of audio data.
// @param data Pointer to the audio data.
// @param numSamples Number of samples to process.
// @return The calculated RMS level.
float DJAudioPlayer::calculateRMSLevel(const float*
data, int numSamples)
{
    float sumSquares = 0.0f;

    for (int i = 0; i < numSamples; ++i)
        sumSquares += data[i] * data[i];

    return std::sqrt(sumSquares / numSamples);
```

```

    }

    // getRMSLevel: Returns the last calculated RMS level.
    // @return The RMS level.
    float DJAudioPlayer::getRMSLevel() const
    {
        return rmsLevel;
    }

```

vii) Bulb Meter

1. In LevelMeterBulb.h

Bulb class

```

//=====
//=====
// Bulb is a custom JUCE Component representing an
// individual light bulb in the level meter.
// It can be turned on or off based on the current level
// of the meter.
class Bulb : public juce::Component
{
public:
    // Constructor: Initializes the Bulb with a specific
    // colour.
    Bulb(const juce::Colour& c);

    // paint: Handles the drawing of the Bulb. The bulb
    // will be drawn as an ellipse.
    void paint(juce::Graphics& g) override;

    // setState: Sets the state of the Bulb (on or off)
    // and repaints it if the state changes.
    void setState(bool state);

private:
    // isOn: Indicates whether the bulb is currently on
    // or off.
    bool isOn = false;

    // colour: The colour of the bulb when it is on.
    juce::Colour colour;

    // Macro to prevent copying and memory leaks
    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(Bulb)
};

```

2. In LevelMeterBulb.cpp

Bulb class implementation:

Code Flow Explanation:

- **Bulb Constructor:**
 - The constructor takes a colour argument, which defines the colour the bulb will have when it is turned on.
- **paint Method:**

- If the bulb is on, it is drawn as a filled ellipse with the specified colour. A glowing effect is added by using a radial gradient.
- If the bulb is off, it is drawn in black.
- In both cases, an outline is drawn around the bulb for visual clarity.
- **setState Method:**
 - This method allows the bulb to be turned on or off based on the audio level. If the bulb's state changes, it triggers a repaint of the bulb to reflect the new state visually.

```
//=====
// Bulb Implementation

// Constructor: Initializes the Bulb with the specified
// colour.
Bulb::Bulb(const juce::Colour& c) : colour(c) {}

// paint: This is where the Bulb's visual representation
// is drawn.
// If the bulb is on, it is drawn in its specified
// colour with a glowing effect. Otherwise, it is drawn in
// black.
void Bulb::paint(juce::Graphics& g)
{
    // Define the bounds of the bulb as a square within
    // the component's bounds
    auto bounds =
        getLocalBounds().toFloat().reduced(2.0f);
    auto diameter = std::min(bounds.getWidth(),
        bounds.getHeight());
    juce::Rectangle<float> bulbBounds(bounds.getX(),
        bounds.getY(), diameter, diameter);

    if (isOn)
    {
        // Draw the bulb as a filled ellipse in the
        // specified colour
        g.setColour(colour);
        g.fillEllipse(bulbBounds);

        // Add a glowing effect around the bulb using a
        // radial gradient
        g.setGradientFill(juce::ColourGradient(colour.withAlpha(
            0.3f), bulbBounds.getCentre(),
            colour.withLightness(1.5f).withAlpha(0.f),
            {}, true));
        g.fillEllipse(bulbBounds.expanded(4.0f)); //
        // Expand the glow slightly beyond the bulb's bounds
    }
    else
    {
        // If the bulb is off, draw it as a black
        // ellipse
        g.setColour(juce::Colours::black);
        g.fillEllipse(bulbBounds);
    }
}
```

```

        // Draw an outline around the bulb, regardless of
        its state
        g.setColour(juce::Colours::black);
        g.drawEllipse(bulbBounds, 1.3f);
    }

    // setState: Updates the state of the bulb (on or off)
    and triggers a repaint if the state changes.
    void Bulb::setState(bool state)
    {
        if (isOn != state)
        {
            isOn = state;
            repaint(); // Repaint the bulb to reflect the
            new state
        }
    }

```

3. In LevelMeterBulb.h

```

//=====
// LevelMeterBulb is a custom JUCE Component that
// displays an array of Bulbs to represent the audio level.
// The bulbs light up progressively as the audio level
// increases.
class LevelMeterBulb : public juce::Component, //
    Inherits from juce::Component to create a GUI component
    public juce::Timer // Inherits from
    juce::Timer to update the component at regular intervals
{
public:
    // Constructor: Initializes the LevelMeterBulb with
    a function to supply its value.
    LevelMeterBulb(std::function<float()>&&
    valueFunction);

    // Destructor: Ensures proper cleanup when the
    LevelMeterBulb is destroyed.
    ~LevelMeterBulb() override;

    // paint: Handles the drawing of the LevelMeterBulb.
    This controls which bulbs are lit.
    void paint(juce::Graphics& g) override;

    // resized: Adjusts the component's layout when its
    size changes.
    // Used here to create and position the Bulbs.
    void resized() override;

    // timerCallback: Called periodically by the Timer
    to update and repaint the component.
    void timerCallback() override;

private:
    // valueSupplier: A function that provides the
    current value to be displayed by the meter.
    // This value controls how many Bulbs are lit.
    std::function<float()> valueSupplier;

```

```

        // bulbs: A vector of unique pointers to Bulb
        objects, representing the individual lights in the
        meter.
        std::vector<std::unique_ptr<Bulb>> bulbs;

        // totalNumberOfBulbs: The total number of Bulbs in
        the LevelMeterBulb component.
        const int totalNumberOfBulbs = 10;

        // Macro to prevent copying and memory leaks
        JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(LevelMeterB
        ulb)
        };

```

4. In LevelMeterBulb.cpp LevelMeterBulb

Code Flow Explanation:

- **Constructor:**
 - The constructor takes a function (valueSupplier) that provides the current RMS audio level. This function is used to determine how many bulbs should be lit.
 - A timer is started that updates the meter 24 times per second, ensuring real-time visual feedback.
- **paint Method:**
 - The paint method calculates how many bulbs should be lit based on the current value (supplied by valueSupplier). For each bulb, it checks whether it should be on or off and updates its state accordingly.
- **resized Method:**
 - This method handles the layout of the bulbs within the meter. It calculates the height for each bulb and arranges them vertically. A gradient from green to red is applied, representing a transition from low to high audio levels.
- **timerCallback Method:**
 - The timerCallback method is called regularly by the timer to repaint the meter and update the state of the bulbs in real time.

```

//=====
// LevelMeterBulb Implementation

// Constructor: Initializes the LevelMeterBulb with the
// provided value-supplying function.
// It also starts a timer to regularly update the meter
// at 24Hz.
LevelMeterBulb::LevelMeterBulb(std::function<float()>&&
valueFunction)
    : valueSupplier(std::move(valueFunction)) // Move
the value supplier function to avoid unnecessary copies

```

```

{
    startTimerHz(24); // Start a timer that calls
timerCallback 24 times per second
}

// Destructor: Automatically stops the timer when the
LevelMeterBulb is destroyed.
LevelMeterBulb::~LevelMeterBulb()
{
}

// paint: This is where the LevelMeterBulb's visual
representation is drawn.
// The function checks the current value and determines
which bulbs should be lit.
void LevelMeterBulb::paint(juce::Graphics& g)
{
    // Map the supplied value to a range between 0 and 1
    auto value = juce::jmap(valueSupplier(), 0.0f, 0.1f,
0.0f, 1.0f);

    // Determine which bulbs should be lit based on the
current value
    for (int i = 0; i < totalNumberOfBulbs; ++i)
    {
        bool shouldBeOn = value >= static_cast<float>(i
+ 1) / totalNumberOfBulbs;
        bulbs[i]->setState(shouldBeOn); // Update each
bulb's state
    }
}

// resized: This function is called whenever the
LevelMeterBulb is resized.
// It creates and positions the bulbs based on the
component's new size.
void LevelMeterBulb::resized()
{
    // Define the bounds of the component
    auto bounds = getLocalBounds().toFloat();

    // Create a vertical gradient for the bulbs,
transitioning from green at the bottom to red at the top
    juce::ColourGradient gradient(juce::Colours::green,
bounds.getBottomLeft(),
    juce::Colours::red, bounds.getTopLeft(), false);
    gradient.addColour(0.5, juce::Colours::yellow); //
Add yellow at the middle of the gradient

    // Calculate the height for each bulb based on the
total number of bulbs
    auto bulbHeight = getHeight() / totalNumberOfBulbs;
    auto bulbBounds = getLocalBounds();
    bulbs.clear(); // Clear any existing bulbs

    // Create and position each bulb
    for (int i = 0; i < totalNumberOfBulbs; ++i)
    {
        // Create a new Bulb with a colour from the
gradient, based on its position in the stack
        auto bulb =
std::make_unique<Bulb>(gradient.getColourAtPosition(stat
ic_cast<double>(i) / totalNumberOfBulbs));

```

```

        addAndMakeVisible(bulb.get()); // Add the bulb
        to the component
        bulb-
        >setBounds(bulbBounds.removeFromBottom(bulbHeight)); //
        Position the bulb
        bulbs.push_back(std::move(bulb)); // Store the
        bulb in the vector
    }
}

// timerCallback: This method is called periodically by
// the Timer.
// It forces a repaint, ensuring the LevelMeterBulb's
// display is updated in real time.
void LevelMeterBulb::timerCallback()
{
    repaint(); // Repaint the component to reflect any
    changes in value or appearance
}

```

5. In DeckGUI

Code Flow Explanation:

- **Initialization in Header:**
 - In the DeckGUI header, an instance of LevelMeterBulb is declared as levelMeterBulb.
- **Layout in resized Function:**
 - In the resized function of DeckGUI.cpp, the position and size of the levelMeterBulb are set to ensure it is visually aligned with other deck components.

```

//initialize in header
LevelMeterBulb levelMeterBulb; // Displays the
audio level using a series of bulbs
//resized in cpp
levelMeterBulb.setBounds(rowW + sliderWidth * 4 -
30, rowH * 13, 20, sliderHeight * 2);

```

6. DJAudioPlayer

Code Flow Explanation:

- **calculateRMSLevel Method:**
 - This method computes the RMS level of a block of audio data. RMS is a measure of the audio signal's power and is used to control how many bulbs light up in the LevelMeterBulb.
- **getRMSLevel Method:**
 - This method returns the most recently calculated RMS level. The LevelMeterBulb uses this value (through valueSupplier) to determine the number of bulbs that should be lit.

```

// calculateRMSLevel: Calculates the RMS (Root Mean
Square) level for a block of audio data.
// @param data Pointer to the audio data.
// @param numSamples Number of samples to process.
// @return The calculated RMS level.
float DJAudioPlayer::calculateRMSLevel(const float*
data, int numSamples)
{
    float sumSquares = 0.0f;

    for (int i = 0; i < numSamples; ++i)
        sumSquares += data[i] * data[i];

    return std::sqrt(sumSquares / numSamples);
}

// getRMSLevel: Returns the last calculated RMS level.
// @return The RMS level.
float DJAudioPlayer::getRMSLevel() const
{
    return rmsLevel;
}

```

viii) Tracks Load in table component

1. TrackEntry.h

```

/*
=====
=====

TrackEntry.h
Created: 3 Aug 2024 9:02:48am
Author: Admin

=====
=====
*/

#pragma once

#include <JuceHeader.h> // JUCE framework header for
URL, String, and other utilities
#include <string> // Standard library header for
string manipulation

//=====
=====
// TrackEntry is a class representing an audio track
with associated metadata,
// such as its URL, title, duration, and whether it is
marked as a favourite.
class TrackEntry
{
public:
    // Constructor: Initializes the TrackEntry with an
audio URL and its duration.

```

```

// The title is derived from the file name of the
URL, and the track is not a favourite by default.
TrackEntry(URL _audioURL, juce::String _duration);

// Member variables:

juce::URL audioURL;        // The URL of the audio
file associated with this track
juce::String title;        // The title of the track,
derived from the file name of the audioURL
juce::String duration;     // The duration of the
track, typically in "mm:ss" format
bool isFavourite;         // A flag indicating
whether this track is marked as a favourite

// toString: Returns a string representation of the
TrackEntry, including the title, duration, and favourite
status.
juce::String toString();

// toCSV: Returns a CSV-formatted string
representation of the TrackEntry, including the audio
URL, duration, and favourite status.
juce::String toCSV();

// Operator overloading:

// operator==: Compares two TrackEntry objects for
equality based on their audioURL and title.
bool operator==(const TrackEntry& other) const;
};

```

2. TrackEntry.cpp

Code Flow Explanation:

- **TrackEntry Constructor:**
 - The constructor takes the URL of the audio file and its duration. The title of the track is automatically extracted from the URL's file name. The track is marked as "not a favourite" by default.
- **toString / toCSV Methods:**
 - toString returns a formatted string of the track's details (title, duration, favourite status), while toCSV formats the data as a CSV string, which is used for saving and reading the playlist from a file.
- **operator== Method:**
 - This method compares two TrackEntry objects based on their audio URLs and titles to check if they represent the same track.

```

/*
=====

TrackEntry.cpp
Created: 3 Aug 2024 9:02:48am
Author: Admin

```

```

=====
=====
*/

#include "TrackEntry.h" // Include the header file for
the TrackEntry class
#include <string>         // Standard library header for
string manipulation
#include <cmath>          // Standard library header for
mathematical functions (not used in current code)

//=====
=====
// Constructor: Initializes the TrackEntry object with
the given audio URL and duration.
// The title is extracted from the file name of the
audio URL, and the track is not a favourite by default.
TrackEntry::TrackEntry(URL _audioURL, juce::String
_duration)
    : audioURL(_audioURL),
// Initialize the audioURL member variable

title(URL::removeEscapeChars(_audioURL.getFileName())),
// Derive the title from the URL's file name
duration(_duration),
// Initialize the duration with the provided value
isFavourite(false)
// Set the isFavourite flag to false by default
{
}

// toString: Returns a human-readable string
representation of the TrackEntry.
// The string includes the title, duration, and whether
the track is marked as a favourite.
juce::String TrackEntry::toString()
{
    return "Title: " + title + ", Duration: " + duration
+ ", Favourite: " + (isFavourite ? "Yes" : "No");
}

// toCSV: Returns a CSV-formatted string representation
of the TrackEntry.
// The string includes the audio URL, duration, and
whether the track is marked as a favourite.
juce::String TrackEntry::toCSV()
{
    return audioURL.toString(true) + "," + duration +
", " + (isFavourite ? "Yes" : "No");
}

// operator==: Compares two TrackEntry objects for
equality based on their audioURL and title.
// Two tracks are considered equal if they have the same
URL and title, regardless of other properties.
bool TrackEntry::operator==(const TrackEntry& other)
const
{
    return audioURL == other.audioURL && title ==
other.title;
}

```


3. CSVTrack.h

```
/*
=====
=====

    CSVtrack.h
    Created: 3 Aug 2024 9:02:21am
    Author: Admin

=====
=====
*/

#pragma once

#include "JuceHeader.h" // JUCE framework header
                        // for file handling, strings, and other utilities
#include "TrackEntry.h" // Include TrackEntry
                        // class for handling track data
#include <string>         // Standard library
                        // header for string manipulation
#include <vector>        // Standard library
                        // header for using vectors

//=====
//=====
// CSVtrack is a utility class for handling the reading
// and writing of TrackEntry objects
// to and from CSV (Comma-Separated Values) files. It
// provides methods to read from and
// write to CSV files, as well as to tokenize CSV lines
// into TrackEntry components.
class CSVtrack
{
public:
    // Constructor: Initializes a CSVtrack object.
    CSVtrack();

    // readCSV: Reads a CSV file to retrieve TrackEntry
    // objects.
    // Takes a pointer to a JUCE File object
    // representing the CSV file.
    // Returns a vector of TrackEntry objects parsed
    // from the CSV file.
    static std::vector<TrackEntry> readCSV(File*
resourceFile);

    // writeCSV: Writes a vector of TrackEntry objects
    // to a CSV file.
    // Takes a pointer to a JUCE File object
    // representing the CSV file and a pointer to a vector of
    // TrackEntry objects.
    // If the file does not exist, it creates a new
    // file.
```

```

        static void writeCSV(File* resourceFile,
std::vector<TrackEntry>* trackEntries);

        // tokenise: Splits a CSV line into tokens based on
the separator character.
        // Takes a JUCE String representing a CSV line and a
character separator.
        // Returns a vector of JUCE Strings representing the
tokens.
        static std::vector<juce::String>
tokenise(juce::String csvLine, char separator);

private:
        // stringsToTrackEntry: Converts a vector of JUCE
Strings into a TrackEntry object.
        // Takes a vector of JUCE Strings representing the
CSV tokens.
        // Returns a TrackEntry object constructed from the
tokens.
        static TrackEntry
stringsToTrackEntry(std::vector<juce::String> strings);
};

```

4. CSVTrack.cpp

Code Flow Explanation:

- **readCSV Method:**
 - This method reads a CSV file and converts each line into a TrackEntry object. It tokenizes each line, extracts the necessary metadata (audio URL and duration), and constructs a TrackEntry object. If the file doesn't exist or the data is malformed, an error message is displayed.
- **writeCSV Method:**
 - This method writes the TrackEntry objects from a vector to a CSV file. It ensures safe writing by using a TemporaryFile to avoid partial or failed writes, truncating the existing file and replacing it with the new content.
- **tokenise Method:**
 - This utility method splits a CSV line into individual tokens based on a separator (usually a comma). The tokens are then used to create a TrackEntry object.

```

/*
=====

CSVtrack.cpp
Created: 3 Aug 2024 9:02:21am
Author: Admin

```

```

=====
=====
*/

#include "CSVtrack.h"          // Include the header
                               // file for the CSVtrack class
#include <iostream>             // Standard library
                               // header for input/output operations
#include <fstream>              // Standard library
                               // header for file stream operations

//=====
//=====
// Constructor: Initializes a CSVtrack object.
// Currently, this constructor does not perform any
// operations but is defined for completeness.
CSVtrack::CSVtrack()
{
}

/* reading from file to retrieve TrackEntry objects
based on CSV strings in file */
// readCSV: Reads the contents of a CSV file and parses
it into a vector of TrackEntry objects.
// Takes a pointer to a JUCE File object representing
the CSV file.
// Returns a vector of TrackEntry objects parsed from
the file.
std::vector<TrackEntry> CSVtrack::readCSV(File*
resourceFile)
{
    // Initialize a vector to store the TrackEntry
    objects to be returned
    std::vector<TrackEntry> entries;

    // Check if the file exists, and output an error
    message if it does not
    if (!resourceFile->existsAsFile())
        std::cout << "File does not exist" << std::endl;

    // Create a FileInputStream to read the file's
    contents
    // The input stream is created by dereferencing the
    resourceFile pointer
    FileInputStream input(*resourceFile);

    // Check if the input stream was opened
    successfully, and output an error message if not
    if (!input.openedOk())
        std::cout << "Error getting input" << std::endl;
    else
    {
        // Read the file line by line until the end is
        reached
        while (!input.isExhausted())
        {
            try
            {
                // Read the next line from the file
                juce::String line =
                input.readLine(); // '%20' represents spaces in URLs
            }
            catch (...)
            {
                // Handle error
            }
        }
    }
}

```

```

        // Convert the CSV line to a TrackEntry
        object
            // The line is tokenized, and special
            characters are removed
            TrackEntry entry =
stringsToTrackEntry(tokenise(URL::removeEscapeChars(line
), ', '));

        // Add the TrackEntry object to the
        vector
            entries.push_back(entry);
        }
        catch (const std::exception& e)
        {
            // Handle any exceptions that occur
            during parsing, typically due to malformed CSV data
            std::cout << "CSVtrack::readCSV bad
data" << std::endl;
        }
    }

    // Output the number of tracks read from the CSV
    file
    std::cout << "CSVtrack::readCSV read " <<
entries.size() << " tracks." << std::endl;

    // Return the vector of TrackEntry objects
    return entries;
}

/* for save function: write <TrackEntry> data to
existing file, if not creates new file to store
<TrackEntry> data as CSV strings */
// writeCSV: Writes a vector of TrackEntry objects to a
CSV file.
// Takes a pointer to a JUCE File object representing
the CSV file and a pointer to a vector of TrackEntry
objects.
// If the file does not exist, it creates a new file.
void CSVtrack::writeCSV(File* resourceFile,
std::vector<TrackEntry>* trackEntries)
{
    // Check if the file exists, and output an error
    message if it does not
    if (!resourceFile->existsAsFile())
        std::cout << "File does not exist" << std::endl;

    // Create a TemporaryFile to write data, which
    ensures safe writing by avoiding partial writes
    TemporaryFile tempFile(*resourceFile);

    // Create a FileOutputStream to write the data to
    the file
    FileOutputStream output(tempFile.getFile());

    // Check if the output stream was opened
    successfully, and output an error message if not
    if (!output.openedOk())
        std::cout << "Error getting output" <<
std::endl;
    else
    {

```

```

        // Reset the output stream position to the
        beginning of the file to prevent appending data
        output.setPosition(0);
        output.truncate(); // Truncate the file to
        remove any existing data

        // Iterate over the TrackEntry vector and write
        each entry as a CSV line
        for (int i = 0; i < trackEntries->size(); i++)
        {
            // Convert each TrackEntry object to a CSV-
            formatted string
            juce::String entry =
            (*trackEntries)[i].toCSV();

            // Write the CSV string to the file
            output.writeText(entry + "\n", false, false,
            nullptr);

            // Flush the output stream to ensure data is
            written to disk
            output.flush();
        }

        // Replace the original file with the temporary
        file
        bool success =
        tempFile.overwriteTargetFileWithTemporary();

        // Output a message indicating whether the file
        write was successful
        if (success)
        {
            std::cout << "File write was successful." <<
            std::endl;
        }
    }

    /* reads line in file one by one and retrieve available
    tokens for TrackEntry objects */
    // tokenise: Splits a CSV line into tokens based on the
    separator character.
    // Takes a JUCE String representing a CSV line and a
    character separator.
    // Returns a vector of JUCE Strings representing the
    tokens.
    std::vector<juce::String>
    CSVtrack::tokenise(juce::String csvLine, char separator)
    {
        // Initialize a vector to store the tokens to be
        returned
        std::vector<juce::String> tokens;
        signed int start, end;
        juce::String token;

        // Convert the JUCE String to a standard string for
        easier manipulation with standard library functions
        std::string csvLineStdString =
        csvLine.toStdString(); // std::string

        // Set the start point for tokenization, ignoring
        leading separator characters

```

```

        start =
int(csvLineStdString.find_first_not_of(separator, 0));
// call std::string functions

        do
        {
            // Find the end of the current token by locating
            the next separator character
            end =
int(csvLineStdString.find_first_of(separator, start));
// call std::string functions

            // If the start point is at the end of the line,
            break out of the loop
            if (start == csvLine.length() || start == end)
                break;

            // Extract the token from the line
            if (end >= 0)
                token = csvLineStdString.substr(start, end -
start); // std::string to juce::String
            else
                token = csvLineStdString.substr(start,
csvLine.length() - start); // std::string to
juce::String

            // Add the extracted token to the vector
            tokens.push_back(token); // juce::String

            // Update the start point for the next iteration
            start = end + 1;

        } while (end > 0);

        // Return the vector of tokens
        return tokens;
    }

    /* reading line from csv file -> TrackEntry object */
    // stringsToTrackEntry: Converts a vector of JUCE
    Strings into a TrackEntry object.
    // Takes a vector of JUCE Strings representing the CSV
    tokens.
    // Returns a TrackEntry object constructed from the
    tokens.
    TrackEntry
    CSVtrack::stringsToTrackEntry(std::vector<juce::String>
tokens)
    {
        URL audioURL;
        juce::String duration;

        // If the number of tokens is not exactly 2 (URL and
        duration), throw an exception
        if (tokens.size() != 2)
            throw std::exception{};

        // Parse the tokens to create a TrackEntry object
        try
        {
            audioURL = URL{ tokens[0] }; // Convert the
            first token to a URL

```

```

        duration = tokens[1];           // Assign the
second token as the duration
    }
    catch (const std::exception& e)
    {
        // Handle any exceptions that occur during
parsing
        std::cout << "CSVtrack::stringsToTE Bad URL " <<
tokens[0] << std::endl;
        std::cout << "CSVtrack::stringsToTE Bad double "
<< tokens[1] << std::endl;
        throw; // Re-throw the exception to be handled
by the calling function
    }

    // Return a new TrackEntry object constructed from
the parsed URL and duration
    TrackEntry entry{ audioURL, duration };
    return entry;
}

```

5. In MainComponent

Code Flow Explanation:

- **Initialization in Header:**
 - The PlaylistComponent is initialized with references to the two DeckGUI instances (for loading tracks into decks).
- **Resizing in resized Method:**
 - The playlist occupies the lower quarter of the UI. The tableComponent inside the playlist is dynamically resized based on the overall component size.

```

//In header file
PlaylistComponent playlistComponent{ &deckGUI1,
&deckGUI2 };

//In cpp file
void MainComponent::resized()
{
    int unit = getWidth() / 5;

    // Set the bounds of the PlaylistComponent.
    playlistComponent.setBounds(0, getHeight() * 3 / 4,
getWidth(), getHeight() * 1 / 4);
}

```

6. In PlaylistComponent.h

Code Flow Explanation:

- **addToTrackEntries Method:**
 - This method adds a new track to the trackEntries vector and updates the table to display the new track. It triggers an asynchronous UI

update, ensuring the table is refreshed without blocking the main thread.

- **getNumRows and paintCell Methods:**
 - getNumRows returns the number of tracks to be displayed in the table.
 - paintCell handles drawing the cells in the table, displaying track titles and durations based on the column and row.
- **buttonClicked Method (Handling Insert Button):**
 - When the insert button is clicked, a file chooser is launched to allow the user to select audio files. If a selected file is already in the playlist, it is skipped, and a message is shown to the user to avoid duplicates.
- **filesDropped Method (Handling Drag-and-Drop):**
 - This method is called when files are dragged and dropped onto the playlist. The dropped files are added to the playlist, and duplicates are skipped, just like when adding files via the insert button.

```
class PlaylistComponent : public juce::Component,
    public juce::TextEditor::Listener, // Interface to
    handle text editor events (e.g., search bar input)
    public juce::Button::Listener, // Interface to
    handle button click events (e.g., insert, filter
    buttons)
    public juce::TableListBoxModel, // Interface to
    handle table data (e.g., track list table)
    public LookAndFeel_V2, // Used for customising the
    look and feel of the component (e.g., buttons, sliders)
    public juce::FileDragAndDropTarget // Interface to
    handle file drag-and-drop events for track imports
{
public:
    // Constructor for the PlaylistComponent, takes
    pointers to DeckGUI1 and DeckGUI2 to manage deck
    interactions
    PlaylistComponent(DeckGUI* _deckGUI1, DeckGUI*
    _deckGUI2);

    // Destructor for cleaning up resources when
    PlaylistComponent is destroyed
    ~PlaylistComponent() override;

    // Override paint method to handle custom drawing of
    the component (e.g., background, text)
    void paint(juce::Graphics&) override;

    // Override resized method to handle dynamic
    resizing and repositioning of child components
    void resized() override;

    // Override buttonClicked method to handle click
    events for buttons (insert, filter)
    void buttonClicked(Button* button) override;

    // Override getNumRows method to return the number
    of rows in the table (tracks to display)
    int getNumRows() override;
```



```

        // Override paintRowBackground method to handle
        custom painting of the row background (e.g., colour when
        selected)
        void paintRowBackground(juce::Graphics& g, int
        rowNumber, int width, int height, bool rowIsSelected)
        override;

        // Override paintCell method to handle custom
        painting of the cell contents (e.g., track title,
        duration)
        void paintCell(Graphics& g, int rowNumber, int
        columnId, int width, int height, bool rowIsSelected)
        override;

        // Override refreshComponentForCell to update custom
        components in table cells (e.g., buttons, waveform
        display)
        Component* refreshComponentForCell(int rowNumber,
        int columnId, bool isRowSelected, Component*
        existingComponentToUpdate) override;

        /* Method to add a track to the <trackEntries>
        vector and update the table component with the new track
        */
        void addToTrackEntries(URL audioURL, juce::String
        duration);

        /* Method to print all TrackEntry objects in the
        trackEntries vector (for debugging or logging) */
        void printTrackEntries();

        /* Method to get a pointer to the trackEntries
        vector, used for external access */
        std::vector<TrackEntry*> getTrackEntries();

        // Public member variables for DeckGUI to access
        TableListBox tableComponent; // The table that
        displays the list of tracks
        bool trackFound = false; // Boolean flag indicating
        whether a track search result is displayed
        int trackCounter; // Integer counter to track the
        currently playing track index

        // Override method to check if file dragging is
        accepted, always return true to allow all file types
        bool isInterestedInFileDrag(const juce::StringArray&
        files);

        // Override method to handle file drop events (e.g.,
        when files are dropped into the component)
        void filesDropped(const juce::StringArray& files,
        int x, int y);

        // Utility method to check if a string contains
        another string, ignoring case (helper for search
        functionality)
        bool containsIgnoreCase(const juce::String& str,
        const juce::String& searchString);

private:
        // Vector to store the loaded tracks (TrackEntry
        objects) and save them

```

```

std::vector<TrackEntry> trackEntries;

// Vector to temporarily store search results
// (TrackEntry objects)
std::vector<TrackEntry> trackEntriesFound;

// Pointers to the DeckGUI components (Deck 1 and
// Deck 2) created in the MainComponent
DeckGUI* deckGUI1; // Pointer to DeckGUI1
DeckGUI* deckGUI2; // Pointer to DeckGUI2

// Boolean flags to indicate whether a track is

// Buttons for adding tracks, filtering tracks (All
// or Favourites)
TextButton insertButton{ "Insert" }; // Button for
inserting tracks into the playlist

// File chooser component for selecting and
// importing tracks into the playlist
juce::FileChooser mChooser{ "Select a file..." };

// Audio format manager for handling different audio
// file formats (e.g., MP3, WAV)
juce::AudioFormatManager formatManager;

// Audio thumbnail cache for efficient waveform
// rendering of audio tracks
juce::AudioThumbnailCache thumbnailCache{ 100 };
}

```

7. Constructor in PlaylistComponent.cpp

```

PlaylistComponent::PlaylistComponent(DeckGUI* _deckGUI1,
DeckGUI* _deckGUI2)

// Initialising member variables in constructor
: trackFound(false), // Boolean to indicate if track
is found during search
deckGUI1(_deckGUI1), // Deck 1 object (pass pointer
from GUI)
deckGUI2(_deckGUI2), // Deck 2 object (pass pointer
from GUI)
filterManager(trackEntries) // Instantiate
PlaylistFilterManager to manage track filtering
{
// Register audio formats (e.g. .mp3, .wav) with
format manager to handle various file types
formatManager.registerBasicFormats();

// Adding buttons for track insert and filter
actions to the GUI and making them visible
addAndMakeVisible(insertButton);

// Adding and defining columns for the playlist
table (e.g. title, duration, preview, etc.)
tableComponent.getHeader().addColumn("Track title",
1, 200);
}

```

```

        tableComponent.getHeader().addColumn("Duration", 2,
100);
        tableComponent.getHeader().addColumn("Add to
favourite", 3, 100);
        tableComponent.getHeader().addColumn("Preview", 4,
200);
        tableComponent.getHeader().addColumn("Delete", 5,
100);
        tableComponent.getHeader().addColumn("Left", 6,
100);
        tableComponent.getHeader().addColumn("Right", 7,
100);

        // Set the table model to this class to manage data
for the table
        tableComponent.setModel(this);
        addAndMakeVisible(tableComponent);

        // Adding event listeners to the insert and filter
buttons (action listeners handle button clicks)
        insertButton.addListener(this);
    }

```

8. In destructor of PlaylistComponent

```

// Destructor to handle clean-up of resources and
pointers
PlaylistComponent::~PlaylistComponent()
{
    // Reset the table model to null to prevent invalid
access if PlaylistComponent is destroyed before table
    tableComponent.setModel(nullptr);
}

```

9. In resized function

```

// resized method is called when the component size is
adjusted, used to position child components dynamically
void PlaylistComponent::resized()
{
    // Calculate row height (rowH) and column width
(colW) based on the component size
    double rowH = getHeight() / 5;
    double colW = getWidth() / 10;

    // Set the bounds for the playlist table (fills most
of the component's area)
    tableComponent.setBounds(0, rowH * 0.8, getWidth(),
getHeight());

    // Positioning search bar and buttons dynamically
based on calculated row height and column width
    search.setBounds(getWidth() * 0.6, rowH * 0.1,
getWidth() * 0.2, rowH * 0.6);
    insertButton.setBounds(0, rowH * 0.2, colW, rowH *
0.6);
}

```

10. Print all tracks

```

// Get the number of rows in the playlist table, which
// is determined by the number of filtered tracks
int PlaylistComponent::getNumRows()
{
    // Fetch the filtered tracks from the filter manager
    // (all tracks or favourites based on the filter)
    auto& entries = filterManager.getFilteredTracks();

    // Return the size of the filtered track list, cast
    // to an integer
    return static_cast<int>(entries.size());
}

// Paint the contents of individual cells in the
// playlist table
void PlaylistComponent::paintCell(Graphics& g, int
rowNumber, int columnId, int width, int height, bool
rowIsSelected)
{
    // Get the filtered track entries to display in the
    // table
    auto& entries = filterManager.getFilteredTracks();

    // Ensure the row number is within valid bounds of
    // the track list
    if (rowNumber >= 0 && rowNumber < entries.size())
    {
        // Get the track entry for the current row
        const TrackEntry& entry = entries[rowNumber];

        // If the current column is the Title column (ID
        // 1), draw the track title
        if (columnId == 1) // Title column
            g.drawText(entry.title, 2, 0, width - 4,
            height, juce::Justification::centredLeft, true);
        // If the current column is the Duration column
        // (ID 2), draw the track duration
        else if (columnId == 2) // Duration column
            g.drawText(entry.duration, 2, 0, width - 4,
            height, juce::Justification::centred, true);
    }
}

/* Adds a new track to the trackEntries vector and
// updates the table to reflect the new entry */
void PlaylistComponent::addToTrackEntries(URL audioURL,
juce::String duration)
{
    // Perform the track addition asynchronously to avoid
    // blocking the UI thread
    juce::MessageManager::callAsync([this, audioURL,
    duration]()
    {
        // Add the new track (with URL and duration) to the
        // trackEntries vector
        trackEntries.push_back(TrackEntry{ audioURL, duration
    });

        // Refresh the filter to show all tracks (including the
        // newly added one)
        filterManager.filterAll();

        // Update the table to include the new track

```

```

tableComponent.updateContent();

// Repaint the UI to reflect the changes
repaint();
});
}

```

11. Load tracks through 'Insert' button and prevent duplicate file to be loaded

```

// buttonClicked method is called when any button in the
// component is clicked (insert, filter, etc.)
void PlaylistComponent::buttonClicked(juce::Button*
button)
{
    // If insertButton is clicked, launch file chooser
    // to select audio files to add to the playlist
    if (button == &insertButton)
    {
        auto fileChooserFlags =
            FileBrowserComponent::canSelectFiles |
            FileBrowserComponent::canSelectMultipleItems; // Enable
            multi-select

        // Launch the file chooser asynchronously and
        // process selected files
        mChooser.launchAsync(fileChooserFlags,
        [this](const FileChooser& chooser)
        {
            auto chosenFiles = chooser.getResults();
            // Get selected files as an array
            juce::String alreadyLoadedTracks; //
            String to store names of already loaded files

            if (chosenFiles.size() > 0)
            {
                // Iterate through the selected
                files
                for (const File& chosenFile :
                chosenFiles)
                {
                    if (chosenFile.existsAsFile())
                    // Ensure the file exists
                    {
                        URL audioURL = URL{
                        chosenFile }; // Create a URL object for the audio file

                        // Check if the track is
                        already in the playlist
                        if
                        (std::any_of(trackEntries.begin(), trackEntries.end(),
                        [&audioURL](const
                        TrackEntry& entry) { return entry.audioURL == audioURL;
                        })))
                        {
                            alreadyLoadedTracks +=
                            audioURL.getFileName() + "\n"; // Add to duplicate list
                        }
                        else
                        {
                            // Add the new track to
                            the playlist (without loading it into deckGUI)
                        }
                    }
                }
            }
        });
    }
}

```

```

addToTrackEntries(audioURL, deckGUI1->player-
>getTrackDurationWithoutLoadingIntoDeck(audioURL));
    }
    }

    // If duplicates were found, show a
    message box to notify the user
    if (!alreadyLoadedTracks.isEmpty())
    {

juce::AlertWindow::showMessageBoxAsync(juce::AlertWindow
::WarningIcon,

                                "Duplicate Tracks",
                                "The following tracks were
already loaded and not added again:\n" +
alreadyLoadedTracks,

                                "OK");
    }
    });
}

```

12. Multiple files to drag and drop in PlaylistComponent

```

// Function to determine if the component is interested
in file drag-and-drop events
// Returns true to allow file drag-and-drop
functionality
bool PlaylistComponent::isInterestedInFileDrag(const
juce::StringArray& files)
{
    // Accept all file types for dragging, or add
    additional checks for specific file formats
    return true;
}

// Function to handle file drop events (e.g., when files
are dropped onto the playlist component)
void PlaylistComponent::filesDropped(const
juce::StringArray& files, int x, int y)
{
    juce::String alreadyLoadedTracks; // String to store
tracks that are already loaded

    // Iterate through each file dropped onto the
component
    for (const auto& filePath : files)
    {
        juce::File droppedFile(filePath); // Create a
File object from the file path

        // Check if the dropped file exists
        if (droppedFile.existsAsFile())
        {
            juce::URL audioURL{ droppedFile }; // Create
a URL object for the dropped file

```

```

        // Check if the track is already loaded in
        the playlist
        if (std::any_of(trackEntries.begin(),
            trackEntries.end(),
            [&audioURL](const TrackEntry& entry) {
                return entry.audioURL == audioURL; }))
        {
            // If the track is already loaded, add
            its file name to the list of already loaded tracks
            alreadyLoadedTracks +=
            audioURL.GetFileName() + "\n";
        }
        else
        {
            // If the track is not already loaded,
            add it to the trackEntries vector
            addToTrackEntries(audioURL, deckGUI1->player->
            getTrackDurationWithoutLoadingIntoDeck(audioURL));
        }
    }

    // If any tracks were already loaded, display a
    warning message to the user
    if (!alreadyLoadedTracks.isEmpty())
    {
        juce::AlertWindow::showMessageBoxAsync(juce::AlertWindow
        ::WarningIcon,
            "Duplicate Tracks",
            "The following tracks were already loaded
            and not added again:\n" + alreadyLoadedTracks,
            "OK");
    }
}

```

13. In DJAudioPlayer

- **getTrackDurationWithoutLoadingIntoDeck Method:**
 - This method reads the audio file using an AudioFormatReader and calculates the track's length in seconds. The result is formatted as "MM " and returned as a string for display in the playlist.

```

// getTrackDurationWithoutLoadingIntoDeck: Returns the
// track duration without initializing the transportSource.
// This method is useful for getting the duration
// without loading the track into the player.
// @param audioURL The URL of the audio file.
// @return The track duration as a string.
juce::String
DJAudioPlayer::getTrackDurationWithoutLoadingIntoDeck(UR
L audioURL)
{
    juce::String duration;
}

```

```

        std::unique_ptr<juce::AudioFormatReader>
        reader(formatManager.createReaderFor(audioURL.createInputStream(false)));

        if (reader != nullptr) // Good file!
        {
            double lengthInSeconds = reader->lengthInSamples
            / reader->sampleRate;
            int minutes = static_cast<int>(lengthInSeconds)
            / 60;
            int seconds = static_cast<int>(lengthInSeconds)
            % 60;

            juce::String formattedMinutes =
            juce::String(minutes);
            juce::String formattedSeconds =
            juce::String(seconds).paddedLeft('0', 2);

            duration = formattedMinutes + ":" +
            formattedSeconds;
        }
        else
        {
            DBG("Error: Could not load reader for the
            file.");
        }

        return duration;
    }

```

ix) Preview In TableComponent

1. In PlaylistComponent constructor

Code Flow Explanation:

- In the constructor, the columns for the playlist table are added, including "Track title," "Duration," "Preview," and others.
- Each column is given a unique ID, which will be used later to determine how to render the contents of each column.
- The "Preview" column is given an ID of 4, which is used to render the waveform display in that cell.
- The tableComponent is set to use PlaylistComponent as its model, meaning all data (track information) and behavior (interactivity) are handled in this class.

```

        // Adding and defining columns for the playlist
        table (e.g. title, duration, preview, etc.)
        tableComponent.getHeader().addColumn("Track
        title", 1, 200);
        tableComponent.getHeader().addColumn("Duration",
        2, 100);
        tableComponent.getHeader().addColumn("Add to
        favourite", 3, 100);
        tableComponent.getHeader().addColumn("Preview",
        4, 200);

```



```

        tableComponent.getHeader().addColumn("Delete", 5,
        100);
        tableComponent.getHeader().addColumn("Left", 6,
        100);
        tableComponent.getHeader().addColumn("Right", 7,
        100);

        // Set the table model to this class to manage
        data for the table
        tableComponent.setModel(this);
        addAndMakeVisible(tableComponent);

```

2. Title and duration

Code Flow Explanation:

- The paintCell method is responsible for rendering the contents of each cell in the table.
- The function checks which column is being rendered using the columnId.
 - **Title Column (ID 1):**
 - The track's title is drawn in the first column (columnId == 1), using the g.drawText() method.
 - The text is left-aligned for better readability.
 - **Duration Column (ID 2):**
 - The track's duration is drawn in the second column (columnId == 2), and the text is centred.
- This method only handles drawing static information (like text) directly in the table cells.

```

// Paint the contents of individual cells in the
playlist table
void PlaylistComponent::paintCell(Graphics& g, int
rowNumber, int columnId, int width, int height, bool
rowIsSelected)
{
    // Get the filtered track entries to display in the
    table
    auto& entries = filterManager.getFilteredTracks();

    // Ensure the row number is within valid bounds of
    the track list
    if (rowNumber >= 0 && rowNumber < entries.size())
    {
        // Get the track entry for the current row
        const TrackEntry& entry = entries[rowNumber];

        // If the current column is the Title column (ID
        1), draw the track title
        if (columnId == 1) // Title column
            g.drawText(entry.title, 2, 0, width - 4,
            height, juce::Justification::centredLeft, true);
        // If the current column is the Duration column
        (ID 2), draw the track duration
        else if (columnId == 2) // Duration column
            g.drawText(entry.duration, 2, 0, width - 4,
            height, juce::Justification::centred, true);
    }
}

```

```
}
}
```

3. Preview

Code Flow Explanation:

- The refreshComponentForCell method handles rendering more complex components like the waveform preview in the "Preview" column (ID 4).
- **WaveFormDisplay Component:**
 - If the current column is the "Preview" column (columnId == 4), the method checks if an existing WaveFormDisplay component is available for reuse. If not, a new WaveFormDisplay is created.
 - The waveform display is loaded with the corresponding track's audio URL using loadURL().
 - Depending on whether the track is found via a search or from the full list, the appropriate track is loaded into the waveform display.
 - Finally, the updated or newly created waveform component is returned to the table, allowing the user to see a preview of the audio file in that row.

```
Component*
PlaylistComponent::refreshComponentForCell(int
rowNumber,
    int columnId, bool isRowSelected,
    Component* existingComponentToUpdate)
{
    // Check if the column is the Preview column (ID 4),
    // where the waveform display is rendered
    if (columnId == 4)
    {
        // Try to reuse the existing WaveFormDisplay
        // component if it exists
        auto* waveFormDisplay =
        dynamic_cast<WaveFormDisplay*>(existingComponentToUpdate
        );

        // If there is no existing waveform display,
        // create a new one
        if (waveFormDisplay == nullptr)
        {
            waveFormDisplay = new
            WaveFormDisplay(formatManager, thumbnailCache);
            existingComponentToUpdate = waveFormDisplay;
        }

        // Load the appropriate track's waveform into
        // the display (either from search results or the main
        // list)
        if (trackFound)
        {
            waveFormDisplay->loadURL(trackEntriesFound[rowNumber].audioURL);
        }
        else
        {

```

```

        waveformDisplay-
        >loadURL(trackEntries[rowNumber].audioURL);
        }

        // Return the updated waveform display component
        return waveformDisplay;
    }
}

```

4. Background color of the row

Code Flow Explanation:

- The paintRowBackground method customizes the appearance of each row based on whether it is selected or not.
- **Selected Row:**
 - If the row is selected, the background is filled with a light blue color to visually indicate selection.
- **Unselected Row:**
 - If the row is not selected, it is filled with a pink color.
- This method ensures that the table rows have distinct colors for selected and non-selected states, improving the user experience by making interactions with the table clearer.

```

// Paint the background for each row in the playlist
// table, with different colors for selected rows
void
PlaylistComponent::paintRowBackground(juce::Graphics& g,
int rowNumber, int width, int height, bool
rowIsSelected)
{
    // If the row is selected, fill it with light blue,
    // otherwise fill it with pink
    if (rowIsSelected)
    {
        g.fillAll(juce::Colours::lightblue);
    }
    else
    {
        g.fillAll(juce::Colours::pink);
    }
}

```

x) Track name and time

1. In DeckGUI.h

Code Flow Explanation:

- Two Label objects are declared in the DeckGUI.h file:
 - trackName: Displays the name of the currently loaded track.
 - timeDisplay: Displays the current playback time and total duration of the track.

- These labels are part of the GUI, providing essential feedback to the user on the current state of the deck.

```
Label trackName; // Displays the name of the
                  currently loaded track
Label timeDisplay; // Displays the current playback
                  time and total duration
```

2. In DeckGUI.cpp constructor

Code Flow Explanation:

- In the constructor of DeckGUI, both labels (trackName and timeDisplay) are initialized and configured with various properties:
 - **Alpha Transparency:** Set to 0.8 to make the labels slightly transparent.
 - **Font and Size:** trackName uses a bold font with a size of 20.0f, while timeDisplay uses a slightly smaller bold font of 17.0f.
 - **Colour:** Both labels have their text colour set to black.
 - **Justification:** The labels are left-aligned for readability.
 - **Visibility:** Both labels are added to the DeckGUI component using addAndMakeVisible() to ensure they are rendered.
- The startTimer(500) call starts a periodic timer that updates the GUI every 500 milliseconds, ensuring that the playback time is regularly updated.

```
// Initialize and set up the track name display
trackName.setAlpha(0.8);
addAndMakeVisible(trackName);

trackName.setJustificationType(juce::Justification::left);
trackName.setFont(juce::Font(20.0f, juce::Font::bold));
trackName.setColour(juce::Label::textColourId,
juce::Colours::black);

// Initialize and set up the time display
timeDisplay.setAlpha(0.8);
addAndMakeVisible(timeDisplay);

timeDisplay.setJustificationType(juce::Justification::left);
timeDisplay.setFont(juce::Font(17.0f,
juce::Font::bold));
timeDisplay.setColour(juce::Label::textColourId,
juce::Colours::black);

// Start the timer to update the GUI periodically
startTimer(500); // Update every 500ms
```

3. Resized function

Code Flow Explanation:

- In the resized method, the bounds for trackName and timeDisplay are set:

- **trackName** is positioned based on the calculated rowW, sliderWidth, and rowH values, which are typically derived from the overall component size.
- **timeDisplay** is similarly positioned just below the track name label, ensuring the UI maintains a clean, organized layout.

```
// Set bounds for track name and time display
trackName.setBounds(rowW + sliderWidth * 2, rowH *
2, getWidth() / 4, rowH);
timeDisplay.setBounds(rowW + sliderWidth * 2, rowH *
3, getWidth() / 4, rowH);
```

4. When Any file load to the deck

- When an audio file is loaded into the deck, the name of the track is extracted from the file URL and displayed:
 - `trackName.setText(audioURL.getFileName(), dontSendNotification);` is called to set the text of the trackName label.
 - This ensures that the user immediately sees the track name in the UI when a new file is loaded into the deck.

```
trackName.setText(audioURL.getFileName(),
dontSendNotification);
```

5. Displaying the Current Time and Total Duration

Code Flow Explanation:

- The `timerCallback()` method is called every 500ms due to the timer started in the constructor.
- **Current Position and Total Length:**
 - The `getCurrentPosition()` method retrieves the current playback position of the track in seconds.
 - The `getTotalLength()` method retrieves the total duration of the loaded track.
- These values are converted into a string format (e.g., "mm") using the `secondsToString()` function.
- **Updating the Time Display:**
 - The time display is updated with the current playback time and total track duration using `timeDisplay.setText()`, formatted as `currentTime / totalTime`.
 - This ensures that users can see both how far into the track they are and how long the track is in total.

```
// timerCallback: Periodically updates the GUI elements,
such as waveform position and time display.
void DeckGUI::timerCallback()
{
```

```

        if (deckCleared)
        {
            return; // Skip updates if the deck is cleared
        }
        if (!isnan(player->getPositionRelative()))
        {
            waveFormDisplay.setPositionRelative(player->getPositionRelative());

            double currentTime = player->getCurrentPosition();
            double totalTime = player->getTotalLength();

            juce::String currentTimeString =
secondsToString(currentTime);
            juce::String totalTimeString =
secondsToString(totalTime);

            timeDisplay.setText(currentTimeString + " / " +
totalTimeString, juce::dontSendNotification);

            // Skip updates if the deck is cleared
            if (deckCleared)
            {
                return;
            }
        }
    }

```

6. In DJAudioPlayer

Code Flow Explanation:

- The DJAudioPlayer class contains methods to retrieve the current playback position and total track length:
 - **getCurrentPosition():** Returns the current playback position in seconds, allowing the DeckGUI to display it in real-time.
 - **getTotalLength():** Returns the total length of the loaded track, also in seconds.
 - **getTrackDuration():** This method formats the total track length into a string in the "mm" format, useful for displaying the total duration of the track in a human-readable form.

```

// getCurrentPosition: Returns the current playback
position in seconds.
// @return The current playback position in seconds.
double DJAudioPlayer::getCurrentPosition()
{
    return transportSource.getCurrentPosition();
}

// getTotalLength: Returns the total length of the
loaded track in seconds.
// @return The total length in seconds.
double DJAudioPlayer::getTotalLength() const
{
    return transportSource.getLengthInSeconds();
}

```

```

// getTrackDuration: Returns the track duration
formatted as "mm:ss".
// This method works only after the transportSource is
initialized.
// @return The track duration as a string.
juce::String DJAudioPlayer::getTrackDuration()
{
    double tempLengthInMin =
transportSource.getLengthInSeconds() / 60;
    double secToMin = (tempLengthInMin -
floor(tempLengthInMin)) * 0.6;
    double _duration = (floor(tempLengthInMin) +
secToMin);
    juce::String duration =
juce::String(_duration).substring(0,
4).replaceCharacter('.', ':');
    return duration;
}

```

xi) Search

1. In PlaylistComponent.h

Code Flow Explanation:

- Public Methods:
 - textEditorReturnKeyPressed(): Triggered when the return key is pressed in the search bar. This method initiates the search process by calling searchForTrack() based on user input.
 - searchForTrack(): Performs the actual search by iterating through the list of tracks and finding those that match the user's input.
 - containsIgnoreCase(): A utility method that checks if one string contains another, ignoring case. This is useful for case-insensitive searches.
- Private Member:
 - TextEditor search: The search bar component where users input their search queries.

```

Public:
/* TextEditor override function: called when the return
key is pressed in the search bar (used for searching
tracks) */
void textEditorReturnKeyPressed(TextEditor&
textEditor);

/* Method to search for tracks based on user input */
void searchForTrack(juce::String searchInput);

// Utility method to check if a string contains
another string, ignoring case (helper for search
functionality)
bool containsIgnoreCase(const juce::String& str,
const juce::String& searchString);

```

```
// Search bar component for filtering tracks in the
playlist

Private:
    TextEditor search{ "search", 0 };
```

2. In PlaylistComponent.cpp constructor

Code Flow Explanation:

- In the PlaylistComponent constructor, the search bar is added to the component and configured:
 - **Visibility and Listener:** The search bar is made visible with `addAndMakeVisible()`, and a listener is attached to handle the search actions when the user presses the return key.
 - **Placeholder Text:** The search bar shows a placeholder ("Search") when it is empty.
 - **Text Alignment:** The text in the search bar is left-aligned for easier readability.

```
/* SEARCH BAR */
// Adding and configuring search bar (for user
input) to filter tracks in playlist
addAndMakeVisible(search);
search.addListener(this); // Attach listener to
handle search actions
search.setTextToShowWhenEmpty("Search",
juce::Colours::white); // Placeholder text in search bar

search.setJustification(juce::Justification::centredLeft
); // Align the placeholder text to the left
```

3. In PlaylistComponent.cpp resized function

Code Flow Explanation:

- In the PlaylistComponent constructor, the search bar is added to the component and configured:
 - **Visibility and Listener:** The search bar is made visible with `addAndMakeVisible()`, and a listener is attached to handle the search actions when the user presses the return key.
 - **Placeholder Text:** The search bar shows a placeholder ("Search") when it is empty.
 - **Text Alignment:** The text in the search bar is left-aligned for easier readability.

```
// Positioning search bar and buttons dynamically
based on calculated row height and column width
search.setBounds(getWidth() * 0.6, rowH * 0.1,
getWidth() * 0.2, rowH * 0.6);
```


4. Search functions

Handling Return Key Press (textEditorReturnKeyPressed)

Code Flow Explanation:

- When the return key is pressed in the search bar, the textEditorReturnKeyPressed() method is called:
 - **Getting Search Input:** The user's input is retrieved and trimmed of any leading or trailing whitespace.
 - **Empty Search:** If the search input is empty, a warning message is shown, and all tracks are displayed by resetting the filter to show all tracks.
 - **Search Execution:** If a valid search input is provided, the searchForTrack() method is called to search for matching tracks.
 - **UI Update:** After the search is complete, the table is updated to reflect the results, and the component is repainted.

Performing the Search (searchForTrack)

Code Flow Explanation:

- The searchForTrack() method is responsible for finding tracks that match the user's input:
 - **Search Process:** It iterates through the trackEntries and checks if the track title contains the search input (ignoring case) using the containsIgnoreCase() method.
 - **Search Results:**
 - If matching tracks are found, they are passed to the filterManager, which filters the playlist to display only the matching tracks.
 - If no matches are found, a warning message is displayed, and the playlist is reset to show all tracks.
 - **UI Update:** The table is updated to display the filtered tracks or the full list if no matches were found.

String Matching (containsIgnoreCase)

Code Flow Explanation:

- The containsIgnoreCase() method is a utility function that checks if a string contains another string, ignoring case. This is used to perform case-insensitive searches, making it easier for users to find tracks without worrying about letter case.

```
// Handle actions when the user presses the return key
in the search bar (used to search for tracks)
void
PlaylistComponent::textEditorReturnKeyPressed(juce::Text
Editor& textEditor)
{
    // Get the user's search input and remove any
    leading or trailing whitespace
    juce::String searchInput =
    textEditor.getText().trim();
```

```

        // If the search input is empty, show all tracks and
        display a warning
        if (searchInput.isEmpty())
        {
            // Reset the filter to show all tracks
            filterManager.filterAll();

            // Show an alert box warning the user that the
            search box is empty

juce::AlertWindow::showMessageBoxAsync(juce::AlertWindow
::WarningIcon,
            "Search",
            "Search box is empty. Displaying all
tracks.",
            "OK");
        }
        else
        {
            // If search input is provided, search for
            tracks that match the input
            searchForTrack(searchInput);
        }

        // Refresh the table to reflect any changes from the
        search
        tableComponent.updateContent();
        repaint(); // Redraw the component
    }

    // Search for tracks in the playlist that match the
    user's search input
    void PlaylistComponent::searchForTrack(juce::String
searchInput)
    {
        std::vector<TrackEntry> foundEntries; // Vector to
        store tracks that match the search input

        // Iterate through all the tracks in the playlist
        and check if the title contains the search input
        for (const auto& entry : trackEntries)
        {
            if (entry.title.containsIgnoreCase(searchInput))
            // Case-insensitive search
            {
                foundEntries.push_back(entry); // Add
                matching tracks to the foundEntries vector
            }
        }

        // If matching tracks are found, update the filter
        manager with the search results
        if (!foundEntries.empty())
        {
            filterManager.setFilteredTracks(foundEntries);
        }
        // If no matching tracks are found, display a
        warning message
        else
        {
juce::AlertWindow::showMessageBoxAsync(juce::AlertWindow
::WarningIcon,

```

```

        "Search",
        "No tracks found!",
        "OK");

        // Reset the filter to show all tracks again
        filterManager.filterAll();
    }

    // Update the table to display the search results
    (or all tracks if no match was found)
    tableComponent.updateContent();
    repaint();
}

// Utility function to check if one string contains
// another, ignoring case (helper function for searching)
bool PlaylistComponent::containsIgnoreCase(const
juce::String& str, const juce::String& searchString)
{
    return str.containsIgnoreCase(searchString); //
    Perform case-insensitive substring search
}

```

xii) Load to left right deck and delete ,clear deck

1. In PlaylistComponent.h

Code Flow Explanation:

- **Flags for Deck Loading:**
 - isLoadingtoDeck1: Indicates if a track is currently loaded into Deck 1.
 - isLoadingtoDeck2: Indicates if a track is currently loaded into Deck 2.
- **Track Playing Variables:**
 - trackPlaying1: Stores the index of the track currently playing in Deck 1.
 - trackPlaying2: Stores the index of the track currently playing in Deck 2.

```

        // Boolean flags to indicate whether a track is
        loaded into Deck 1 or Deck 2
        bool isLoadingtoDeck1 = false; // Flag for Deck 1
        bool isLoadingtoDeck2 = false; // Flag for Deck 2

        // Variables to keep track of which track is
        currently playing in each deck
        int trackPlaying1; // Index of the track playing in
        Deck 1
        int trackPlaying2; // Index of the track playing in
        Deck 2

```

2. In PlaylistComponent.cpp constructor

Code Flow Explanation:

- The columns "Delete", "Left", and "Right" are added to the table header:
 - Column 5 ("Delete"): Displays a button to delete the track.
 - Column 6 ("Left"): Loads the track into Deck 1.
 - Column 7 ("Right"): Loads the track into Deck 2.
- The table model is set to the current class (this), so that PlaylistComponent manages the data and interaction.

```

        tableComponent.getHeader().addColumn("Delete", 5,
        100);
        tableComponent.getHeader().addColumn("Left", 6, 100);
        tableComponent.getHeader().addColumn("Right", 7,
        100);

        // Set the table model to this class to manage data
        for the table
        tableComponent.setModel(this);
        addAndMakeVisible(tableComponent);

```

3. In ButonClicked function for delete

Code Flow Explanation:

- When the delete button (column 5) is clicked:
 - **Clear Decks if Loaded:** If the track is currently loaded into either Deck 1 or Deck 2, it will be cleared from that deck using clearDeck().
 - **Remove Track:** The track is removed from the trackEntries vector.
 - **Filter Reapplication:** After removal, the playlist is re-filtered based on the current filter (all tracks or favourites).
 - **UI Refresh:** The table is updated, and the component is repainted to reflect the changes.

```

void PlaylistComponent::buttonClicked(juce::Button* button)
{
    if(//Other conditions){
        // Other functionalities
    }

    else if (colIndex == 5) // If "Delete" column is
        clicked, remove the track from the playlist
    {
        // Check if the track is loaded into deckGUI1 or
        deckGUI2, and clear it if necessary
        if (isLoadingtoDeck1 && deckGUI1->player-
        >getLoadedURL() == entry.audioURL)
        {
            deckGUI1->clearDeck(); // Clear the track
            from deckGUI1
            isLoadingtoDeck1 = false;
        }
        if (isLoadingtoDeck2 && deckGUI2->player-
        >getLoadedURL() == entry.audioURL)
        {
            deckGUI2->clearDeck(); // Clear the track
            from deckGUI2
            isLoadingtoDeck2 = false;
        }
    }
}

```

```

        // Remove the track from the main trackEntries
        vector
        auto it = std::remove(trackEntries.begin(),
trackEntries.end(), entry);
        if (it != trackEntries.end())
        {
            trackEntries.erase(it, trackEntries.end());
// Erase the track from the vector
        }

        // Reapply the filter (all tracks or favourites)
        to reflect the removal
        if (filterManager.getCurrentFilterType() ==
PlaylistFilterManager::FilterType::All)
        {
            filterManager.filterAll();
        }
        else
        {
            filterManager.filterFavourites();
        }

        tableComponent.updateContent(); // Refresh the
        table to reflect changes
        repaint(); // Redraw the component
    }
}

```

4. In DeckGUI to clear deck

Code Flow Explanation:

- When the delete button (column 5) is clicked:
 - **Clear Decks if Loaded:** If the track is currently loaded into either Deck 1 or Deck 2, it will be cleared from that deck using clearDeck().
 - **Remove Track:** The track is removed from the trackEntries vector.
 - **Filter Reapplication:** After removal, the playlist is re-filtered based on the current filter (all tracks or favourites).
 - **UI Refresh:** The table is updated, and the component is repainted to reflect the changes.

```

// clearDeck: Resets the DeckGUI, clearing the track
// information and resetting controls to their default
// states.
void DeckGUI::clearDeck()
{
    deckCleared = true;
    trackName.setText("");
    juce::NotificationType::dontSendNotification();
    timeDisplay.setText("00:00 / 00:00",
juce::NotificationType::sendNotification);
    waveformDisplay.fileLoaded = false;
    waveformDisplay.repaint(); // Ensure the waveform
display is cleared
    player->stop();
    player->setPosition(0); // Reset the player's
position
    isLooping = false;
}

```

```

        playButton.setButtonText("Play");

        loopButton.setColour(juce::TextButton::textColourOffId,
            juce::Colours::black);
        cueStartPosition1 = -1.0;
        cueStartPosition2 = -1.0;

        setCueButton1.setColour(juce::TextButton::buttonColourId,
            juce::Colours::blue);

        setCueButton2.setColour(juce::TextButton::buttonColourId,
            juce::Colours::blue);

        bassSlider.setValue(1.0);
        midSlider.setValue(1.0);
        trebSlider.setValue(1.0);
    }

```

5. In PlaylistComponent to load in left deck in ButtonClicked function

Code Flow Explanation:

- When the left deck button (column 6) is clicked:
 - **Track Loading:** The selected track is loaded into Deck 1's player using `deckGUI1->player->loadURL(entry.audioURL)`.
 - **Waveform Display:** The waveform is synced with the track position using `setPositionRelative()`.
 - **Track Name:** The track name is displayed in Deck 1.
 - **Flag Setting:** The `isLoadingToDeck1` flag is set to true, indicating that Deck 1 has a loaded track.

```

else if (colIndex == 6)    // If "Left" column is
    clicked, load the track into deckGUI1
{
    deckGUI1->deckCleared = false; // Set the
    deckCleared flag to false (deck is not empty)
    deckGUI1->playButton.setToggleState(false,
    dontSendNotification); // Ensure play button is in a
    stopped state
    deckGUI1->player->loadURL(entry.audioURL); // Load
    the selected track into deckGUI1's player
    deckGUI1->waveFormDisplay.loadURL(entry.audioURL);
    // Load the waveform for visual representation
    deckGUI1-
    >waveFormDisplay.setPositionRelative(deckGUI1->player-
    >getPositionRelative()); // Sync the waveform with the
    player's position
    deckGUI1-
    >trackName.setText(entry.audioURL.getFileName(),
    dontSendNotification); // Display the track name in
    deckGUI1
    isLoadingToDeck1 = true; // Set the flag to indicate
    that the deck has a loaded track
}

```

6. In PlaylistComponent to load in right deck in ButtonClicked function

Code Flow Explanation:

- When the right deck button (column 7) is clicked:
 - **Track Loading:** The selected track is loaded into Deck 2's player using `deckGUI2->player->loadURL(entry.audioURL)`.
 - **Waveform Display:** The waveform is synced with the track position using `setPositionRelative()`.
 - **Track Name:** The track name is displayed in Deck 2.

Flag Setting: The `isLoadingtoDeck2` flag is set to `true`, indicating that Deck 2 has a loaded track

```
else if (colIndex == 7) // If "Right" column is
    clicked, load the track into deckGUI2
{
    deckGUI2->deckCleared = false; // Set the
    deckCleared flag to false (deck is not empty)
    deckGUI2->playButton.setToggleState(false,
    dontSendNotification); // Ensure play button is in a
    stopped state
    deckGUI2->player->loadURL(entry.audioURL); // Load
    the selected track into deckGUI2's player
    deckGUI2->waveFormDisplay.loadURL(entry.audioURL);
    // Load the waveform for visual representation
    deckGUI2-
    >waveFormDisplay.setPositionRelative(deckGUI1->player-
    >getPositionRelative()); // Sync the waveform with the
    player's position
    deckGUI2-
    >trackName.setText(entry.audioURL.getFileName(),
    dontSendNotification); // Display the track name in
    deckGUI2
    isLoadingtoDeck2 = true; // Set the flag to indicate
    that the deck has a loaded track
}
```

7. Dynamic Button Creation for Deck ActionsCode Flow Explanation:

- In this function, buttons for delete ("Delete"), load into Deck 1 ("Left"), and load into Deck 2 ("Right") are dynamically created for each row in the table:
 - **Reusing Components:** If the button already exists, it is reused.
 - **Button Creation:** New buttons are created for each cell in the respective columns.
 - **Button Identification:** Each button is assigned a unique component ID based on the row and column to ensure proper tracking of button clicks.

```
// Refresh the custom components (e.g., buttons,
// waveform displays) used for each cell in the table
Component*
PlaylistComponent::refreshComponentForCell(int
rowNumber,
    int columnId, bool isRowSelected,
    Component* existingComponentToUpdate)
{
    // If the column is for Delete, Left (Deck 1), or
    // Right (Deck 2) buttons, create buttons accordingly
```

```

else if (columnId >= 5 && columnId <= 7)
{
    // Try to reuse an existing button component if
    it exists
    auto* btn =
dynamic_cast<TextButton*>(existingComponentToUpdate);

    // If no button exists, create a new one with
    appropriate labels ("Delete", "Left", "Right")
    if (btn == nullptr)
    {
        btn = new TextButton{ columnId == 5 ?
"Delete" : columnId == 6 ? "Left" : "Right" };
        btn->addListener(this); // Add a listener to
        handle button clicks

        // Assign a unique component ID based on the
        row and column index
        btn->
setComponentID(std::to_string(rowNumber) +
std::to_string(columnId));
        existingComponentToUpdate = btn;
    }
    return btn;
}
else
{
    return nullptr; // If the column is not handled,
return nullptr
}
}

```

xiii) Add favourite and filter

1. In PlaylistFilterManager.h

Code Flow Explanation:

- **Filter Types:** The playlist can be filtered by:
 - All tracks (FilterType::All)
 - Only favourite tracks (FilterType::Favourites)
 - Custom filters (FilterType::Custom) such as search results.
- **Constructor:** PlaylistFilterManager is initialized with a reference to the full list of tracks (allEntries). By default, the "All" filter is applied to show all tracks.
- **Methods:**
 - filterAll(): Copies all tracks to the filteredEntries vector.
 - filterFavourites(): Filters and copies only favourite tracks to the filteredEntries.
 - setFilteredTracks(): Allows setting a custom list of filtered tracks (e.g., from search results).
 - getFilteredTracks(): Returns the currently filtered list of tracks.


```

/*
=====
=====

    PlaylistFilterManager.h
    Created: 29 Aug 2024 9:20:00pm
    Author: Admin

=====
=====
*/

#pragma once

#include <JuceHeader.h>           // JUCE framework
header for utilities
#include "TrackEntry.h"         // Include TrackEntry
class for handling track data

//=====
=====
// PlaylistFilterManager is a class that manages the
// filtering of a playlist of TrackEntry objects.
// It provides methods to filter tracks based on
// different criteria like "All", "Favourites", or custom
// filters.
class PlaylistFilterManager
{
public:
    // Enum to represent the different types of filters
    // that can be applied.
    enum class FilterType { All, Favourites, Custom };

    // Constructor: Initializes the
    // PlaylistFilterManager with a reference to a vector of
    // TrackEntry objects.
    // The constructor starts by applying the "All"
    // filter to include all tracks.
    // @param entries A reference to the vector of
    // TrackEntry objects representing the full playlist.
    PlaylistFilterManager(std::vector<TrackEntry>&
entries);

    // filterAll: Filters the playlist to include all
    // tracks.
    // Copies all entries from the full playlist to the
    // filtered playlist.
    void filterAll();

    // filterFavourites: Filters the playlist to include
    // only favourite tracks.
    // Iterates through the full playlist and copies
    // only the favourite tracks to the filtered playlist.
    void filterFavourites();

    // getFilteredTracks: Returns a reference to the
    // current list of filtered tracks.
    // @return A reference to the vector of TrackEntry
    // objects representing the filtered playlist.
    std::vector<TrackEntry>& getFilteredTracks();

```

```

        // setFilteredTracks: Sets a custom list of filtered
        tracks.
        // Marks the current filter type as "Custom".
        // @param foundEntries A vector of TrackEntry
        objects representing the custom filtered tracks.
        void setFilteredTracks(const
std::vector<TrackEntry>& foundEntries);

        // getCurrentFilterType: Returns the current filter
        type that is applied.
        // @return The current filter type, which could be
        All, Favourites, or Custom.
        FilterType getCurrentFilterType() const { return
currentFilterType; }

private:
        std::vector<TrackEntry>& allEntries;           //
Reference to the full list of all TrackEntry objects
        std::vector<TrackEntry> filteredEntries;       //
Vector to store the currently filtered list of
TrackEntry objects
        FilterType currentFilterType;                 // The
type of filter currently applied to the playlist
};

```

2. In PlaylistFilterManager.cpp

```

/*
=====
=====

        PlaylistFilterManager.cpp
        Created: 29 Aug 2024 9:20:00pm
        Author: Admin

=====
=====
*/

#include "PlaylistFilterManager.h" // Include the
header file for the PlaylistFilterManager class

//=====
=====
// Constructor: Initializes the PlaylistFilterManager
with a reference to the full list of TrackEntry objects.
// The constructor applies the "All" filter by default,
including all tracks in the filtered list.
// @param entries A reference to the vector of
TrackEntry objects representing the full playlist.
PlaylistFilterManager::PlaylistFilterManager(std::vector
<TrackEntry>& entries)
: allEntries(entries),
currentFilterType(FilterType::All) // Initialize with
all tracks and default filter type
{
        filterAll(); // Start by applying the "All" filter
}

```

```

// filterAll: Filters the playlist to include all
tracks.
// Copies all TrackEntry objects from the full list to
the filtered list.
void PlaylistFilterManager::filterAll()
{
    filteredEntries = allEntries; // Copy all entries
to the filtered list
    currentFilterType = FilterType::All; // Set the
filter type to "All"
}

// filterFavourites: Filters the playlist to include
only favourite tracks.
// Iterates through the full list and adds only the
favourite tracks to the filtered list.
void PlaylistFilterManager::filterFavourites()
{
    filteredEntries.clear(); // Clear the filtered list
before adding new entries
    for (const auto& entry : allEntries)
    {
        if (entry.isFavourite) // Check if the track is
marked as a favourite
        {
            filteredEntries.push_back(entry); // Add
the favourite track to the filtered list
        }
    }
    currentFilterType = FilterType::Favourites; // Set
the filter type to "Favourites"
}

// setFilteredTracks: Sets a custom list of filtered
tracks provided by the user.
// Replaces the current filtered list with the provided
list and marks the filter type as "Custom".
// @param foundEntries A vector of TrackEntry objects
representing the custom filtered tracks.
void PlaylistFilterManager::setFilteredTracks(const
std::vector<TrackEntry>& foundEntries)
{
    filteredEntries = foundEntries; // Replace the
filtered list with the provided custom list
    currentFilterType = FilterType::Custom; // Mark the
filter type as "Custom"
}

// getFilteredTracks: Returns a reference to the current
list of filtered tracks.
// The list reflects the tracks that meet the criteria
of the currently applied filter.
// @return A reference to the vector of TrackEntry
objects representing the filtered playlist.
std::vector<TrackEntry>&
PlaylistFilterManager::getFilteredTracks()
{
    return filteredEntries; // Return the current
filtered list of tracks
}

```

3. In PlaylistComponent.h

Code Flow Explanation:

- **Buttons:**
 - filterAll: Button to show all tracks.
 - filterFav: Button to show only favourite tracks.
- **Favourite Button:**
 - favouriteButton: An image button used to toggle the favourite status for tracks.
 - **Heart Icons:** heartBlackImg (black heart for non-favourites) and heartRedImg (red heart for favourites) are used to visually indicate the favourite status.

```
TextButton filterAll{ "All" }; // Button for showing all
tracks
TextButton filterFav{ "Favourite" }; // Button for
showing only favourite tracks

// Favourite button for marking tracks as favourites
juce::ImageButton* favouriteButton = nullptr;

// Images for displaying a black heart (unfavourited)
and red heart (favourited) for tracks
juce::Image heartBlackImg; // Image for non-favourite
track (black heart)
juce::Image heartRedImg; // Image for favourite track
(red heart)
```

4. In PlaylistComponent.cpp constructor

Code Flow Explanation:

- **Filter Buttons:**
 - filterAll.addListener(this) and filterFav.addListener(this) are set to capture user clicks.
- **Favourite Icon Loading:**
 - The heart icons (heartBlackImg and heartRedImg) are loaded from binary data and used in the favourite button.

```
filterAll.addListener(this);
filterFav.addListener(this);

// Loading images for favourite (heart icon) in black
and red for toggling favourite status
heartBlackImg =
juce::ImageCache::getFromMemory(BinaryData::heart_black_
png, BinaryData::heart_black_pngSize);
heartRedImg =
juce::ImageCache::getFromMemory(BinaryData::heart_red_pn
g, BinaryData::heart_red_pngSize);
```

5. In resized function

Code Flow Explanation:

- The filter buttons (filterAll and filterFav) are dynamically resized and positioned based on the component's size. This ensures that the UI layout adapts to window resizing.

```
void PlaylistComponent::resized()
{
    // Calculate row height (rowH) and column width
    (colW) based on the component size
    double rowH = getHeight() / 5;
    double colW = getWidth() / 10;

    filterAll.setBounds(colW, rowH * 0.2, colW, rowH *
0.6);
    filterFav.setBounds(colW * 2, rowH * 0.2, colW, rowH
* 0.6);
}
```

6. In ButtonClicked function

Code Flow Explanation:

- **Filter Buttons:**
 - filterAll: When clicked, resets the filter to show all tracks using filterManager.filterAll().
 - filterFav: When clicked, filters the playlist to show only favourite tracks using filterManager.filterFavourites().
- **Favourite Button:**
 - When the favourite button (column 3) is clicked:
 - The isFavourite flag of the track is toggled (set to true/false).
 - The corresponding track in the main trackEntries vector is updated with the new favourite status.
 - The heart icon is updated based on the favourite status (red for favourite, black for non-favourite).

The current filter is re-applied to ensure the view is consistent.

```
if(//Other condition){
    ///other codes
}
else if (button == &filterAll) // If filterAll button
is clicked, reset the filter to show all tracks
{
    filterManager.filterAll(); // Show all tracks
    tableComponent.updateContent(); // Refresh the
table to reflect the filter change
    repaint(); // Redraw the component
}
else if (button == &filterFav) // If filterFav
button is clicked, filter to show only favourite tracks
{
    filterManager.filterFavourites(); // Show only
favourite tracks
    tableComponent.updateContent(); // Refresh the
table to reflect the filter change
}
```

```

        repaint(); // Redraw the component
    }
    else if (button->getComponentID().length() >= 2) //
    Handle buttons dynamically by their component IDs (e.g.
    "Delete", "Left", "Right")
    {
        int rowIndex = std::stoi(button-
>getComponentID().toString().substr(0, 1)); //
        Extract row index
        int colIndex = std::stoi(button-
>getComponentID().toString().substr(1, 2)); //
        Extract column index

        auto& entries =
        filterManager.getFilteredTracks(); // Get filtered track
        entries

        // Ensure the rowIndex is valid (within bounds
        of entries vector)
        if (rowIndex >= 0 && rowIndex < entries.size())
        {
            TrackEntry& entry = entries[rowIndex]; //
            Get the track entry for the clicked row

            if (colIndex == 3) // If "Add to Favourite"
            column is clicked, toggle the favourite status
            {
                entry.isFavourite = !entry.isFavourite;
            // Toggle favourite status of the track

            // Find the entry in the main
            trackEntries vector and update its status
            auto it =
            std::find(trackEntries.begin(), trackEntries.end(),
            entry);
            if (it != trackEntries.end())
            {
                it->isFavourite = entry.isFavourite;
            // Update the favourite status
            }

            // Update the favourite button's icon
            based on the new status (red heart or black heart)
            auto* imgButton =
            dynamic_cast<juce::ImageButton*>(button);
            if (imgButton)
            {
                imgButton-
                >setToggleState(entry.isFavourite,
                juce::sendNotification);
                imgButton->setImages(true, true,
                true,
                entry.isFavourite ? heartRedImg
                : heartBlackImg, 1.0f, juce::Colours::transparentWhite,
                entry.isFavourite ? heartRedImg
                : heartBlackImg, 1.0f, juce::Colours::transparentWhite,
                entry.isFavourite ? heartRedImg
                : heartBlackImg, 1.0f, juce::Colours::transparentWhite);
            }

            // Reapply the filter (show all or only
            favourites, depending on current filter type)

```

```

        if (filterManager.getCurrentFilterType()
== PlaylistFilterManager::FilterType::Favourites)
        {
            filterManager.filterFavourites();
        }
        else
        {
            filterManager.filterAll(); // Ensure
all tracks are visible in case of filterAll
        }
        tableComponent.updateContent(); //
Refresh the table content to reflect changes
        repaint(); // Redraw the component
    }

```

7. Refresh Component for Favourite Button in refreshComponentForCell()

Code Flow Explanation:

- For the favourite column (column 3), the favouriteButton is created or reused:
 - **Button Toggle State:** The button toggles its state (on/off) when clicked, indicating if the track is marked as a favourite.
 - **Heart Icon:** The heart icon is updated based on whether the track is marked as a favourite.
- The favouriteButton is assigned a unique component ID based on the row and column, and a listener is attached to handle clicks.

```

// Refresh the custom components (e.g., buttons,
// waveform displays) used for each cell in the table
Component*
PlaylistComponent::refreshComponentForCell(int
rowNumber,
    int columnId, bool isRowSelected,
    Component* existingComponentToUpdate)
{
    // Check if the column is the Preview column (ID 4),
    // where the waveform display is rendered
    if (columnId == 4)
    {
        // Try to reuse the existing WaveFormDisplay
        // component if it exists
    }
    // If the column is the Favourite column (ID 3),
    // handle the favourite button
    else if (columnId == 3) // Add to favourite column
    {
        // Try to reuse the existing favourite button
        // component if it exists
        auto* favouriteButton =
dynamic_cast<juce::ImageButton*>(existingComponentToUpdate);

        // If there is no existing favourite button,
        // create a new one
    }
}

```

```

        if (favouriteButton == nullptr)
        {
            favouriteButton = new juce::ImageButton();
            favouriteButton->setClickingTogglesState(true); // Set button to toggle state on click

            // Assign a unique component ID based on the row and column index (e.g., "03" for row 0, column 3)
            favouriteButton->setComponentID(std::to_string(rowNumber) + std::to_string(columnId));
            favouriteButton->addListener(this); // Add a listener to handle button clicks

            existingComponentToUpdate = favouriteButton;
        }

        // Update the button image based on whether the track is marked as a favourite or not (red or black heart)
        auto& entries = filterManager.getFilteredTracks();
        favouriteButton->setImages(true, true, true, entries[rowNumber].isFavourite ? heartRedImg : heartBlackImg, 1.0f, juce::Colours::transparentWhite, entries[rowNumber].isFavourite ? heartRedImg : heartBlackImg, 1.0f, juce::Colours::transparentWhite, entries[rowNumber].isFavourite ? heartRedImg : heartBlackImg, 1.0f, juce::Colours::transparentWhite);

        return favouriteButton;
    }

    // If the column is for Delete, Left (Deck 1), or Right (Deck 2) buttons, create buttons accordingly
    else if (columnId >= 5 && columnId <= 7)
    {
        // Try to reuse an existing button component if it exists
    }
    else
    {
        return nullptr; // If the column is not handled, return nullptr
    }
}

```

R2B: GUI code of new event listeners.

i) Equaliser

1. EQProcessor.cpp

- **prepareToPlay(double sampleRate, int samplesPerBlock):** Prepares the EQ processor before playback starts by setting the sample rate and updating filter coefficients based on the initial slider values.
- **processBlock(juce::AudioBuffer<float>& buffer):** Applies the bass, mid, and treble filters to each channel of the audio buffer in real-time during playback.

- **setBassValue(float value), setMidValue(float value), and setTrebleValue(float value):** Set the gain values for the bass, mid, and treble frequency bands respectively. These are linked to the rotary sliders in the GUI.
- **updateFilters():** Updates the filter coefficients for the bass, mid, and treble filters based on the current slider values and recalculates the frequency and gain for each band.

```
#include "EQProcessor.h" // Include the header file that
                           // declares the EQProcessor class

// Constructor for the EQProcessor class
EQProcessor::EQProcessor()
{
    // Constructor body is empty because no initialisation
    // logic is required here.
}

// Destructor for the EQProcessor class
EQProcessor::~EQProcessor()
{
    // Destructor body is also empty as no cleanup of
    // resources is necessary.
}

// Prepare the processor before playback starts. This
// method is called to initialise the audio processing
// environment.
void EQProcessor::prepareToPlay(double _sampleRate, int
samplesPerBlock)
{
    sampleRate = _sampleRate; // Store the sample rate,
    // which is needed for filter coefficient calculations
    updateFilters(); // Update filter coefficients based on
    // initial settings (slider values)
}

// Process audio samples block by block. This is where the
// filters are applied to the audio buffer.
void EQProcessor::processBlock(juce::AudioBuffer<float>&
buffer)
{
    // Loop through each audio channel in the buffer
    // (stereo or more)
    for (int channel = 0; channel <
buffer.getNumChannels(); ++channel)
    {
        // Apply the bass filter to the current channel's
        // audio samples

        bassFilter.processSamples(buffer.getWritePointer(channel),
buffer.getNumSamples());

        // Apply the mid filter to the current channel's
        // audio samples

        midFilter.processSamples(buffer.getWritePointer(channel),
buffer.getNumSamples());

        // Apply the treble filter to the current channel's
        // audio samples
    }
}
```

```

    trebleFilter.processSamples(buffer.getWritePointer(channel)
    , buffer.getNumSamples());
    }
}

// Method to set the bass slider value (range between 0.0
// and 1.0) and update the filter coefficients accordingly
void EQProcessor::setBassValue(float value)
{
    bassSliderValue = value; // Store the new value for the
    bass slider
    updateFilters(); // Recalculate the filter coefficients
    based on the new slider value
}

// Method to set the mid slider value (range between 0.0
// and 1.0) and update the filter coefficients accordingly
void EQProcessor::setMidValue(float value)
{
    midSliderValue = value; // Store the new value for the
    mid slider
    updateFilters(); // Recalculate the filter coefficients
    based on the new slider value
}

// Method to set the treble slider value (range between 0.0
// and 1.0) and update the filter coefficients accordingly
void EQProcessor::setTrebleValue(float value)
{
    trebleSliderValue = value; // Store the new value for
    the treble slider
    updateFilters(); // Recalculate the filter coefficients
    based on the new slider value
}

// Private method to update the filter coefficients based
// on the current slider values and the sample rate
void EQProcessor::updateFilters()
{
    // Convert the bass slider value (0.0 to 1.0) to a
    frequency range (30 Hz to 250 Hz)
    float bassFreq = juce::jmap(bassSliderValue, 30.0f,
    250.0f);

    // Convert the mid slider value (0.0 to 1.0) to a
    frequency range (250 Hz to 4000 Hz)
    float midFreq = juce::jmap(midSliderValue, 250.0f,
    4000.0f);

    // Convert the treble slider value (0.0 to 1.0) to a
    frequency range (4000 Hz to 18000 Hz)
    float trebleFreq = juce::jmap(trebleSliderValue,
    4000.0f, 18000.0f);

    // Map the bass slider value to a gain range (0 dB to 1
    dB)
    float bassGain = juce::jmap(bassSliderValue, 0.0f,
    1.0f);

    // Map the mid slider value to a gain range (0 dB to 3
    dB)
    float midGain = juce::jmap(midSliderValue, 0.0f, 3.0f);
}

```

```

        // Map the treble slider value to a gain range (0 dB to
        10 dB)
        float trebleGain = juce::jmap(trebleSliderValue, 0.0f,
        10.0f);

        // Set the bass filter coefficients using a low-shelf
        filter (boosts or cuts low frequencies)

        bassFilter.setCoefficients(juce::IIRCoefficients::makeLowShelf(
            sampleRate, bassFreq, 0.707f,
            juce::Decibels::decibelsToGain(bassGain)));

        // Set the mid filter coefficients using a peak filter
        (boosts or cuts a specific frequency range)

        midFilter.setCoefficients(juce::IIRCoefficients::makePeakFilter(
            sampleRate, midFreq, 0.707f,
            juce::Decibels::decibelsToGain(midGain)));

        // Set the treble filter coefficients using a high-
        shelf filter (boosts or cuts high frequencies)

        trebleFilter.setCoefficients(juce::IIRCoefficients::makeHighShelf(
            sampleRate, trebleFreq, 0.707f,
            juce::Decibels::decibelsToGain(trebleGain)));
    }

```

2. In DJAudioPlayer

- **prepareToPlay(int samplesPerBlockExpected, double sampleRate):** Initializes the equaliser processor (EQProcessor) and ensures the filters are set before playback.
- **getNextAudioBlock(const juce::AudioSourceChannelInfo& bufferToFill):** This method fetches the next block of audio samples and applies the equaliser filters to the audio buffer.

```

DJAudioPlayer::DJAudioPlayer(AudioFormatManager&
    _formatManager)
    : formatManager(_formatManager),
    resampleSource(&transportSource, false)
{
    eqProcessor.prepareToPlay(44100, 512); // Prepare
    the EQProcessor with default sample rate and block size
}

// Destructor: Cleans up any resources used by the
DJAudioPlayer.
DJAudioPlayer::~DJAudioPlayer()
{
}

// prepareToPlay: Prepares the audio player for playback
by initializing buffers and sources.

```

```

// This method is called before playback starts to
ensure everything is set up correctly.
void DJAudioPlayer::prepareToPlay(int
samplesPerBlockExpected, double sampleRate)
{

transportSource.prepareToPlay(samplesPerBlockExpected,
sampleRate);

resampleSource.prepareToPlay(samplesPerBlockExpected,
sampleRate);
    eqProcessor.prepareToPlay(sampleRate,
samplesPerBlockExpected);
}

// getNextAudioBlock: Retrieves the next block of audio
samples and processes it through the EQProcessor.
// This method is called continuously during playback to
provide audio data to the output device.
void DJAudioPlayer::getNextAudioBlock(const
juce::AudioSourceChannelInfo& bufferToFill)
{
    resampleSource.getNextAudioBlock(bufferToFill); //
Get the audio block from the resampler
    eqProcessor.processBlock(*bufferToFill.buffer); //
Apply EQ processing to the audio block

}

```

3. In DJAudioPlayer

Bass, Mid, Treble Setters (setBass(double value), setMid(double value), setTreble(double value)): These methods allow the equaliser values to be set from the GUI.

```

// setBass: Sets the gain for the bass frequency band
using the EQProcessor.
// @param value The gain value for the bass band in
decibels.
void DJAudioPlayer::setBass(double value)
{
    eqProcessor.setBassValue(value);
}

// setMid: Sets the gain for the mid-frequency band
using the EQProcessor.
// @param value The gain value for the mid band in
decibels.

void DJAudioPlayer::setMid(double value)
{
    eqProcessor.setMidValue(value);
}

// setTreble: Sets the gain for the treble frequency
band using the EQProcessor.
// @param value The gain value for the treble band in
decibels.

```

```

void DJAudioPlayer::setTreble(double value)
{
    eqProcessor.setTrebleValue(value);
}

```

4. In DeckGUI constructor

In DeckGUI, the bass, mid, and treble controls are implemented as rotary sliders with corresponding labels. These sliders are linked to the DJAudioPlayer methods to adjust the equaliser.

- **Rotary Sliders and Labels:**

- The sliders are set up to control the bass, mid, and treble values in a range from 0.0 to 1.0.
- The labels are placed below each slider and indicate the band they control (Bass, Mid, Treble).

```

// Initialize and set up the bass slider
addAndMakeVisible(bassSlider);
bassSlider.addListener(this);
bassSlider.setRange(0.0, 1.0);
bassSlider.setValue(0.0);

// Initialize and set up the bass label
bassLabel.setText("Bass",
juce::dontSendNotification);
addAndMakeVisible(bassLabel);

bassLabel.setJustificationType(juce::Justification::centred);
bassLabel.setFont(juce::Font(16.0f,
juce::Font::bold));
bassLabel.setColour(juce::Label::textColourId,
juce::Colours::black);
bassSlider.setSliderStyle(Slider::Rotary);
bassSlider.setLookAndFeel(&customLookAndFeel);

bassSlider.setTextBoxStyle(Slider::TextEntryBoxPosition::
NoTextBox, true, 0, 0);

// Initialize and set up the mid slider
addAndMakeVisible(midSlider);
midSlider.addListener(this);
midSlider.setRange(0.0, 1.0);
midSlider.setValue(0.0);

// Initialize and set up the mid label
midLabel.setText("Mid", juce::dontSendNotification);
addAndMakeVisible(midLabel);

midLabel.setJustificationType(juce::Justification::centred);
midLabel.setFont(juce::Font(16.0f,
juce::Font::bold));
midLabel.setColour(juce::Label::textColourId,
juce::Colours::black);
midSlider.setSliderStyle(Slider::Rotary);
midSlider.setLookAndFeel(&customLookAndFeel);

```

```

midSlider.setTextBoxStyle(Slider::TextEntryBoxPosition::
NoTextBox, true, 0, 0);

// Initialize and set up the treble slider
addAndMakeVisible(trebSlider);
trebSlider.addListener(this);
trebSlider.setRange(0.0, 1.0);
trebSlider.setValue(0.0);

// Initialize and set up the treble label
trebLabel.setText("Treble",
juce::dontSendNotification);
addAndMakeVisible(trebLabel);

trebLabel.setJustificationType(juce::Justification::cent
red);
trebLabel.setFont(juce::Font(16.0f,
juce::Font::bold));
trebLabel.setColour(juce::Label::textColourId,
juce::Colours::black);
trebSlider.setSliderStyle(Slider::Rotary);
trebSlider.setLookAndFeel(&customLookAndFeel);

trebSlider.setTextBoxStyle(Slider::TextEntryBoxPosition:
:NoTextBox, true, 0, 0);

```

5. In DeckGUI resized functionality

```

// Set bounds for EQ sliders and labels
bassSlider.setBounds(rowW - sliderWidth * 2, rowH *
15, sliderWidth, sliderHeight);
midSlider.setBounds(rowW, rowH * 15, sliderWidth,
sliderHeight);
trebSlider.setBounds(rowW + sliderWidth * 2, rowH *
15, sliderWidth, sliderHeight);

bassLabel.setBounds(rowW - sliderWidth * 2, rowH *
16 + 15, sliderWidth, sliderHeight);
midLabel.setBounds(rowW, rowH * 16 + 15,
sliderWidth, sliderHeight);
trebLabel.setBounds(rowW + sliderWidth * 2, rowH *
16 + 15, sliderWidth, sliderHeight);

```

6. In DeckGUI sliderValueChanged function

```

// sliderValueChanged: Handles slider value change
events for the bass, mid, and treble EQ sliders.
void DeckGUI::sliderValueChanged(Slider* slider)
{
    if (slider == &bassSlider)
    {
        player->setBass(bassSlider.getValue());
    }
    else if (slider == &midSlider)
    {
        player->setMid(midSlider.getValue());
    }
    else if (slider == &trebSlider)

```

```

        {
            player->setTreble(trebSlider.getValue());
        }
    }
}

```

ii) Forward & previous button

1. In DeckGUI constructor

Two new buttons are introduced in the DeckGUI constructor:

- **prevButton**: For loading the previous track.
- **ffwdButton**: For loading the next track.

```

// Initialize and set up the previous track button
addAndMakeVisible(prevButton);
prevButton.addListener(this);
prevButton.setButtonText("Prev");
prevButton.setLookAndFeel(&customLookAndFeel);

// Initialize and set up the fast forward button
addAndMakeVisible(ffwdButton);
ffwdButton.addListener(this);
ffwdButton.setButtonText("Forward");
ffwdButton.setLookAndFeel(&customLookAndFeel);

```

2. In resized function of DeckGUI

- The layout for the prevButton and ffwdButton is handled in the resized() function. The buttons are aligned with spacing between them, ensuring they fit well into the overall GUI.

```

void DeckGUI::resized()
{
    double rowH = getHeight() / 18;
    double rowW = (getWidth() / 2) - 30;
    int buttonWidth = getWidth() / 6;

    int buttonSpacing = 25.0; // Adjust this value as
    needed for more space

    // Set bounds for each button with adjusted spacing
    prevButton.setBounds(buttonSpacing + buttonWidth,
        rowH * 13, buttonWidth, rowH);
    ffwdButton.setBounds(buttonSpacing + buttonWidth *
        3, rowH * 13, buttonWidth, rowH);
}

```

3. In button clicked function of DeckGUI

- **Fast Forward Button:** Moves to the next track in the playlist. If shuffle mode is enabled, it selects a random track.
- **Previous Button:** Moves to the previous track in the playlist. If shuffle mode is enabled, it selects a random track.

```

        void DeckGUI::buttonClicked(Button* button)
        {
            // Handle fast-forward button click (loads the next track in
            the playlist)
            if (button == &ffwdButton)
            {
                loopButton.setToggleState(false, dontSendNotification);
                if (trackEntries->size() == 0) return;

                player->stop();
                URL audioURL = playlistComponent->getNextTrack(shuffle);
                player->loadURL(audioURL);
                waveFormDisplay.loadURL(audioURL);
                trackName.setText(audioURL.getFileName(),
dontSendNotification);
                player->start();
                playButton.setToggleState(true, dontSendNotification);
            }

            // Handle previous track button click
            if (button == &prevButton)
            {
                loopButton.setToggleState(false, dontSendNotification);
                if (trackEntries->size() == 0) return;

                player->stop();
                URL audioURL = playlistComponent-
>getPreviousTrack(shuffle);
                player->loadURL(audioURL);
                waveFormDisplay.loadURL(audioURL);
                trackName.setText(audioURL.getFileName(),
dontSendNotification);
                player->start();
                playButton.setToggleState(true, dontSendNotification);
            }
        }

```

4. In PlalistComponent

Two functions handle track navigation:

- **getNextTrack(bool shuffle):** Retrieves the next track in the playlist.
- **getPreviousTrack(bool shuffle):** Retrieves the previous track in the playlist.

```

// Function to get the next track in the playlist when
the fast-forward button is clicked in deckGUI
// Returns a random track if shuffle is enabled,
otherwise returns the next track in sequence
URL PlaylistComponent::getNextTrack(bool shuffle)
{
    URL audioURL; // Variable to store the URL of the
next track

    // Check if the trackEntries vector is not empty
    if (trackEntries.size() != 0)

```



```

    {
        // If shuffle is disabled, return the next track
        in sequence
        if (!shuffle)
        {
            // Increment trackCounter if it hasn't
            reached the last track in the list
            if (trackCounter < trackEntries.size() - 1)
            {
                trackCounter++;
            }
            // If the last track is reached, loop back
            to the first track
            else
            {
                trackCounter = 0;
            }
            // Get the URL of the next track
            audioURL =
            trackEntries[trackCounter].audioURL;
        }
        // If shuffle is enabled, return a random track
        else
        {
            /* Random number generation tools */
            std::random_device rand_dev; // Seed the
            random number generator
            std::mt19937 generator(rand_dev()); //
            Mersenne Twister engine for randomness
            std::uniform_int_distribution<int> dist(0,
            int(trackEntries.size() - 1)); // Uniform distribution
            over track entries

            // Generate a random track index
            int r = dist(generator);

            // Ensure the randomly selected track is not
            the same as the currently playing track
            while (r == trackCounter)
            {
                r = dist(generator); // Generate a new
                random number if the same track is selected
            }

            // Get the URL of the randomly selected
            track
            audioURL = trackEntries[r].audioURL;

            // Update trackCounter to keep track of the
            current playing song
            trackCounter = r;
        }
    }

    // Return the URL of the next track (either
    sequential or random)
    return audioURL;
}

// Function to get the previous track in the playlist
when the rewind button is clicked in deckGUI
// Returns a random track if shuffle is enabled,
otherwise returns the previous track in sequence

```

```

URL PlaylistComponent::getPreviousTrack(bool shuffle)
{
    URL audioURL; // Variable to store the URL of the
previous track

    // Check if the trackEntries vector is not empty
    if (trackEntries.size() != 0)
    {
        // If shuffle is disabled, return the previous
track in sequence
        if (!shuffle)
        {
            // Decrement trackCounter if it's greater
than 0 (move to the previous track)
            if (trackCounter > 0)
            {
                trackCounter--;
            }
            // If the first track is reached, loop back
to the last track
            else
            {
                trackCounter = trackEntries.size() - 1;
            }
            // Get the URL of the previous track
            audioURL =
trackEntries[trackCounter].audioURL;
        }
        // If shuffle is enabled, return a random track
        else
        {
            /* Random number generation tools */
            std::random_device rand_dev; // Seed the
random number generator
            std::mt19937 generator(rand_dev()); //
Mersenne Twister engine for randomness
            std::uniform_int_distribution<int> dist(0,
int(trackEntries.size() - 1)); // Uniform distribution
over track entries

            // Generate a random track index
            int r = dist(generator);

            // Ensure the randomly selected track is not
the same as the currently playing track
            while (r == trackCounter)
            {
                r = dist(generator); // Generate a new
random number if the same track is selected
            }

            // Get the URL of the randomly selected
track
            audioURL = trackEntries[r].audioURL;

            // Update trackCounter to keep track of the
current playing song
            trackCounter = r;
        }
    }

    // Return the URL of the previous track (either
sequential or random)
}

```

```
        return audioURL;
    }
```

iii) Rewind button

1. In DeckGUI constructor

- The rewind button (frevButton) is added to the DeckGUI interface. This button is initialized, made visible, and assigned a listener to handle the rewind action when clicked.

```
// Initialize and set up the rewind button
addAndMakeVisible(frevButton);
frevButton.addListener(this);
frevButton.setButtonText("Rev");
frevButton.setLookAndFeel(&customLookAndFeel);
```

2. In resized function of DeckGUI

- In the resized() function, the button's position and size are set. The button is placed alongside the other control buttons, maintaining a consistent layout

```
double rowH = getHeight() / 18;
double rowW = (getWidth() / 2) - 30;
int buttonWidth = getWidth() / 6;

int buttonSpacing = 25.0; // Adjust this value as
needed for more space

frevButton.setBounds(0 + buttonSpacing, rowH * 13,
buttonWidth, rowH);
```

3. In buttonClicked function of DeckGUI

- When the rewind button is clicked, it stops the current playback, resets the track's position to the beginning (setPosition(0)), and starts playback again.

```
// Handle rewind button click
if (button == &frevButton)
{
    player->stop();
    player->setPosition(0);
    player->start();
    playButton.setToggleState(true,
dontSendNotification);
}
```

4. In DJAudioPlayer

- In the DJAudioPlayer class, the setPosition(double posInSecs) function sets the playback position based on the input in seconds. The **Rewind Button** calls this function with a value of 0 to reset the track to the beginning.

```

// setPosition: Sets the playback position in seconds.
// @param posInSecs The position in seconds.
void DJAudioPlayer::setPosition(double posInSecs)
{
    transportSource.setPosition(posInSecs);
}

// setPositionRelative: Sets the playback position
relative to the track length.
// @param pos The relative position, between 0 and 1.
void DJAudioPlayer::setPositionRelative(double pos)
{
    if (pos < 0 || pos > 1.0) {
        std::cout << "DJAudioPlayer::setPositionRelative
pos should be between 0 to 1" << std::endl;
    }
    else {
        double posInSecs = pos *
transportSource.getLengthInSeconds();
        setPosition(posInSecs);
    }
}

```

iv) Loop button

1. In DeckGUI constructor

- The loop button (loopButton) is added to the DeckGUI interface. This button is initialized, made visible, and assigned a listener to handle loop mode toggling when clicked.

```

// Initialize and set up the loop button
addAndMakeVisible(loopButton);
loopButton.addListener(this);
loopButton.setButtonText("Loop");
loopButton.setLookAndFeel(&customLookAndFeel);

```

2. In resized function of DeckGUI

- In the resized() function, the button's position and size are set. The button is placed alongside other control buttons with consistent spacing and layout.

```

double rowH = getHeight() / 18;
double rowW = (getWidth() / 2) - 30;
int buttonWidth = getWidth() / 6;

int buttonSpacing = 25.0; // Adjust this value as
needed for more space

// Set bounds for each button with adjusted spacing
loopButton.setBounds(buttonSpacing + buttonWidth *
4, rowH * 13, buttonWidth, rowH);

```

3. In buttonClicked function of DeckGUI

- When the loop button is clicked, it toggles the loop mode. The button's colour is updated to visually indicate whether looping is enabled (red) or disabled (black).

```
// Handle loop button click
if (button == &loopButton)
{
    isLooping = !isLooping; // Toggle loop mode

    loopButton.setColour(juce::TextButton::textColourOffId,
isLooping ? juce::Colours::red : juce::Colours::black);
}
```

4. In timer callback function

- The timerCallback() function checks if the track has reached its end (i.e., the position is greater than or equal to 1). If looping is enabled, the track's position is reset to the beginning, and playback restarts. Otherwise, playback stops.

```
void DeckGUI::timerCallback()
{
    if (player->getPositionRelative() >= 1)
    {
        if (isLooping)
        {
            player->setPositionRelative(0);
            player->start();
        }
        else if (!isLooping)
        {
            player->setPositionRelative(0);
            player->stop();
        }
    }
}
```

5. In DJAudioPlayer

- In DJAudioPlayer, functions are provided for controlling playback and handling track position.
 - start(): Starts the audio playback.
 - stop(): Stops the audio playback.
 - getPositionRelative(): Returns the current playback position as a value between 0 (start) and 1 (end) relative to the track length.
 - setPositionRelative(double pos): Sets the playback position based on a relative value between 0 and 1.

```

// start: Starts audio playback.
void DJAudioPlayer::start()
{
    transportSource.start();
}

// stop: Stops audio playback.
void DJAudioPlayer::stop()
{
    transportSource.stop();
}

// isPlaying: Checks if the audio is currently playing.
// @return True if playing, false otherwise.
bool DJAudioPlayer::isPlaying()
{
    return transportSource.isPlaying();
}

// getPositionRelative: Returns the current playback
// position relative to the track length.
// @return The relative position, between 0 and 1.
double DJAudioPlayer::getPositionRelative()
{
    double pos = transportSource.getLengthInSeconds() >
0 ? transportSource.getCurrentPosition() /
transportSource.getLengthInSeconds() : 0;

    if (isnan(pos)) {
        return 0;
    }
    return pos;
}

// setPositionRelative: Sets the playback position
// relative to the track length.
// @param pos The relative position, between 0 and 1.
void DJAudioPlayer::setPositionRelative(double pos)
{
    if (pos < 0 || pos > 1.0) {
        std::cout << "DJAudioPlayer::setPositionRelative
pos should be between 0 to 1" << std::endl;
    }
    else {
        double posInSecs = pos *
transportSource.getLengthInSeconds();
        setPosition(posInSecs);
    }
}

```

v) Cue button

1. In DeckGUI constructor

- In the constructor, six buttons are added to handle setting, playing, and clearing two separate cue points (Cue 1 and Cue 2).

```

// Initialize and set up the first cue buttons
addAndMakeVisible(setCueButton1);
setCueButton1.addListener(this);
setCueButton1.setButtonText("Set Cue 1");
setCueButton1.setColour(juce::TextButton::buttonColourId,
    juce::Colours::blue);

addAndMakeVisible(playCueButton1);
playCueButton1.addListener(this);
playCueButton1.setButtonText("Play Cue");
playCueButton1.setColour(juce::TextButton::buttonColourId,
    juce::Colours::magenta);

addAndMakeVisible(clearCueButton1);
clearCueButton1.addListener(this);
clearCueButton1.setButtonText("Clear Cue");
clearCueButton1.setColour(juce::TextButton::buttonColourId,
    juce::Colours::purple);

// Initialize and set up the second cue buttons
addAndMakeVisible(setCueButton2);
setCueButton2.addListener(this);
setCueButton2.setButtonText("Set Cue 2");
setCueButton2.setColour(juce::TextButton::buttonColourId,
    juce::Colours::blue);

addAndMakeVisible(playCueButton2);
playCueButton2.addListener(this);
playCueButton2.setButtonText("Play Cue");
playCueButton2.setColour(juce::TextButton::buttonColourId,
    juce::Colours::magenta);

addAndMakeVisible(clearCueButton2);
clearCueButton2.addListener(this);
clearCueButton2.setButtonText("Clear Cue");
clearCueButton2.setColour(juce::TextButton::buttonColourId,
    juce::Colours::purple);

```

2. In resized function of DeckGUI

- In the resized() function, the buttons are laid out on the GUI. Each cue set has its own row, with three buttons for setting, playing, and clearing the cue.

```

double rowW = (getWidth() / 2) - 30;
// Set bounds for cue buttons
    setCueButton1.setBounds(20, rowH * 5, rowW * 2,
rowH);
    playCueButton1.setBounds(20, rowH * 6, rowW * 2,
rowH);
    clearCueButton1.setBounds(20, rowH * 7, rowW * 2,
rowH);

    setCueButton2.setBounds(20, rowH * 9, rowW * 2,
rowH);
    playCueButton2.setBounds(20, rowH * 10, rowW * 2,
rowH);

```

```
clearCueButton2.setBounds(20, rowH * 11, rowH * 2,  
rowH);
```

3. In button clicked function of DeckGUI

When a cue button is clicked, its corresponding action is performed: setting the cue, playing the cue, or clearing the cue.

Cue 1

- **Set Cue 1:** Sets the cue point at the current track position and changes the button colour to red to indicate that the cue has been set.
- **Play Cue 1:** Plays from the set cue point if it exists.
- **Clear Cue 1:** Clears the cue point and resets the button colour.

Cue 2

- The same logic is applied for the second cue point (Cue 2).

```
// Handle cue setting, playing, and clearing for cue 1  
if (button == &setCueButton1)  
{  
    cueStartPosition1 = player->getPositionRelative();  
  
    setCueButton1.setColour(juce::TextButton::buttonColourId  
    , juce::Colours::red);  
}  
if (button == &playCueButton1)  
{  
    if (cueStartPosition1 >= 0.0 && cueStartPosition1 <=  
    1.0)  
    {  
        player->setPositionRelative(cueStartPosition1);  
        player->start();  
    }  
}  
if (button == &clearCueButton1)  
{  
    cueStartPosition1 = -1;  
  
    setCueButton1.setColour(juce::TextButton::buttonColourId  
    , juce::Colours::blue);  
}  
  
// Handle cue setting, playing, and clearing for cue 2  
if (button == &setCueButton2)  
{  
    cueStartPosition2 = player->getPositionRelative();  
  
    setCueButton2.setColour(juce::TextButton::buttonColourId  
    , juce::Colours::red);  
}  
if (button == &playCueButton2)  
{
```



```

        if (cueStartPosition2 >= 0.0 && cueStartPosition2 <=
1.0)
        {
            player->setPositionRelative(cueStartPosition2);
            player->start();
        }
    }
    if (button == &clearCueButton2)
    {
        cueStartPosition2 = -1;

        setCueButton2.setColour(juce::TextButton::buttonColourId
, juce::Colours::blue);
    }
}

```

4. In DJAudioPlayer

- The DJAudioPlayer class manages playback control, including setting and getting the current position in the track.

```

// start: Starts audio playback.
void DJAudioPlayer::start()
{
    transportSource.start();
}

// stop: Stops audio playback.
void DJAudioPlayer::stop()
{
    transportSource.stop();
}

// isPlaying: Checks if the audio is currently playing.
// @return True if playing, false otherwise.
bool DJAudioPlayer::isPlaying()
{
    return transportSource.isPlaying();
}

// getPositionRelative: Returns the current playback position
relative to the track length.
// @return The relative position, between 0 and 1.
double DJAudioPlayer::getPositionRelative()
{
    double pos = transportSource.getLengthInSeconds() > 0 ?
transportSource.getCurrentPosition() /
transportSource.getLengthInSeconds() : 0;

    if (isnan(pos)) {
        return 0;
    }
    return pos;
}

// setPosition: Sets the playback position in seconds.
// @param posInSecs The position in seconds.
void DJAudioPlayer::setPosition(double posInSecs)
{
    transportSource.setPosition(posInSecs);
}

```

```

// setPositionRelative: Sets the playback position relative to
the track length.
// @param pos The relative position, between 0 and 1.
void DJAudioPlayer::setPositionRelative(double pos)
{
    if (pos < 0 || pos > 1.0) {
        std::cout << "DJAudioPlayer::setPositionRelative pos should be
between 0 to 1" << std::endl;
    }
    else {
        double posInSecs = pos * transportSource.getLengthInSeconds();
        setPosition(posInSecs);
    }
}

```

vi) Crossfader

1. In MiddleDeck constructor

- In the constructor, the crossfader is initialised with a custom look and feel. It has a range between 0.1 and 1.0, where a value of 0.5 represents equal volume for both decks. The crossfaderLabel is also created to indicate the function of the crossfader.

```

MiddleDeck::MiddleDeck(DJAudioPlayer* _player1,
DJAudioPlayer* _player2)
: player1(_player1),
  player2(_player2)
{
    // Initialize the custom LookAndFeel and apply it to
the sliders
    customLookAndFeel =
std::make_unique<CustomLookAndFeel>();

    crossfader.setLookAndFeel(customLookAndFeel.get());

    // Set up slider ranges and initial values
    crossfader.setRange(0.1, 1.0);
    crossfader.setValue(0.5);

    // Set slider styles (no text box, rotary for
volume, linear vertical for speed)
    crossfader.setTextBoxStyle(Slider::NoTextBox, true,
0, 0);

    // Make sliders visible
    addAndMakeVisible(crossfader);

    // Initialize and style labels
    crossfaderLabel.setText("Crossfader",
dontSendNotification);

    //positioning center

    crossfaderLabel.setJustificationType(juce::Justification
::centred);

```

```

//Set font style

crossfaderLabel.setFont(juce::Font(16.0f,
juce::Font::bold));
crossfaderLabel.setColour(juce::Label::textColourId,
juce::Colours::black);

// Make labels visible
addAndMakeVisible(crossfaderLabel);

// Add listeners to sliders to handle value changes
crossfader.addListener(this);

```

2. In MiddleDeck Destructor

- When the MiddleDeck is destroyed, the custom look and feel is removed to avoid memory issues.

```

MiddleDeck::~MiddleDeck()
{
    crossfader.setLookAndFeel(nullptr);
}

```

3. In MiddleDeck resized function

- In the resized() function, the crossfader and its label are positioned within the MiddleDeck component.

```

void MiddleDeck::resized()
{
    int halfWidth = getWidth() / 3;
    int smallHeight = getHeight() / 7;

    // Declare label height and Y offset
    int labelHeight = 20;
    int labelYOffset = 5;

    // Set the bounds for the sliders
    crossfader.setBounds(10, smallHeight * 6 +
labelHeight, getWidth() - 20, smallHeight / 2);

    // Set the bounds for the labels
    crossfaderLabel.setBounds(0, getHeight() -
labelHeight, getWidth(), labelHeight);
}

```

4. In sliderValueChaged function

- Here 'player' from DJAudioPlayer is pointing to setMixRation function

- Whenever the user moves the crossfader, the sliderValueChanged() function is triggered. This adjusts the volume mix ratio of the two decks (player1 and player2).

```
void MiddleDeck::sliderValueChanged(Slider* slider)
{
    if (slider == &crossfader)
    {
        // Adjust the mix ratio for both players based
        on the crossfader's value
        player1->setMixRatio(1.0f - slider->getValue());
        player2->setMixRatio(slider->getValue());
    }
}
```

5. In DJAudioPlayer

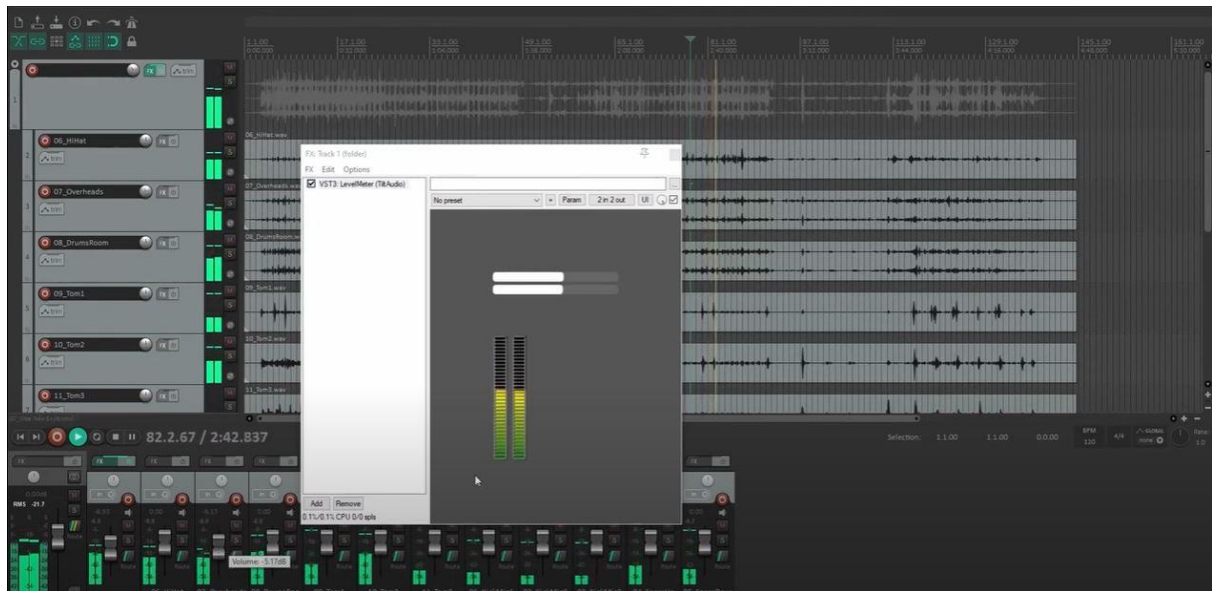
- In the DJAudioPlayer class, the setMixRatio() function adjusts the mix ratio, which in turn calls setGain() to control the deck's volume based on the crossfader value.

```
void DJAudioPlayer::setMixRatio(double ratio)
{
    mixRatio = ratio;
    setGain(mixRatio);
}

//set gain function
void DJAudioPlayer::setGain(double gain)
{
    if (gain < 0.0f) {
        transportSource.setGain(0.0f);
    }
    else if (gain > 1.0f) {
        transportSource.setGain(1.0f);
    }
    else {
        transportSource.setGain(gain);
    }
}
```

R3: New feature inspired by a real DJ program

i) DJ Application – 1: Level Meter for Audio Visualization (Inspired by Reaper DAW)



I have taken The Level Meter idea from this Application found in youtube

Research & Analysis:

- Another common feature in professional DJ and Digital Audio Workstation (DAW) software is the real-time level meter, which visualizes the audio levels during playback. This is vital for monitoring whether the audio levels are clipping or too low. The inspiration for this feature came from Reaper DAW, which showcases audio levels dynamically as the tracks play.
- In Reaper, the level meter is presented for each audio track, giving a real-time view of audio peaks. This feature is not only aesthetically pleasing but also functional, allowing DJs and sound engineers to adjust levels before any audio distortion occurs.

Feature Engineering:

Classes Involved:

- **LevelMeter:** A custom class created to represent the vertical audio level meter.
- **DJAudioPlayer:** This class was extended to calculate the RMS (Root Mean Square) audio level, which provides a basis for visualizing audio amplitude.

OOP Constructs:

- Custom Component Creation: The LevelMeter class inherits from juce::Component and juce::Timer to periodically update the audio level visualization.
- Encapsulation: The RMS level calculation is encapsulated within DJAudioPlayer. The audio player provides the current audio level to the LevelMeter, which renders it as a vertical bar that adjusts in real-time.

Functionality:

- The LevelMeter component calculates the RMS audio level of the currently playing track. The value is updated in real-time as the audio plays.
- The component displays this as a vertical bar, where the height of the bar correlates with the audio level. As the track becomes louder, the bar increases in height, and vice versa.

Code Integration:

```
void LevelMeter::paint(juce::Graphics& g) {
    auto bounds = getLocalBounds().toFloat().reduced(2.0f);
    g.setColour(juce::Colours::black);
    g.fillRect(bounds);
    g.setGradientFill(gradient);

    const auto scaledY = juce::jmap(valueSupplier(), 0.0f, 0.1f, 0.0f,
static_cast<float>(getHeight()));
    g.fillRect(bounds.removeFromBottom(scaledY)); // Adjust meter height based on RMS level
}

float DJAudioPlayer::getRMSLevel() const {
    return rmsLevel; // Return calculated RMS level
}
```

Explanation:

- The LevelMeter receives the current RMS level from the audio player via a valueSupplier function and adjusts the visual representation accordingly.
- The gradient fill gives the meter a visual cue (green to red), where red indicates higher audio levels approaching distortion.

ii) DJ Application – 2: Preview feature



I have taken the idea of Preview present in table component from this Application found in slack channel.

Research & Analysis:

- In professional DJ software such as Serato DJ Pro, there is a highly convenient feature that allows the user to preview tracks directly from the playlist. This is particularly useful for DJs to quickly check the sound of a track without loading it onto the main decks. By providing a preview option in the playlist, the workflow becomes more efficient, enabling DJs to make faster decisions when selecting the next track.
- The implementation of this feature is illustrated in the image that was uploaded.
- The playlist includes a preview button next to each track. DJs can hover over this button or click it to hear a snippet of the track directly from the playlist.

Feature Engineering:

Classes Involved:

- **PlaylistComponent:** The core class that holds the track entries in the playlist and renders the table with the TableListBox component.

- **WaveFormDisplay:** This class is responsible for rendering the waveform of the track and handling preview playback.

OOP Constructs:

- **Inheritance:** PlaylistComponent inherits from JUCE's TableListBoxModel to implement custom components within the table cells, such as the preview button.
- **Custom Component:** A custom preview button was added in the playlist table using the refreshComponentForCell() method. When clicked, the button loads the track into a WaveFormDisplay object, which plays the audio snippet for the preview.
- **Polymorphism:** The WaveFormDisplay object dynamically updates based on the track selected for preview, ensuring that it handles various file formats and audio streams.

Functionality:

- When the preview button is clicked, the selected track's URL is loaded into the WaveFormDisplay.
- The waveform is displayed, and the track starts playing from the start.
- If a track is already being previewed, the new track stops the previous one and plays instead.

Code Integration:

```
Component* PlaylistComponent::refreshComponentForCell(int rowNumber, int columnId,
bool isRowSelected, Component* existingComponentToUpdate) {
    if (columnId == 4) { // Assuming column 4 is for preview
        auto* waveFormDisplay =
dynamic_cast<WaveFormDisplay*>(existingComponentToUpdate);
        if (waveFormDisplay == nullptr) {
            waveFormDisplay = new WaveFormDisplay(formatManager, thumbnailCache);
            existingComponentToUpdate = waveFormDisplay;
        }
        waveFormDisplay->loadURL(trackEntries[rowNumber].audioURL); // Load track URL
        return waveFormDisplay;
    }
    return nullptr;
}
```